



Oracle Database 19c: Advanced PL/SQL

Student Guide - Volume I
D1101121GC10

Author

Brent Dayley

**Technical Contributor
and Reviewer**

Don Bates

Editors

Aju Kumar

Moushmi Mukherjee

Publishers

Sujatha Nagendra

Asief Baig

1011172020

Copyright © 2020, Oracle and/or its affiliates.

Disclaimer

This document contains proprietary information and is protected by copyright and other intellectual property laws. You may copy and print this document solely for your own use in an Oracle training course. The document may not be modified or altered in any way. Except where your use constitutes "fair use" under copyright law, you may not use, share, download, upload, copy, print, display, perform, reproduce, publish, license, post, transmit, or distribute this document in whole or in part without the express authorization of Oracle.

The information contained in this document is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

Restricted Rights Notice

If this documentation is delivered to the United States Government or anyone using the documentation on behalf of the United States Government, the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

Trademark Notice

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

Third-Party Content, Products, and Services Disclaimer

This documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

For Instructor Use Only.
This document should not be distributed.

Contents

1 Introduction

Course Agenda	1-2
Lesson Agenda	1-3
Assumptions	1-4
Course Objectives	1-5
Course Agenda	1-6
Appendixes Used in This Course	1-7
Lesson Agenda	1-8
PL/SQL Development Environments	1-9
Oracle SQL Developer	1-10
Specifications of SQL Developer	1-11
SQL Developer 19.x.x Interface	1-12
Coding PL/SQL in SQL*Plus	1-13
Lesson Agenda	1-14
Tables Used in This Course	1-15
Order Entry Schema	1-16
Human Resources Schema	1-18
Lesson Agenda	1-19
Oracle Database 19c: Focus Areas	1-20
Oracle Database 19c	1-21
Lesson Agenda	1-23
Introduction to Oracle Cloud	1-24
Oracle Cloud Services	1-26
Cloud Deployment Models	1-28
Lesson Agenda	1-30
Oracle SQL and PL/SQL Documentation	1-31
Lesson Agenda	1-32
New Features in 18c and 19c	1-33
Polymorphic Table Functions	1-34
Qualified Expressions	1-36
SQL Macros	1-38
Non-Persistence Support for Object Types	1-40
Summary	1-41

2 Working with Exadata Express Cloud Service

- Course Agenda 2-2
- Lesson Objectives 2-3
- Lesson Agenda 2-4
- Evolving from On-premises to Exadata Express 2-5
- Exadata Express for Users 2-6
- Exadata Express for Developers 2-7
- Getting Started with Exadata Express 2-8
- Managing Exadata 2-10
- Lesson Agenda 2-11
- Service Console 2-12
- Web Access Through Service Console 2-13
- Client Access Configuration Through Service Console 2-14
- Database Administration Through Service Console 2-15
- Lesson Agenda 2-16
- SQL Workshop 2-17
- Lesson Agenda 2-20
- Connecting Through Database Clients 2-21
- Enabling SQL*Net Access for Client Applications 2-22
- Downloading Client Credentials 2-23
- Connecting Oracle SQL Developer 2-24
- Connecting Oracle SQLcl 2-25
- Summary 2-26
- Practice 2: Overview 2-27

3 Overview of Collections

- Course Agenda 3-2
- Objectives 3-3
- Lesson Agenda 3-4
- Collections 3-5
- Why Collections? 3-6
- Collection Types 3-7
- Lesson Agenda 3-9
- Using Associative Arrays 3-10
- Creating an Associative Array 3-11
- Traversing an Associative Array 3-12
- Collection Methods 3-14
- Lesson Agenda 3-15
- Nested Tables 3-16
- Creating Nested Table Types 3-17
- Nested Tables: Example 3-18

Collection Constructors 3-19
Declaring Collections: Nested Table 3-20
Using Nested Tables 3-21
Referencing Collection Elements 3-23
Using Nested Tables in PL/SQL 3-24
Lesson Agenda 3-26
Varrays 3-27
Declaring Collections: Varray 3-29
Using Varrays 3-30
Quiz 3-32
Summary 3-34
Practice 3: Overview 3-35

4 Using Collections

Course Agenda 4-2
Objectives 4-3
Lesson Agenda 4-4
Usage of Collections in Applications 4-5
Working with Collections in PL/SQL 4-6
Assigning Values to Collection Variables 4-9
Accessing Values in the Collection 4-12
Working with Collection Methods 4-13
Using Collection Methods 4-14
Manipulating Individual Elements 4-16
Querying a Collection by Using the TABLE Operator 4-18
Querying a Collection with the TABLE Operator 4-19
Lesson Agenda 4-21
Collection Exceptions 4-22
Collection Exceptions: Example 4-23
Lesson Agenda 4-24
Listing Characteristics for Collections 4-25
Lesson Agenda 4-26
PL/SQL Bind Types 4-27
Subprogram with a BOOLEAN Parameter 4-28
Quiz 4-31
Summary 4-33
Practice 4: Overview 4-34

5 Handling Large Objects

Course Agenda 5-2
Objectives 5-3

Lesson Agenda 5-4
What Is a LOB? 5-5
Types of LOBs 5-6
LOB Locators and LOB Values 5-7
Lesson Agenda 5-8
DBMS_LOB Package 5-9
Security Model of the DBMS_LOB Package 5-10
What Is a DIRECTORY Object? 5-11
Managing BFILES: Role of a DBA 5-12
Managing BFILES: Role of a Developer 5-13
Lesson Agenda 5-14
Working on BFILES 5-15
Preparing to Use BFILES 5-16
Creating BFILE Columns in the Table 5-17
Populating a BFILE Column with PL/SQL 5-18
Using Data in the BFILE Column 5-19
Lesson Agenda 5-20
Working on CLOBs 5-21
Initializing the LOB Columns Added to a Table 5-22
Populating LOB Columns 5-24
Loading Data to a LOB Column 5-25
Writing Data to a LOB 5-26
Lesson Agenda 5-29
Reading LOBs from the Table 5-30
Updating LOB by Using DBMS_LOB in PL/SQL 5-31
Selecting CLOB Values by Using SQL 5-32
Selecting CLOB Values by Using DBMS_LOB 5-33
Selecting CLOB Values in PL/SQL 5-34
Removing LOBs 5-35
Quiz 5-36
Lesson Agenda 5-39
Temporary LOBs 5-40
Creating a Temporary LOB 5-41
Lesson Agenda 5-42
SecureFile LOBs 5-43
Storage of SecureFile LOBs 5-44
Creating a SecureFile LOB 5-45
Quiz 5-46
Summary 5-47
Practice 5: Overview 5-48

6 JSON Data in Database

Course Agenda	6-2
Objectives	6-3
Lesson Agenda	6-4
What Is JSON?	6-5
Structure of JSON Data	6-6
JSON Data: Example	6-7
Why JSON?	6-9
Lesson Agenda	6-10
JSON Data in Oracle Database: Scenario	6-11
JSON Data in Oracle Database	6-12
Creating a Table with JSON Column	6-14
JSON or Not?	6-16
Inserting Data into JSON Columns	6-17
Lesson Agenda	6-18
SQL/JSON Generation Functions	6-19
JSON_OBJECT Function	6-20
JSON_ARRAY Function	6-21
JSON_OBJECTAGG Function	6-22
JSON_ARRAYAGG Function	6-23
SQL/JSON Functions	6-24
Lesson Agenda	6-25
Retrieving SQL Data from JSON Object	6-26
Accessing JSON Data	6-27
JSON_VALUE Function	6-29
Using SQL/JSON Functions	6-30
Lesson Agenda	6-31
PL/SQL Objects for JSON	6-32
JSON Object Types in PL/SQL	6-33
JSON Object Methods	6-34
Getter and Setter Methods	6-35
JSON Methods in PL/SQL: Example	6-36
Summary	6-38
Practice 6: Overview	6-39

7 Advanced Interface Methods

Course Agenda	7-2
Objectives	7-3
Lesson Agenda	7-4
PL/SQL External Procedures	7-5

Oracle Database with Different Languages 7-6
Scenario 7-7
Lesson Agenda 7-8
External Procedure Execution Architecture 7-9
Components for External C Procedure Execution 7-10
Defining an External C Procedure 7-11
Define a C Function 7-12
Creating an Alias Library 7-13
Publishing External C Procedures 7-15
Call Specification Syntax 7-16
Call Specification 7-17
Publishing an External C Routine 7-18
Executing an External C Procedure 7-19
Lesson Agenda 7-20
Executing Java Programs from PL/SQL 7-21
External Procedure Execution Architecture 7-22
Development Steps for Java Class Methods 7-23
Loading Java Class Methods 7-24
Publishing a Java Class Method 7-25
Executing the Java Routine 7-27
Creating Call Specifications in Packages 7-28
Quiz 7-29
Summary 7-32
Practice 7: Overview 7-33

8 Performance and Tuning

Course Agenda 8-2
Objectives 8-3
Lesson Agenda 8-4
Compiling a PL/SQL Unit 8-5
Deciding on a Compilation Method 8-6
Configuring the Compiler 8-7
Viewing the Compilation Settings 8-9
Setting Up a Database for Native Compilation 8-11
Modifying Compilation Mode of a Program Unit 8-12
Lesson Agenda 8-13
PL/SQL Optimizer 8-14
Subprogram Inlining: Introduction 8-15
Using Inlining 8-16
Inlining Concepts 8-17
Inlining: How to Enable It? 8-20

PRAGMA INLINE Example 8-21
Inlining: Summary 8-22
Lesson Agenda 8-23
Why PL/SQL Tuning? 8-24
Tuning PL/SQL Code 8-25
Avoid Implicit Data Type Conversion 8-26
NOT NULL Constraint 8-27
PLS_INTEGER Data Type for Integers 8-28
Using the SIMPLE_INTEGER Data Type 8-29
Modularizing Your Code 8-30
Bulk Binding 8-31
FORALL Instead of FOR 8-32
BULK COLLECT 8-34
Exception While Bulk Collecting 8-37
Handling FORALL Exceptions 8-38
Tuning Conditional Control Statements 8-39
Passing Data Between PL/SQL Programs 8-41
Quiz 8-44
Summary 8-47
Practice 8: Overview 8-48

9 Improving Performance with Caching

Course Agenda 9-2
Objectives 9-3
Lesson Agenda 9-4
What Is Caching? 9-5
Memory Architecture 9-6
Caching in the Database Instance 9-7
What Is Result Caching? 9-8
Lesson Agenda 9-9
Configuring the Server Result Cache 9-10
Setting Result_Cache_Max_Size 9-12
Setting the Result Cache Mode 9-13
Using the DBMS_RESULT_CACHE Package 9-14
Lesson Agenda 9-15
SQL Query Result Cache 9-16
Examining the Memory Cache 9-18
Examining the Execution Plan for a Query 9-19
Examining Another Execution Plan 9-20
Executing Both Queries 9-21
Viewing Cache Results Created 9-22

Viewing Cache Results Found 9-23
Lesson Agenda 9-24
PL/SQL Function Result Cache 9-25
Marking PL/SQL Function Results to Be Cached 9-26
Clearing the Shared Pool and Result Cache 9-27
Creating a PL/SQL Function by Using the RESULT_CACHE Clause 9-28
Calling the PL/SQL Function Inside a Query 9-29
Viewing Cache Results Created 9-30
Calling the PL/SQL Function Again 9-31
Viewing Cache Results Found 9-32
Confirming That the Cached Result Was Used 9-33
Lesson Agenda 9-34
Oracle Database In-Memory 9-35
Quiz 9-36
Summary 9-41
Practice 9: Overview 9-42

10 Analyzing PL/SQL Code

Course Agenda 10-2
Objectives 10-3
Lesson Agenda 10-4
PL/SQL Code Analysis 10-5
Data Dictionary Views 10-7
Analyzing PL/SQL Code 10-8
Using SQL Developer for Code Analysis 10-13
Using ALL_ARGUMENTS 10-15
ALL_ARGUMENTS 10-18
Using SQL Developer to Report on Arguments 10-19
Lesson Agenda 10-21
PL/Scope 10-22
Using PL/Scope 10-24
USER_IDENTIFIERS View 10-25
Sample Data for PL/Scope 10-26
Collecting Information on Identifiers 10-27
Viewing Identifier Information 10-28
Performing a Basic Identifier Search 10-30
Using USER_IDENTIFIERS to Find All Local Variables 10-31
Finding Identifier Actions 10-32
Lesson Agenda 10-34
Oracle Supplied Packages for Code Analysis 10-35
Using DBMS_DESCRIBE 10-36

- DBMS_UTILITY Package 10-39
- Using DBMS_UTILITY.FORMAT_CALL_STACK 10-40
- Using DBMS_UTILITY 10-42
- Finding Error Information 10-44
- DBMS_METADATA Package 10-47
- DBMS_METADATA Subprograms 10-48
- FETCH_xxx Subprograms 10-50
- Filters on Metadata 10-51
- SET_FILTER Procedure 10-52
- Examples of Setting Filters 10-53
- Programmatic Use: Example 1 10-54
- Programmatic Use: Example 2 10-56
- Browsing APIs 10-58
- Using the UTL_CALL_STACK Package 10-59
- DEPRECATE Pragma 10-60
- Quiz 10-61
- Summary 10-64
- Practice 10: Overview 10-65

11 Profiling and Tracing PL/SQL Code

- Course Agenda 11-2
- Objectives 11-3
- Lesson Agenda 11-4
- Tracing PL/SQL Execution 11-5
- Tracing PL/SQL: Steps 11-9
 - Step 1: Enable Specific Subprograms 11-10
 - Steps 2 and 3: Identify a Trace Level and Start Tracing 11-11
 - Step 4 and Step 5: Turn Off and Examine the Trace Data 11-12
- plsql_trace_runs and plsql_trace_events 11-13
- Lesson Agenda 11-15
- Profiling PL/SQL Code 11-16
 - Hierarchical Profiling 11-17
 - Hierarchical Profiling Concepts 11-18
 - Using the PL/SQL Profiler 11-19
 - Understanding Raw Profiler Data 11-24
 - Using the Hierarchical Profiler Tables 11-25
 - Using DBMS_HPROF.ANALYZE 11-26
 - Using DBMS_HPROF.ANALYZE to Write to Hierarchical Profiler Tables 11-27
 - Analyzer Output from the DBMSHP_RUNS Table 11-28
 - Analyzer Output from the DBMSHP_FUNCTION_INFO Table 11-29
 - plshprof: A Simple HTML Report Generator 11-30

Using plshprof 11-31
Using the HTML Reports 11-34
Quiz 11-37
Summary 11-40
Practice 11: Overview 11-41

12 Securing Applications through PL/SQL

Course Agenda 12-2
Objectives 12-3
Lesson Agenda 12-4
Invoker's Rights and Definer's Rights 12-5
Why Invoker's Rights? 12-6
AUTHID clause 12-7
Lesson Agenda 12-8
White Lists 12-9
ACCESSIBLE BY Clause 12-10
Using ACCESSIBLE BY Clause in Packages 12-11
Lesson Agenda 12-12
What Is an Application Security Policy? 12-13
Implementing Application Security Policy 12-14
DBMS_RLS package 12-15
Defining a Policy 12-16
Defining a Policy Function 12-17
Defining a Policy 12-18
Lesson Agenda 12-19
Application Context - Concept 12-20
Application Context - Implementation 12-21
USERENV Application Context 12-22
Creating an Application Context 12-23
Setting a Context 12-24
Lesson Agenda 12-26
Virtual Private Database 12-27
Implementing a Virtual Private Database 12-29
Setting Up a Context 12-30
Creating the Package 12-31
Define the Security Policy 12-33
Setting Up the Logon Trigger 12-34
Policy in Action 12-35
Data Dictionary Views 12-36
Using the ALL_CONTEXT Dictionary View 12-37
Policy Groups 12-39

Quiz 12-40
Summary 12-43
Practice 12: Overview 12-44

13 Safeguarding Your Code against SQL Injection Attacks

Course Agenda 13-2
Objectives 13-3
Lesson Agenda 13-4
SQL Injection 13-5
SQL Injection: Example 13-6
Scenario 13-7
Types of SQL Injection 13-8
Avoidance Strategies against SQL Injection 13-9
Protecting against SQL Injection: Example 13-10
Lesson Agenda 13-11
Reducing the Attack Surface 13-12
Expose the Database Only Via PL/SQL API 13-14
Using Invoker's Rights 13-15
Strengthen Database Security 13-17
Lesson Agenda 13-18
Using Static SQL 13-19
Using Dynamic SQL 13-22
Lesson Agenda 13-23
Using Bind Arguments with Dynamic SQL 13-24
Using Bind Arguments with Dynamic PL/SQL 13-25
What If You Cannot Use Bind Arguments? 13-26
Lesson Agenda 13-27
DBMS_ASSERT Package 13-28
Understanding DBMS_ASSERT 13-29
Oracle Identifiers 13-30
Working with Identifiers in Dynamic SQL 13-32
Choosing a Verification Route 13-33
Validate Input Using DBMS_ASSERT 13-34
Avoiding Injection by Using DBMS_ASSERT.SIMPLE_SQL_NAME 13-35
DBMS_ASSERT Guidelines 13-37
Quiz 13-41
Summary 13-45
Practice 13: Overview 13-46

14 Advanced Security Mechanisms

Course Agenda 14-2

Objectives 14-3
Lesson Agenda 14-4
Real Application Security 14-5
How It Works Without RAS? 14-6
How It Works with RAS? 14-7
Real Application Security - Components 14-8
Implementing a RAS Data Security Policy 14-9
Application Sessions in RAS 14-10
RAS Sessions 14-11
Lesson Agenda 14-12
Transparent Data Encryption 14-13
Encrypting a Table Column Using TDE 14-14
Encrypting a Tablespace Using TDE 14-15
Keystores in TDE 14-16
Lesson Agenda 14-17
Oracle Data Redaction 14-18
Data Redaction Methods 14-19
Benefits of Data Redaction 14-20
Summary 14-21

A Table Descriptions and Data

B Using SQL Developer

Objectives B-2
What Is Oracle SQL Developer? B-3
Specifications of SQL Developer B-4
SQL Developer Interface B-5
Creating a Database Connection B-7
Browsing Database Objects B-10
Displaying the Table Structure B-11
Browsing Files B-12
Finding Database Objects B-13
Creating a Schema Object B-15
Creating a New Table: Example B-16
Using the SQL Worksheet B-17
Executing SQL Statements B-20
Saving SQL Scripts B-21
Executing Saved Script Files: Method 1 B-22
Executing Saved Script Files: Method 2 B-23
Formatting the SQL Code B-24
Using Snippets B-25

Using Snippets: Example B-26
Using Recycle Bin B-27
Debugging Procedures and Functions B-28
Database Reporting B-29
Creating a User-Defined Report B-30
Search Engines and External Tools B-31
Setting Preferences B-32
Resetting the SQL Developer Layout B-33
Data Modeler in SQL Developer B-34
Summary B-35

C Using SQL*Plus

Objectives C-2
SQL and SQL*Plus Interaction C-3
SQL Statements Versus SQL*Plus Commands C-4
SQL Versus SQL*Plus C-5
Using SQL*Plus C-6
SQL Plus Commands: Categories C-7
Logging In to SQL*Plus C-8
Displaying the Table Structure C-9
SQL*Plus Editing Commands C-11
Using LIST, n, and APPEND C-13
Using the CHANGE Command C-14
SQL*Plus File Commands C-15
Using the SAVE and START Commands C-16
SERVEROUTPUT Command C-17
Using the SQL*Plus SPOOL Command C-18
Using the AUTOTRACE Command C-19
Summary C-20

D PL/SQL Programming Concepts: Review

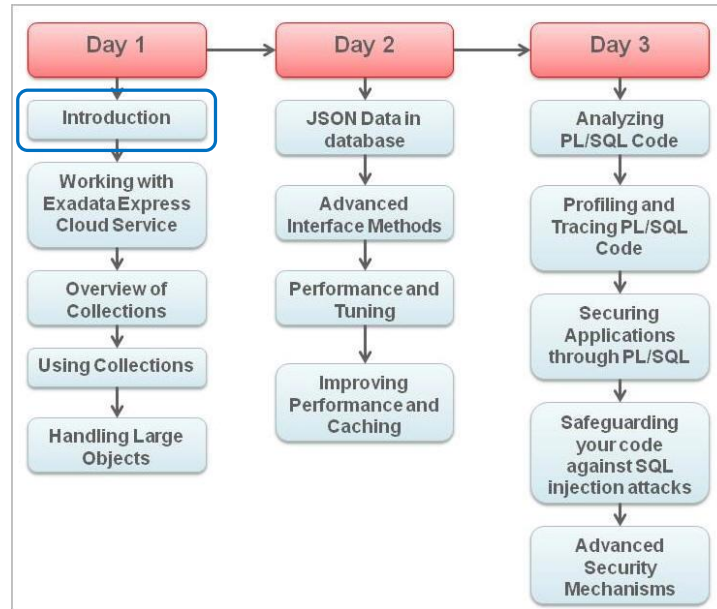
Objectives D-2
Lesson Agenda D-3
PL/SQL Block Structure D-4
Naming Conventions D-5
Procedures D-7
Procedure: Example D-8
Stored Functions D-9
Functions: Example D-10
Ways to Execute Functions D-11
Lesson Agenda D-12

Restrictions on Calling Functions from SQL Expressions D-13
Lesson Agenda D-15
PL/SQL Packages: Review D-16
Components of a PL/SQL Package D-17
Creating the Package Specification D-18
Creating the Package Body D-19
Lesson Agenda D-20
Cursor D-21
Processing Explicit Cursors D-23
Explicit Cursor Attributes D-24
Cursor FOR Loops D-25
Cursor: Example D-26
Lesson Agenda D-27
Handling Exceptions D-28
Exceptions: Example D-30
Predefined Oracle Server Errors D-31
Predefined Oracle Server Exceptions D-32
Trapping Non-Predefined Oracle Server Errors D-34
Trapping User-Defined Exceptions D-36
Lesson Agenda D-37
RAISE_APPLICATION_ERROR Procedure D-38
Lesson Agenda D-40
Dependencies D-41
Displaying Direct and Indirect Dependencies D-43
Lesson Agenda D-44
Using Oracle-Supplied Packages D-45
Some of the Oracle-Supplied Packages D-46
DBMS_OUTPUT Package D-47
UTL_FILE Package D-48
Summary D-49

Introduction

ORACLE

Course Agenda



ORACLE

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Previewing the course agenda
- Describing the development environments
- Identifying the tables and schemas used in this course
- Overview of Oracle Database 19c and related products
- Oracle Cloud
- Oracle documentation and additional resources
- New Features



ORACLE

For Instructor Use Only.
This document should not be distributed.

Assumptions

- You have good knowledge of SQL and can write complex SQL queries.
- You have good knowledge of writing PL/SQL blocks of code.
- You understand terms such as joins and nested queries.
- You have a clear understanding of PL/SQL program constructs – loops, conditional statements, and so on.
- You understand when there are references to terms such as cursors, procedures, functions, and triggers.

ORACLE

This is an advanced course, which assumes that you have good knowledge of SQL and PL/SQL concepts.

In this course, you can see references to terms such as cursors, procedures, joins, and indexes. It is assumed that you can write complex SQL queries by using joins and nested queries. You should know how to write PL/SQL blocks, both anonymous and named. It is assumed that you can write PL/SQL blocks using various PL/SQL constructs, such as cursors, loops, and so on.

For Instructor Use Only.
This document should not be distributed.

Course Objectives

After completing this course, you will be able to:

- Efficiently design PL/SQL packages and program units
- Write code to interface with external applications
- Create PL/SQL applications that can manipulate complex data
- Write and tune PL/SQL code effectively to maximize performance
- Identify the security requirements of applications
- Manage PL/SQL code effectively



ORACLE

In this course, you will learn to handle complex data. You learn about collections, which enable you to work with a logical group of homogenous data. You will also learn to manage large objects and JSON data in the database.

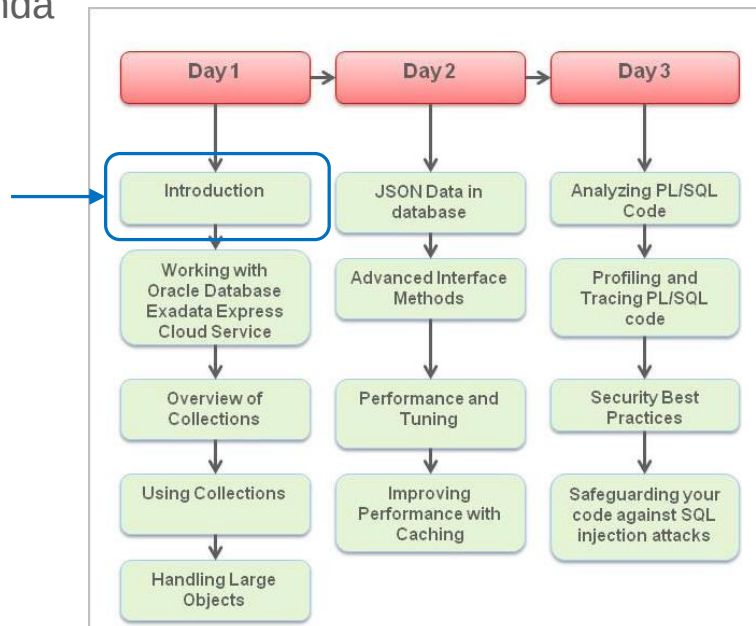
Apart from handling complex data, you will learn how to interface database with other programming languages such as C and Java.

You will be introduced to utilities such as PL/Scope and hierarchical profiler to analyze, trace, and profile PL/SQL code. These functions help you in improving the performance of the application.

You will be introduced to various security mechanisms that can be used to secure applications.

For Instructor Use Only.
This document should not be distributed.

Course Agenda



ORACLE

This course is completed over a duration of three days.

On day one, you will be introduced to the Oracle Database 19c new features, Exadata Express Cloud Service. You will be introduced to the concept of Collections and you will learn how to use collections and large objects in a PL/SQL block.

On day two, you will dive deeper into the course. You will learn to handle JSON data in Oracle database. You will also learn interfacing PL/SQL with Java and C languages. Towards the end of the day you will learn about Performance tuning and caching techniques to improve application performance.

On day three, you will learn about analyzing, tracing, and profiling PL/SQL code. These functions enable you to understand and tune application performance. You will be introduced to various security mechanisms available with Oracle database and discuss Virtual Private Database in detail. You will learn what SQL injection is and how to safeguard applications against SQL injection attacks.

For Instructor Use Only.
This document should not be distributed.

Appendixes Used in This Course

- Appendix A: Table Descriptions and Data
- Appendix B: Using SQL Developer
- Appendix C: Using SQL*Plus

ORACLE

There are three appendixes in the course:

- Appendix A discusses about the data in the tables of the schemas used in the course.
- Appendix B discusses about SQL Developer. All practices in the course are demonstrated through SQL Developer. You execute the practices through SQL Developer.
- Appendix C introduces you to the command-line tool SQL*Plus.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Previewing the course agenda
- **Describing the development environments**
- Identifying the tables and schemas used in this course
- Overview of Oracle Database 19c and related products
- Oracle Cloud
- Oracle documentation and additional resources
- New Features



ORACLE

For Instructor Use Only.
This document should not be distributed.

PL/SQL Development Environments

This course setup provides the following tools for developing PL/SQL code:

- Oracle SQL Developer (used in this course)
- Oracle SQL*Plus

The Oracle logo, consisting of the word "ORACLE" in white capital letters on a red rectangular background.

There are many tools that provide an environment for developing PL/SQL code. Some of the development tools that are available for use in this course are:

- **Oracle SQL Developer:** A graphical tool
- **Oracle SQL*Plus:** A window or command-line application

Note: The code and screen examples presented in the course notes were generated from output in the SQL Developer environment.

For Instructor Use Only.
This document should not be distributed.

Oracle SQL Developer

- Oracle SQL Developer is a free graphical tool that enhances productivity and simplifies database development tasks.
- You can connect to any target Oracle database schema by using standard Oracle database authentication.
- You use SQL Developer in this course.



SQL Developer

ORACLE

What Is Oracle SQL Developer?

Oracle SQL Developer is a free graphical tool that is designed to improve your productivity and simplify the development of everyday database tasks. The user interface of SQL Developer allows you to browse, create, and manage database objects.

Using SQL Developer, you can easily create and maintain stored procedures, test SQL statements, and view optimizer plans, with just a few clicks.

SQL Developer, the visual tool for database development, simplifies the following tasks:

- Browsing and managing database objects
- Executing SQL statements and scripts
- Editing and debugging PL/SQL statements
- Creating reports

You can connect to any target Oracle database schema by using standard Oracle database authentication. When you are connected, you can perform operations on the objects in the database.

Note: The code and screen examples presented in the course notes were generated from the output in the SQL Developer environment.

For Instructor Use Only.
This document should not be distributed.

Specifications of SQL Developer

- Is developed in Java
- Supports Windows, Linux, and Mac OS X platforms
- Enables default connectivity by using the JDBC Thin driver
- Connects to Oracle Database version 9.2.0.1 and later
- Connects to Oracle Database on Cloud also

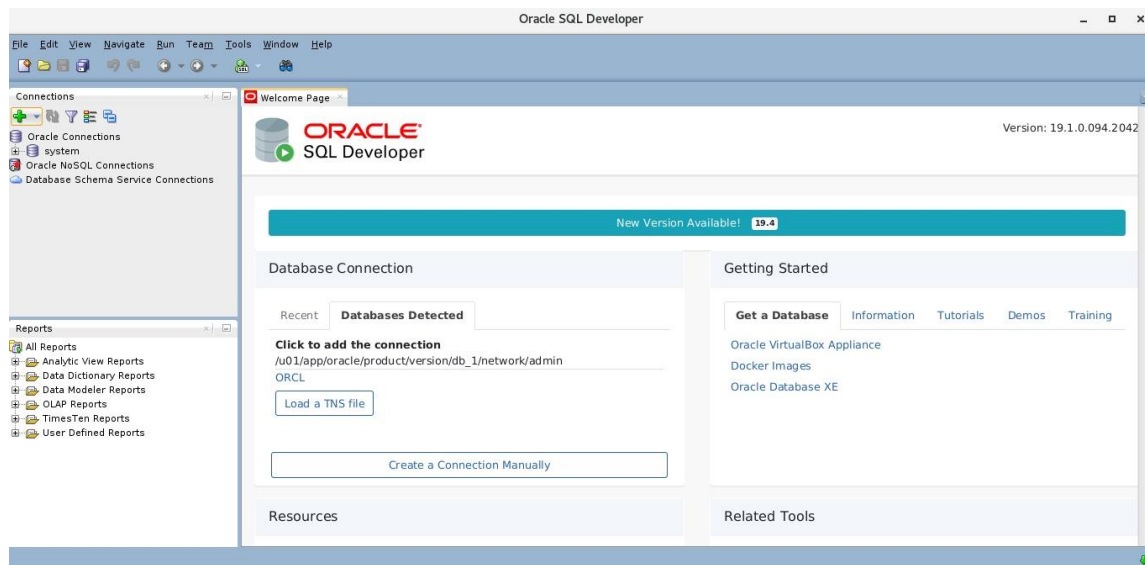
ORACLE

Oracle SQL Developer is developed in Java, leveraging the Oracle JDeveloper integrated development environment (IDE). Therefore, it is a cross-platform tool. The tool runs on the Windows, Linux, and Mac operating system (OS) X platforms.

Default connectivity to the database is through the Java Database Connectivity (JDBC) Thin driver, and therefore, no Oracle Home is required. SQL Developer does not require an installer and you need to simply unzip the downloaded file. With SQL Developer, users can connect to Oracle Databases 9.2.0.1 and later, and all Oracle database editions, including Express Edition. You can connect to Oracle Database Service on cloud also through SQL Developer.

For Instructor Use Only.
This document should not be distributed.

SQL Developer 19.x.x Interface



ORACLE

The SQL Developer interface contains three main navigation tabs, from left to right:

- **Connections tab:** By using this tab, you can browse database objects and users to which you have access.
- **Reports tab:** Identified by the Reports icon, this tab enables you to run predefined reports or create and add your own reports.
- **Start page:** The Start page gives you some links which are helpful while using SQL Developer to create applications.

General Navigation and Use

SQL Developer uses the left pane for navigation to find and select objects. You can customize many aspects of the appearance and behavior of SQL Developer by setting preferences.

Note: You need to define at least one connection to be able to connect to a database schema and issue SQL queries or run procedures and functions. You can start the connection creation wizard by clicking the plus sign (+) on the Connections tab.

For Instructor Use Only.
This document should not be distributed.

Coding PL/SQL in SQL*Plus



```
File Edit View Search Terminal Help
[oracle@localhost ~]$ sqlplus

SQL*Plus: Release 19.0.0.0.0 - Production on Mon Oct 19 02:58:52 2020
Version 19.3.0.0.0

Copyright (c) 1982, 2019, Oracle. All rights reserved.

Enter user-name: system
Enter password:
Last Successful login time: Fri May 31 2019 16:29:02 -04:00

Connected to:
Oracle Database 19c Enterprise Edition Release 19.0.0.0.0 - Production
Version 19.3.0.0.0

SQL> set serveroutput on
SQL> create or replace procedure hello is
2 begin
3 dbms_output.put_line('Hello Class!');
4 end;
5 /

Procedure created.

SQL> execute hello
Hello Class!

PL/SQL procedure successfully completed.

SQL> █
```

ORACLE

Oracle SQL*Plus is a command-line interface that enables you to submit SQL statements and PL/SQL blocks for execution and receive results in an application or a command window.

SQL*Plus is:

- Shipped with the database
- Installed on a client and on the database server system
- Accessed by using an icon or the command line

When you code PL/SQL subprograms by using SQL*Plus, remember the following:

- You create subprograms by using the **CREATE SQL** statement.
- You execute subprograms by using either an anonymous PL/SQL block or the **EXECUTE** command.
- If you use the **DBMS_OUTPUT** package procedures to print text to the screen, you must first execute the **SET SERVEROUTPUT ON** command in your session.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Previewing the course agenda
- Describing the development environments
- Identifying the tables and schemas used in this course
- Overview of Oracle Database 19c and related products
- Oracle Cloud
- Oracle documentation and additional resources
- New Features



ORACLE

For Instructor Use Only.
This document should not be distributed.

Tables Used in This Course

- The sample schemas used are:
 - Order Entry (OE) schema
 - Human Resources (HR) schema
- Primarily, the OE schema is used.
- The OE schema user can read data in the HR schema tables.
- Appendix A contains more information about the sample schemas.

ORACLE

The sample company portrayed by Oracle Database Sample Schemas operates worldwide to fulfill orders for several different products. The company has several divisions:

- The Human Resources division tracks information about the employees and the facilities of the company.
- The Order Entry division tracks product inventories and sales of the company's products through various channels.
- The Sales History division tracks business statistics to facilitate business decisions. Although not used in this course, the SH schema is part of the "Example" sample schemas shipped with the database.

Each of these divisions is represented by a schema.

This course primarily uses the Order Entry (OE) sample schema.

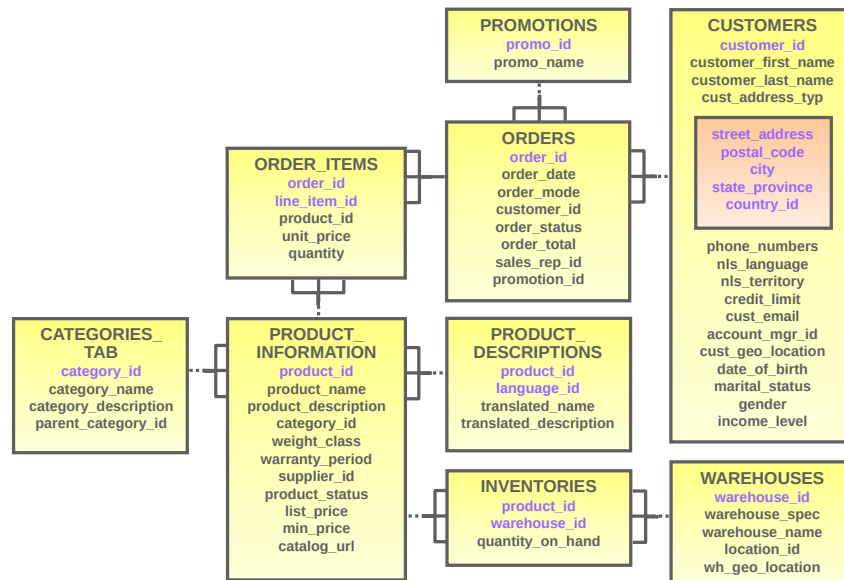
Note: More details about the sample schema are found in Appendix A.

You have the HR schema installed by default. You can install the OE schema and SH schema from Github. You can follow the instructions in the database documentation:

- <https://docs.oracle.com/en/database/oracle/oracle-database/19/comsc/toc.htm>

For Instructor Use Only.
This document should not be distributed.

Order Entry Schema



ORACLE

The company sells several categories of products, including computer hardware and software, music, clothing, and tools. The company maintains product information that includes product identification numbers, the category into which the product falls, the weight group (for shipping purposes), the warranty period if applicable, the supplier, the status of the product, a list price, a minimum price at which a product will be sold, and a URL address for manufacturer information.

Inventory information is also recorded for all products, including the warehouse where the product is available and the quantity on hand. Because products are sold worldwide, the company maintains the names of the products and their descriptions in different languages.

The company maintains warehouses in several locations to facilitate filling customer orders. Each warehouse has a warehouse identification number, name, and location identification number.

For Instructor Use Only.
This document should not be distributed.

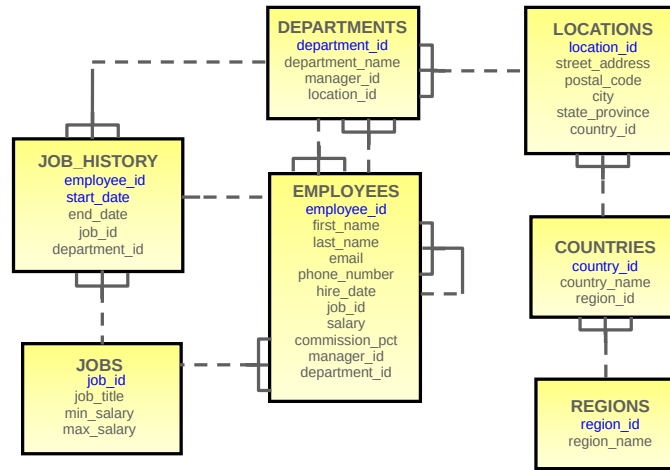
Customer information is tracked in some detail. Each customer is assigned an identification number. Customer records include name, street address, city or province, country, phone numbers (up to five phone numbers for each customer), and postal code. Some customers order through the Internet, so email addresses are also recorded. Because of language differences among customers, the company records the NLS language and territory of each customer. The company places a credit limit on its customers to limit the amount for which they can purchase at one time. Some customers have account managers, whom the company monitors. It keeps track of a customer's phone number. At present, you do not know how many phone numbers a customer might have, but you try to keep track of all of them. Because of the language differences among customers, you also identify the language and territory of each customer.

When a customer places an order, the company tracks the date of the order, the mode of the order, status, shipping mode, total amount of the order, and the sales representative who helped place the order. This may be the same individual as the account manager for a customer, it may be someone else, or, in the case of an order over the Internet, the sales representative is not recorded. In addition to the order information, the company also tracks the number of items ordered, the unit price, and the products ordered.

For each country in which it does business, the company records the country name, currency symbol, currency name, and the region where the country resides geographically. This data is useful to interact with customers who are living in different geographic regions of the world.

For Instructor Use Only.
This document should not be distributed.

Human Resources Schema



ORACLE

In the human resources records, each employee has an identification number, email address, job identification code, salary, and manager. Some employees earn a commission in addition to their salary.

The company also tracks information about the jobs within the organization. Each job has an identification code, job title, and a minimum and maximum salary range for the job. Some employees have been with the company for a long time and have held different positions within the company. When an employee switches jobs, the company records the start date and end date of the former job, the job identification number, and the department.

The sample company is regionally diverse, so it tracks the locations of not only its warehouses but also its departments. Each company employee is assigned to a department. Each department is identified by a unique department number and a short name. Each department is associated with one location. Each location has a full address that includes the street address, postal code, city, state or province, and country code.

For each location where it has facilities, the company records the country name, currency symbol, currency name, and the region where the country resides geographically.

Note: For more information about the “Example” sample schemas, refer to Appendix A.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

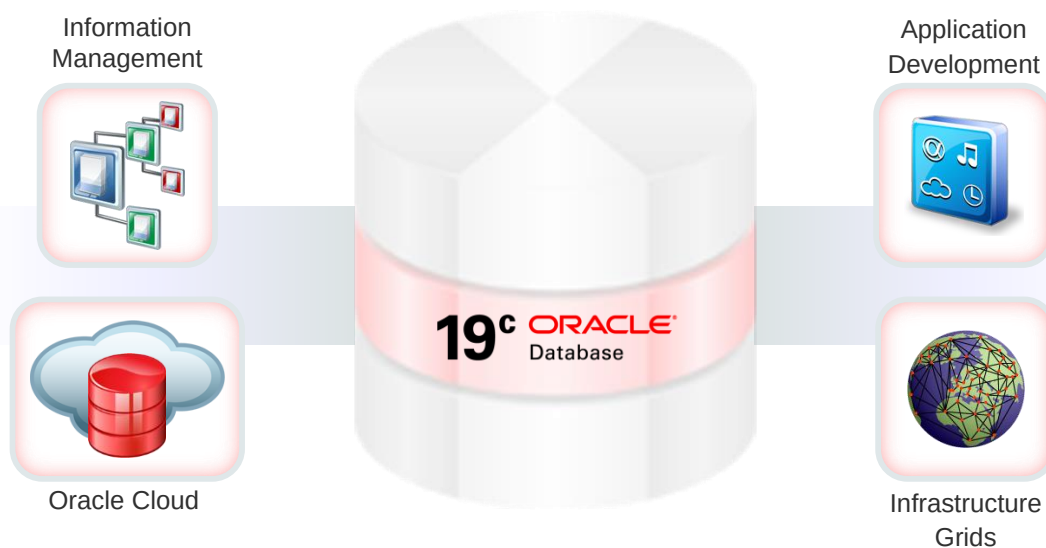
- Previewing the course agenda
- Describing the development environments
- Identifying the tables and schemas used in this course
- **Overview of Oracle Database 19c and related products**
- Oracle Cloud
- Oracle documentation and additional resources
- New Features



ORACLE

For Instructor Use Only.
This document should not be distributed.

Oracle Database 19c: Focus Areas



ORACLE

By using Oracle Database 19c, you can utilize the following features across focus areas:

- With the **Infrastructure Grid** technology of Oracle, you can pool low-cost servers and storage to form systems that deliver the highest quality of service in terms of manageability, high availability, and performance. Oracle Database 19c also helps you to consolidate and extend the benefits of grid computing and manage changes in a controlled and cost-effective manner.
- Oracle Database 19c enables **Information Management** by providing capabilities in content management, information integration, and information lifecycle management areas. You can manage content of advanced data types such as Extensible Markup Language (XML), text, spatial, multimedia, medical imaging, and semantic technologies using the features provided by Oracle.
- With Oracle Database 19c, you can manage all the major **Application Development** environments such as PL/SQL, Java/JDBC, .NET, Windows, PHP, SQL Developer, and Application Express.
- You can now plug into **Oracle Cloud** with Oracle Database 19c. This will help you to standardize, consolidate, and automate database services on the cloud.

For Instructor Use Only.
This document should not be distributed.

Oracle Database 19c



Imagine you have an organization that needs to support multiple terabytes of information for users who demand fast and secure access to business applications round the clock. The database systems must be reliable and must be able to recover quickly in the event of any kind of failure. Oracle Database 19c is designed to help organizations manage infrastructure grids easily and deliver high-quality service:

- **High availability:** By using the high availability features, you can reduce the risk of down time and data loss. These features improve online operations and enable faster database upgrades.
- **Manageability:** By using some of the change assurance, management automation, and fault diagnostics features, the database administrators (DBAs) can increase their productivity, reduce costs, minimize errors, and maximize quality of service. Some of the useful features that promote better management are the Database Replay facility, the SQL Performance Analyzer, the Automatic SQL Tuning facility, and Real-Time Database Operations Monitoring.

Oracle Enterprise Manager Express for 19c is a web-based tool for managing Oracle databases. It greatly simplifies database performance diagnostics by consolidating the relevant database performance screens into a view called Database Performance Hub. DBAs get a single, consolidated view of the current real-time and historical view of the database performance across multiple dimensions such as database load, monitored SQL and PL/SQL, and Active Session History (ASH) on a single page for the selected time period.

- **Performance:** By using capabilities such as SecureFiles, Result Caches, and so on, you can greatly improve the performance of your database. Oracle Database 19c enables organizations to manage large, scalable, transactional, and data warehousing systems that deliver fast data access using low-cost modular storage.
- **Security:** Oracle Database 19c helps in protecting your information with unique secure configurations, data encryption and masking, and sophisticated auditing capabilities.
- **Information integration:** You can utilize Oracle Database 19c features to integrate data throughout the enterprise in a better way. You can also manage the changing data in your database by using Oracle Database 19c's advanced information lifecycle management capabilities.

Lesson Agenda

- Previewing the course agenda
- Describing the development environments
- Identifying the tables and schemas used in this course
- Overview of Oracle Database 19c and related products
- **Oracle Cloud**
- Oracle documentation and additional resources
- New Features



ORACLE

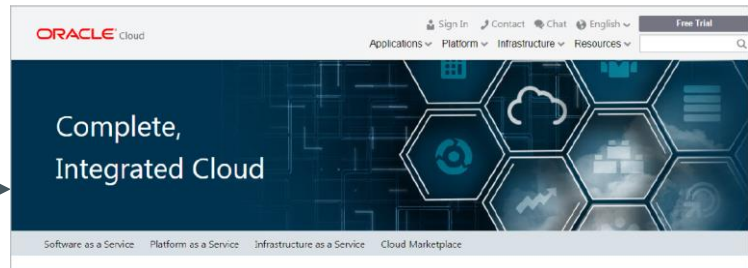
For Instructor Use Only.
This document should not be distributed.

Introduction to Oracle Cloud

Oracle Cloud is an enterprise cloud for business. Oracle Public Cloud consists of many different services that share some common characteristics:

- On-demand self-service
- Resource pooling
- Rapid elasticity
- Measured service
- Broad network access

cloud.oracle.com



ORACLE

Oracle Cloud is an enterprise cloud for business. It provides an integrated collection of application and platform cloud services that are based on best-in-class products and open Java and SQL standards.

Why Oracle Cloud?

The applications and databases that are deployed in Oracle Cloud are portable and can easily migrate from a private cloud or an on-premise (local machine) environment.

You can use the self-service interface to provision the cloud services and resources. These Cloud Services can be delivered onto Integrated Development and Deployment that enables users to quickly extend and create new services.

Oracle Cloud services are built on the Oracle Exalogic Elastic Cloud and Oracle Exadata Database Machine, which together offer a platform that delivers extreme performance, redundancy, and scalability. The top two benefits of cloud computing are speed and cost.

For Instructor Use Only.
This document should not be distributed.

The five essential characteristics are:

- **On-demand self-service:** Provisioning, monitoring, management control
- **Resource pooling:** Sharing and a level of abstraction between consumers and services
- **Rapid elasticity:** The ability to quickly scale up or down as needed
- **Measured service:** Metering utilization for either internal chargeback (private cloud) or external billing (public cloud)
- **Broad network access:** Typically, access through a browser on any networked device

For Instructor Use Only.
This document should not be distributed.

Oracle Cloud Services

Oracle Cloud provides the following three types of services:

- Software as a Service (SaaS)
- Platform as a Service (PaaS)
- Infrastructure as a Service (IaaS)



ORACLE

- **What Is Software as Service (SaaS)?**

SaaS refers to applications that are delivered to end users over the Internet. Oracle CRM On Demand is an example of a SaaS offering that provides both multitenant as well as single-tenant options, depending on the customer's preferences.

- **What Is Platform as a Service (PaaS)?**

PaaS refers to an application development and deployment platform that is delivered as a service to developers, enabling them to quickly build and deploy a SaaS application to end users. The platform typically includes databases, middleware, and development tools, all delivered as a service via the Internet.

- **What Is Infrastructure as a Service (IaaS)?**

IaaS refers to computing hardware (servers, storage, and network) delivered as a service. This typically includes the associated software as well as operating systems, virtualization, clustering, and so on. Examples of IaaS in the public cloud include Amazon's Elastic Compute Cloud (EC2) and Simple Storage Service (S3).

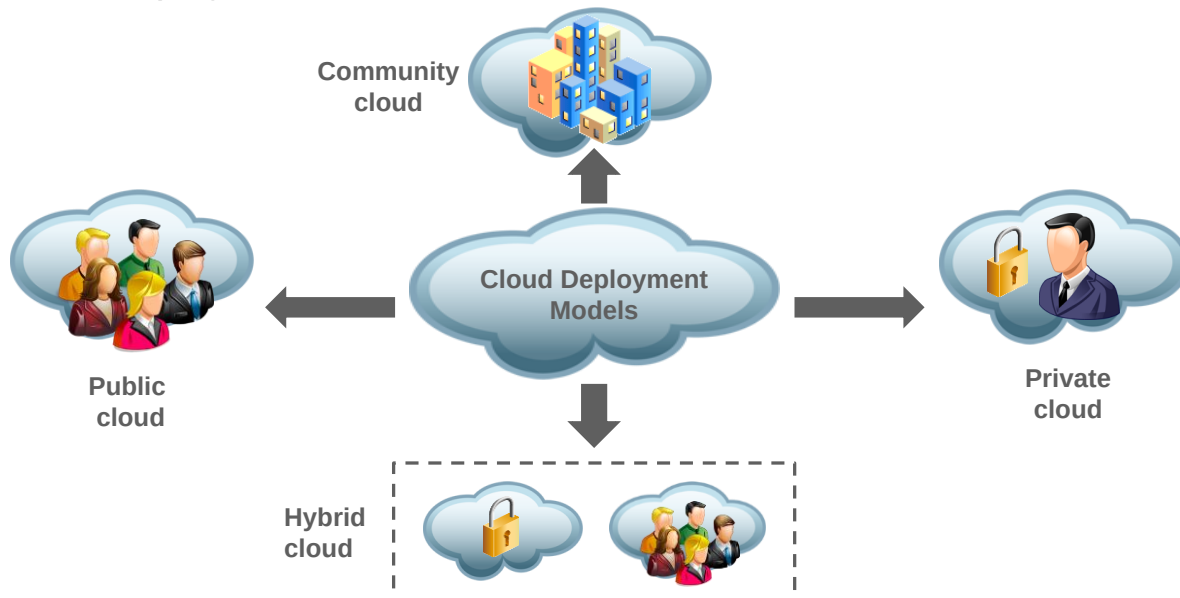
For Instructor Use Only.
This document should not be distributed.

The database cloud is built within an enterprise's private cloud environment, as a PaaS model. It provides on-demand access to database services in a self-service, elastically scalable, and metered manner. Thus offering compelling advantages in cost, quality of service, and agility. A database can also be deployed within a virtual machine on an IaaS platform.

We can quickly deploy database clouds Oracle Exadata, a pre-integrated and optimized hardware platform that supports both online transaction processing (OLTP) and data warehouse (DW) workloads.

For Instructor Use Only.
This document should not be distributed.

Cloud Deployment Models



ORACLE

- **What Is a Private Cloud?**

A private cloud is generally used in private data centers, where a single organization hosts, manages, and controls private data centers or other resources. At times, the hosting is outsourced to a third-party service provider such as Amazon's Virtual Private Cloud.

- **What Is a Public Cloud?**

Public cloud is accessed over a public network such as Internet. The resources provided by the cloud service provider are accessed by multiple users on shared basis, for example, Amazon Elastic Compute Cloud (EC2), IBM's Blue Cloud, Sun Cloud, Google AppEngine, and so on.

- **What is a Community Cloud?**

A group of related organizations, who want to make use of a common cloud computing environment, uses the community cloud. It is managed by the participating organizations or by a third-party managed service provider. It is hosted internally or externally. For example, a community might consist of the different branches of the military, all the universities in a given region, or all the suppliers to a large manufacturer.

For Instructor Use Only.
This document should not be distributed.

- **What Is a Hybrid Cloud?**

A single organization that wants to adopt both private and public clouds for a single application uses the hybrid cloud. A third model, the hybrid cloud, is maintained by both internal and external providers. For example, an organization might use a public cloud service, such as Amazon Simple Storage Service (Amazon S3) for archived data but continue to maintain in-house (private cloud) storage for operational customer data.

Lesson Agenda

- Previewing the course agenda
- Describing the development environments
- Identifying the tables and schemas used in this course
- Overview of Oracle Database 19c and related products
- Oracle Cloud
- Oracle documentation and additional resources
- New Features



ORACLE

For Instructor Use Only.
This document should not be distributed.

Oracle SQL and PL/SQL Documentation

- [Oracle Database New Features Guide](#)
- [Oracle Database Text Application Developer's Guide](#)
- [Oracle Database PL/SQL Language Reference](#)
- [Oracle Database Reference](#)
- [Oracle Database SQL Language Reference](#)
- [Oracle Database Concepts](#)
- [Oracle Database PL/SQL Packages and Types Reference](#)

ORACLE

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Previewing the course agenda
- Describing the development environments
- Identifying the tables and schemas used in this course
- Overview of Oracle Database 19c and related products
- Oracle Cloud
- Oracle documentation and additional resources
- New Features



ORACLE

For Instructor Use Only.
This document should not be distributed.

New Features in 18c and 19c



There are several new features in 18c and 19c:

- Polymorphic Table Functions
- Qualified Expressions
- SQL Macros
- Non-Persistence support for Object Types

ORACLE

For Instructor Use Only.
This document should not be distributed.

Polymorphic Table Functions

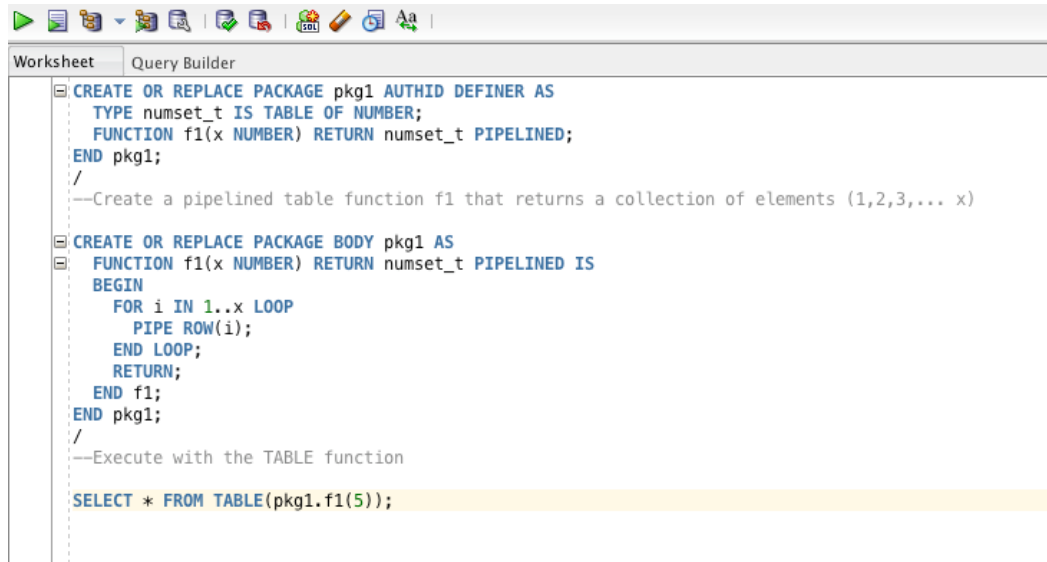
- A polymorphic table function (PTF) is a new type of table function whose return type is determined by the arguments passed into the PTF.
- A table function is a user-defined PL/SQL function that returns a collection of rows (an associative array, nested table, or varray) that can be called from the `FROM` clause of a SQL query block.
- A PTF is useful when SQL developers and database administrators want to provide generic extensions which work for arbitrary input tables or queries.

ORACLE

These are also known as “Pipelined Table Functions.”

For Instructor Use Only.
This document should not be distributed.

Polymorphic Table Functions



```
CREATE OR REPLACE PACKAGE pkg1 AUTHID DEFINER AS
  TYPE numset_t IS TABLE OF NUMBER;
  FUNCTION f1(x NUMBER) RETURN numset_t PIPELINED;
END pkg1;
/
--Create a pipelined table function f1 that returns a collection of elements (1,2,3,... x)

CREATE OR REPLACE PACKAGE BODY pkg1 AS
  FUNCTION f1(x NUMBER) RETURN numset_t PIPELINED IS
  BEGIN
    FOR i IN 1..x LOOP
      PIPE ROW(i);
    END LOOP;
    RETURN;
  END f1;
END pkg1;
/
--Execute with the TABLE function

SELECT * FROM TABLE(pkg1.f1(5));
```

ORACLE

The following are not allowed for POLYMORPHIC table functions:

- PARALLEL_ENABLE clause
- RESULT_CACHE clause
- DETERMINISTIC option
- AUTHID property (Invoker's Rights and Definer's Rights Clause)

For Instructor Use Only.
This document should not be distributed.

Qualified Expressions

- Starting with Oracle Database Release 18c, any PL/SQL value can be provided by an expression (for example for a record or for an associative array), like a constructor provides an abstract datatype value.
- In PL/SQL, we use the terms "qualified expression" and "aggregate" rather than the SQL term "type constructor"; however, the functionality is the same.
- Qualified expressions use an explicit type indication to provide the type of the qualified item. This explicit indication is known as a typemark.
- The syntax for qualified expressions is:

```
qualified_expression ::= typemark ( aggregate )
aggregate ::= [ positional_choice_list ] [explicit_choice_list]
positional_choice_list ::= (expr )+
explicit_choice_list ::= named_choice_list | indexed_choice_list
named_choice_list ::= identifier => expr [,]+
indexed_choice_list ::= expr => expr [,] +
```

ORACLE

For Instructor Use Only.
This document should not be distributed.

Qualified Expressions

This example uses a function to display the values of a table of BOOLEAN:

```
CREATE FUNCTION print_bool (v IN BOOLEAN)
RETURN VARCHAR2
IS
    v_rtn VARCHAR2(10);
BEGIN
    CASE v
        WHEN TRUE THEN
            v_rtn := 'TRUE';
        WHEN FALSE THEN
            v_rtn := 'FALSE';
        ELSE
            v_rtn := 'NULL';
        END CASE;
    RETURN v_rtn;
END print_bool;
```

The `v_aa1` variable is initialized using index key-value pairs:

```
DECLARE
    TYPE t_aa IS TABLE OF BOOLEAN INDEX BY PLS_INTEGER;
    v_aa1 t_aa := t_aa(1=>FALSE,
                      2=>TRUE,
                      3=>NULL);
BEGIN
    DBMS_OUTPUT.PUT_LINE(print_bool(v_aa1(1)));
    DBMS_OUTPUT.PUT_LINE(print_bool(v_aa1(2)));
    DBMS_OUTPUT.PUT_LINE(print_bool(v_aa1(3)));
END;
```

FALSE
TRUE
NULL

ORACLE

See more information at: <https://docs.oracle.com/en/database/oracle/oracle-database/19/lnpls/plsql-collections-and-records.html#GUID-4B379569-4068-4DB1-9D69-E706F4AC5758>.

For Instructor Use Only.
This document should not be distributed.

SQL Macros

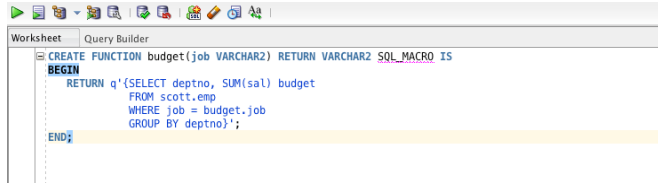
- Starting in 19c, you can create SQL macros (SQM) to factor out common SQL expressions and statements into reusable, parameterized constructs that can be used in other SQL statements.
- SQL table macros increase developer productivity, simplify collaborative development, and improve code quality.
- The AUTHID property cannot be specified.
- A SQL macro can appear only in the `FROM` clause of a query table expression.
- A SQL macro cannot appear in a virtual column expression, functional index, editioning view, or materialized view.
- Type methods cannot be annotated with `SQL_MACRO`.

ORACLE

For Instructor Use Only.
This document should not be distributed.

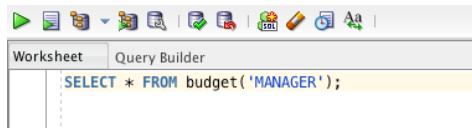
SQL Macros

Create the Function:



```
CREATE FUNCTION budget(job VARCHAR2) RETURN VARCHAR2 SQL_MACRO IS
BEGIN
  RETURN q'(SELECT deptno, SUM(sal) budget
            FROM scott.emp
            WHERE job = budget.job
            GROUP BY deptno)';
END;
```

Execute the Function:



```
SELECT * FROM budget('MANAGER');
```

DEPTNO	BUDGET
20	2975
30	2850
10	2450

ORACLE

For Instructor Use Only.
This document should not be distributed.

Non-Persistence Support for Object Types

- This feature enables you to mark abstract data types used in programs for processing only, for storage only, or for mixed use.
- Instances of non-persistable types cannot persist on disk.
- You cannot specify the `[NOT] PERSISTABLE` clause in a subtype definition.

ORACLE

For Instructor Use Only.
This document should not be distributed.

Summary

In this lesson, you should have learned how to:

- Describe the goals of the course
- Identify the environments that can be used in this course
- Describe the database schema and tables that are used in the course
- List the available documentation and resources



ORACLE

In this lesson, you were introduced to the goals of the course, the SQL Developer and SQL*Plus environments used in the course, and the database schema and tables used in the lectures and practices.

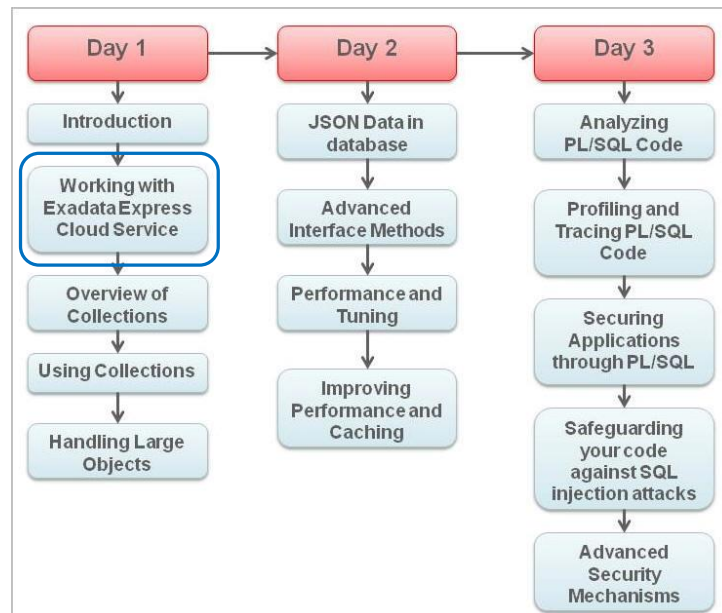
For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

Working with Exadata Express Cloud Service

ORACLE

Course Agenda



ORACLE

For Instructor Use Only.
This document should not be distributed.

Lesson Objectives

After completing this lesson, you should be able to do the following:

- Define Oracle Database Exadata Express Cloud Service
- List the features of Oracle Database Exadata Express Cloud Service
- Discuss the service console and its components of Oracle Database Exadata Express Cloud Service
- Identify the different database clients that can be used for connecting to Oracle Database Exadata Express Cloud Service



ORACLE

In this lesson, you are introduced to Oracle Exadata Express Cloud Service and its features. You will take a tour of its service console and also learn about the different database clients, such as Oracle SQL Developer, SQLcl, SQL Workshop, and SQL* Plus that can be used to connect to Oracle Database Exadata Express Cloud Service.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

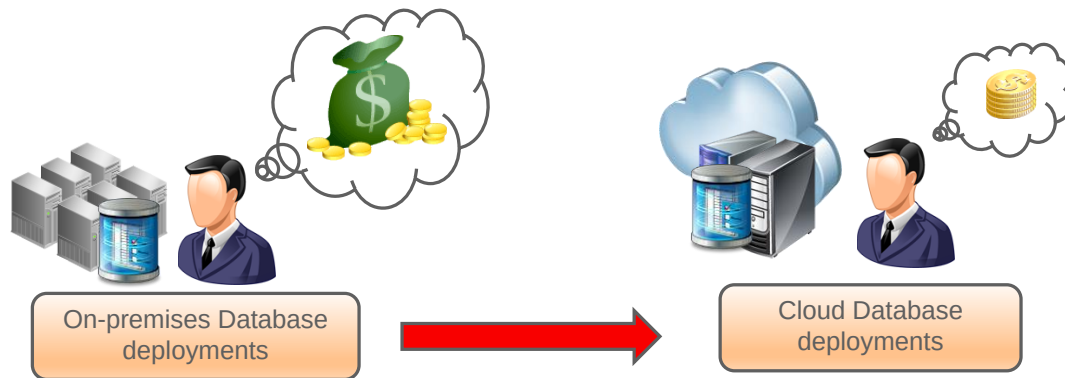
- Overview of Oracle Database Exadata Express Cloud Service
- Understanding the Service Console
- Accessing Cloud Database Using SQL Workshop
- Connecting to Exadata Express Using Database Clients
 - Connecting Oracle SQL Developer
 - Connecting Oracle SQLcl



ORACLE

For Instructor Use Only.
This document should not be distributed.

Evolving from On-premises to Exadata Express



ORACLE

Cloud deployments provide end users and enterprises with different capabilities to store and process data. They enable users to have high performance and huge computing resources at a lower price as compared to traditional on-premises deployments.

Exadata Express is a powerful database machine, extended as a cloud service. End users can use it for Oracle Database 12c deployments. It delivers a complete database experience for developers and enterprises.

Exadata Express being a cloud deployment provides high scalability, performance, and availability to its users.

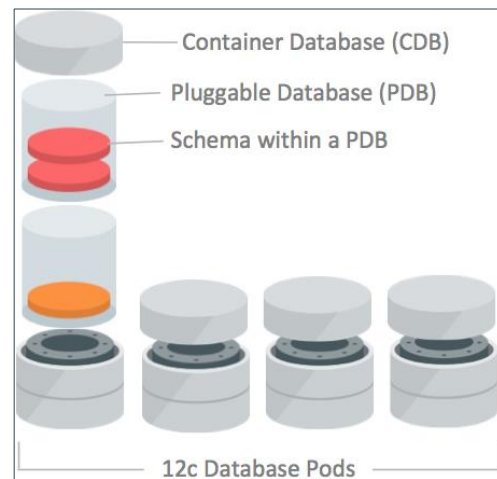
It is fully managed database, therefore, you need not worry about patching, upgrading, or other DBA tasks.

For 19c, consider trying Oracle Cloud's Free Tier. Details are at <https://docs.cloud.oracle.com/en-us/iaas/Content/FreeTier/freetier.htm>.

For Instructor Use Only.
This document should not be distributed.

Exadata Express for Users

- Oracle manages the service as multiple Container databases (CDBs), also known as database pods.
- Each CDB can accommodate up to 1000 Pluggable databases (PDBs).
- Each user is provisioned with a PDB on subscribing to the service, where the user can create several schemas.



ORACLE

Exadata Express is ideal for production applications that consist of small to medium-sized data as well as developers, testers, evaluators and other users who are looking for a full Oracle Database experience at an affordable entry-level price. It is a fully managed database service, organized into container databases (CDBs). These container databases are also known as database pods.

Each container database in turn can contain several pluggable databases (PDBs). When a user subscribes to the Exadata Service, a pluggable database is provisioned. Within the PDB, the user can create several schemas. However, PDB Services are constrained by CPU, storage, and memory.

For Instructor Use Only.
This document should not be distributed.

Exadata Express for Developers

Developers can connect with a wide range of data sources for their applications:

- JSON Document Storage
- Document Style data access
- Oracle Rest Data Services



ORACLE

JSON Document Storage: Oracle Database in Exadata Express provides direct storage, access, and management of JSON documents. See “JSON Support” in *Oracle Database Release 19c New Features Guide*.

Document-Style Data Access: Oracle Database in Exadata Express gives you the ability to store and access data as schema-less documents and collections using the Simple Oracle Document Access (SODA) API. See “Working with JSON and Other Data Using SODA” in *Using Oracle Database Exadata Express Cloud Service*.

Oracle REST Data Services 3: Exadata Express includes the newest Oracle REST Data Services (ORDS). With ORDS 3, it’s easy to develop modern RESTful interfaces for relational data and now JSON documents stored in Oracle Database.

For Instructor Use Only.
This document should not be distributed.

Getting Started with Exadata Express

1. Purchase a subscription.
2. Activate and verify the service.
3. Verify activation.
4. Learn about users and roles.
5. Create accounts for your users and assign them appropriate privileges and roles.
6. Set the password for the database user authorized to perform administrative tasks for your service (PDB_ADMIN).

Note: Refer to *Using Oracle Database Exadata Express Cloud Service* for details on the subscription process:

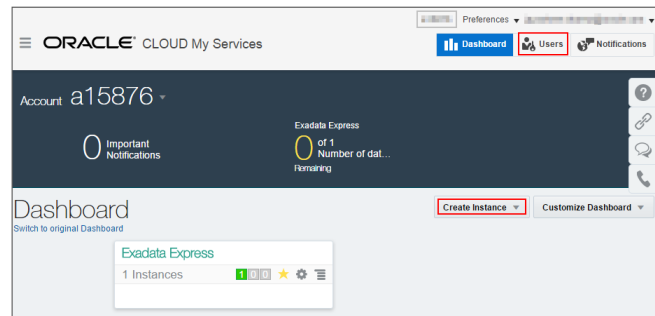
- <https://docs.oracle.com/cloud/latest/exadataexpress-cloud/CSDBP/toc.htm>

ORACLE

For Instructor Use Only.
This document should not be distributed.

Getting Started with Exadata Express

- On signing in to the service, you get access to the dashboard.
- Dashboard allows you to create database instances and users.
- The number of instances you create is limited by the amount of resources you have access to.



ORACLE

After successful subscription to the service, you can log in to your account and access the dashboard. Based on the type of subscription, you can create instances.

The instances would appear on the dashboard. You can see an instance created in the image. To manage the instance, click the instance.

For Instructor Use Only.
This document should not be distributed.

Managing Exadata

Service Instances

Create Service Instance

exa4

Service Type: Exadata Express
Instance ID: 500033811
Status: Active
Size: BASIC
Service SFTP User Name: us148271
Service SFTP Host & Port: den00vrc.us.oracle.com

Administrator: dbcloud-alm_vw@oracle.com
Requested By: dbcloud-alm_vw@oracle.com
Service Instance URL: http://exa4-a1587...

[Open Service Console](#)

Service Console: exa4

Web Access
Develop database and web apps using Oracle Application Express (APEX)
[Learn More](#) [Watch Video](#)

Go to SQL Workshop
Run SQL commands, execute SQL scripts and browse database objects

Develop with App Builder
Declaratively develop and deploy data-driven apps

Define REST Data Services
Create and manage RESTful web service interfaces to your database

Install Productivity Apps
Browse and install productivity apps

Client Access
Enable database client access, then connect using drivers and tools
[Learn More](#) [Watch Video](#)

Download Client Credentials
Download a zip file containing your security credentials and network configuration files

Download Drivers
Get database drivers for Java, .NET, Node.js, Python, PHP, Ruby, C, C++, Instant Client and more

Disable Client Access
Disable SQL*Net access and invalidate all existing client credential files

Download Tools
Get SQL*Plus command-line and developer tools including SQL Developer and JDeveloper

Administration
Manage your cloud database
[Learn More](#) [Watch Video](#)

Create Database Schema
Create a new schema for database objects

Create Document Store
Enable or disable a schema-less document-style interface, with JSON storage and access

Set Administrator Password
Set or reset your database's privileged user (SYS) account password

Manage Application Express
Use Application Express (APEX) administrative options

ORACLE

On clicking the instance on the dashboard, you see various details about the dashboard. You can access the services by clicking Open Service Console.

The service console provides you access to tools for Web Access, Client Access, and Administration.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Overview of Oracle Database Exadata Express Cloud Service
- **Understanding Service Console**
- Accessing Cloud Database Using SQL Workshop
- Connecting to Exadata Express Using Database Clients
 - Connecting Oracle SQL Developer
 - Connecting Oracle SQLcl

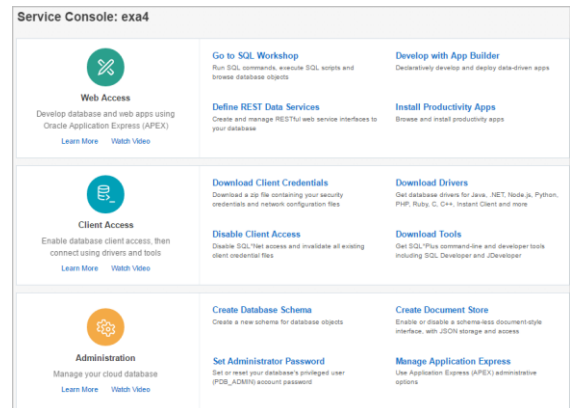


ORACLE

For Instructor Use Only.
This document should not be distributed.

Service Console

- Service Console is the interface to use and manage the Exadata service.
- It provides three different perspectives of the instance:
 - Web Access
 - Client Access
 - Administration



ORACLE

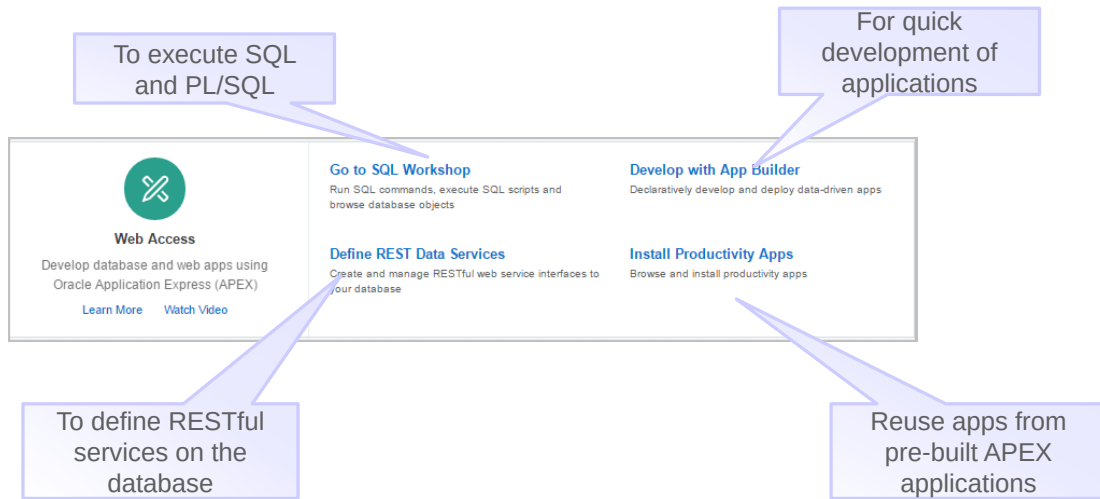
Web access provides utilities that enable you to develop database and web applications using Oracle Application Express (APEX).

Database clients can connect to Exadata Express service by using SQL *Net Access. Some examples of supported database clients are SQLcl, SQL Developer, SQL *Plus, JDBC Thin client, ODP.NET, OCI, and Instant Client. Client Access in the Service Console allows you to configure with the client you use.

Administration in the Service Console enables you to perform administration tasks such as creating new database schemas for database objects, setting or resetting administration password, creating a schema-less documents and collections interface, and using administrative options to manage Oracle Application Express.

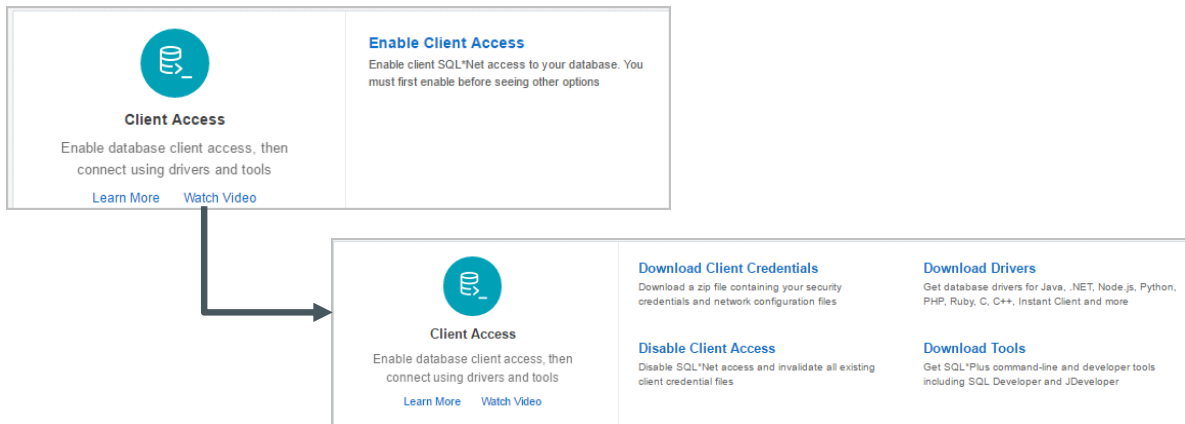
For Instructor Use Only.
This document should not be distributed.

Web Access Through Service Console



Option	Description
Go to SQL Workshop	Allows you to go directly to browser-based SQL Workshop, where you can run SQL statements, execute scripts and explore database objects
Develop with App Builder	Quickly and declaratively develop database and worksheet applications. You can import files such as database applications and plug-ins. There is a dashboard showing metrics about your applications and workspace utilities to manage defaults, themes, metadata, exports, and more.
Define REST Data Services	Directly access the page to define and manage RESTful web services that view and manipulate data objects within your database
Install Productivity Apps	Enables you to install from a gallery of prebuilt Oracle Application Express Productivity Apps

Client Access Configuration Through Service Console

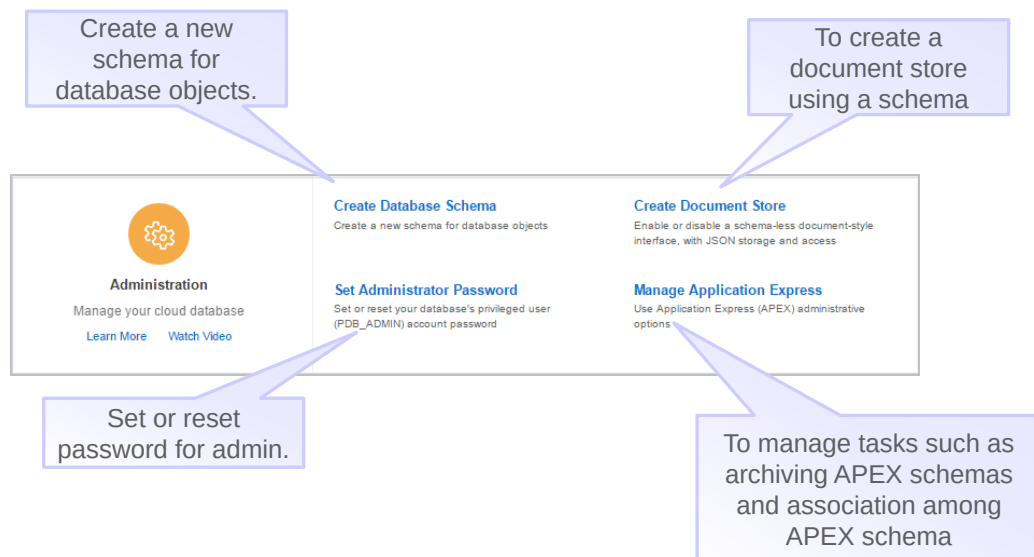


ORACLE

You have to enable client access to allow SQL*Net Access to your service. Using SQL*Net Access software, you can connect the Exadata instance to different clients. This option is available only when client access has not yet been enabled. Once you enable client access, four options appear in the console:

Option	Description
Download Client Credentials	Download client credentials needed for clients to access your service.
Download Drivers	Go directly to the Oracle Technology Network page to download and install database drivers including for Java, Instant Client, C, C++, Microsoft .NET, Node.js, Python, PHP, Ruby, and more.
Disable Client Access	Use this option to disallow SQL*Net access to your service. This option is available only when client access has been enabled.
Download tools	Go directly to the Oracle Technology Network page to download and install tools such as SQL*Plus, SQLcl, and command-line and integrated development environments such as Oracle SQL Developer, JDeveloper, Oracle JET, and more.

Database Administration Through Service Console



ORACLE

You can perform various administration tasks through “Administration” in the service console.

Option	Description
Create Database Schema	Create a new schema for database objects. Schema is the set of database objects, such as tables and views that belong to that user account.
Create Document Store	This option enables you to create a document store, using either an existing schema or new schema, and to enable SODA for REST, which enables REST-based operations on the schema using Oracle’s SODA for REST API. It also enables SODA for Java, which is Oracle’s SODA for Java API for use with Java programs.
Set Administrator Password	Use this option to set the password for the PDB_ADMIN database user that is authorized to perform administrative tasks.
Manage Application Express	Options here allow you to enable application archiving to archive your Oracle Application Express applications to database tables, manage the association between schemas and Oracle Application Express, and manage messages and set preferences for the workspace.

Lesson Agenda

- Overview of Oracle Database Exadata Express Cloud Service
- Understanding the Service Console
- Accessing Cloud Database Using SQL Workshop
- Connecting to Exadata Express Using Database Clients
 - Connecting Oracle SQL Developer
 - Connecting Oracle SQLcl



ORACLE

For Instructor Use Only.
This document should not be distributed.

SQL Workshop



Web Access
Develop database and web apps using
Oracle Application Express (APEX)
[Learn More](#) [Watch Video](#)

Go to SQL Workshop
Run SQL commands, execute SQL scripts and
browse database objects

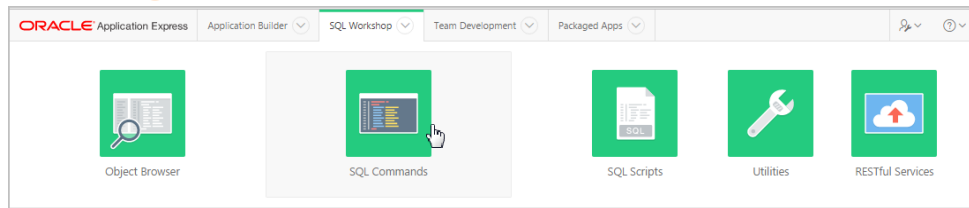
Define REST Data Services
Create and manage RESTful web service interfaces to
your database

Develop with App Builder
Declaratively develop and deploy data-driven apps

Install Productivity Apps
Browse and install productivity apps

1

Clicking SQL workshop will lead you to the APEX interface.



2

To run SQL or PL/SQL, you can use the SQL commands utility.

ORACLE

For Instructor Use Only.
This document should not be distributed.

SQL Workshop

You can run SQL statements in the editor. (To be used in SQL courses)

The screenshot displays the Oracle SQL Workshop interface. At the top, the 'Application Express' header is visible. Below it, the 'SQL Workshop' tab is selected. The 'SQL Commands' editor shows a query: `SELECT * FROM emp;`. The 'Schema' dropdown is set to 'BZJNMKSSA'. The 'Rows' dropdown is set to '10'. The 'Run' button is highlighted. Below the editor, the 'Results' tab is active, displaying a table with 8 columns: EMPNO, ENAME, JOB, MGR, HIREDATE, SAL, COMM, and DEPTNO. The table contains 7 rows of data.

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7839	KING	PRESIDENT	-	11/17/1981	5000	-	10
7698	BLAKE	MANAGER	7839	05/01/1981	2850	-	30
7782	CLARK	MANAGER	7839	06/09/1981	2450	-	10
7566	JONES	MANAGER	7839	04/02/1981	2975	-	20
7788	SCOTT	ANALYST	7566	12/09/1982	3000	-	20
7902	FORD	ANALYST	7566	12/03/1981	3000	-	20

SQL Workshop in APEX enables you to run SQL statements using SQL commands editor. The results of the SQL query entered will be displayed on the Results tab.

For Instructor Use Only.
This document should not be distributed.

SQL Workshop

You can run PL/SQL statements in the editor. (To be used in PL/SQL courses)

The screenshot displays the Oracle SQL Workshop interface. At the top, there is a navigation bar with tabs for 'Application Express', 'Application Builder', 'SQL Workshop' (which is selected), 'Team Development', and 'Packaged Apps'. Below the navigation bar, the 'SQL Commands' tab is active. The 'Schema' dropdown is set to 'BZJNMKSSSA'. The 'Rows' dropdown is set to '10'. The 'Clear Command' and 'Find Tables' buttons are visible. The main editor area contains the following PL/SQL code:

```
BEGIN
DBMS_OUTPUT.PUT_LINE('Hello World');
END;
```

Below the editor, the 'Results' tab is selected, showing the output of the execution:

```
Hello World
Statement processed.
0.34 seconds
```

ORACLE

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Overview of Oracle Database Exadata Express Cloud Service
- Understanding the Service Console
- Accessing Cloud Database Using SQL Workshop
- Connecting to Exadata Express Using Database Clients
 - Connecting Oracle SQL Developer
 - Connecting Oracle SQLcl



ORACLE

For Instructor Use Only.
This document should not be distributed.

Connecting Through Database Clients

You can connect to Exadata Express by using various database clients:

- SQL*Plus
- SQLcl
- SQL Developer
- .Net and Visual Studio
- JDBC Thin Client

ORACLE

You must first configure database clients and Oracle Database Exadata Express Cloud Service to communicate with each other.

Prerequisite tasks for database client connectivity require you to:

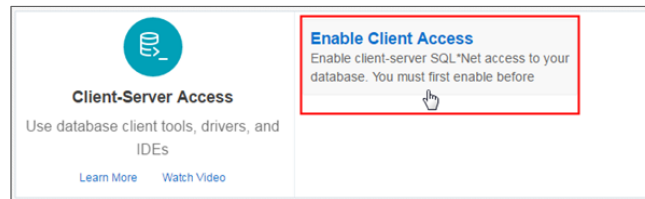
- Enable SQL*Net access to your service
- Download client credentials
- Follow setup instructions for the specific database client you want to connect with

In the following topics, you will learn how to enable SQL*Net access, download client credentials, and make connections using Oracle SQL Developer and SQLcl.

For Instructor Use Only.
This document should not be distributed.

Enabling SQL*Net Access for Client Applications

Enable SQL*Net Access in the Service Console to obtain the various Database Client options.



ORACLE

You can connect to your Exadata Express cloud service from diverse database clients over SQL*Net also called Oracle Database Net Services. Some examples of supported clients include SQL*Plus, SQLcl, SQL Developer, JDBC Thin, ODP.NET, OCI, and Instant Client.

Database drivers for all popular programming and scripting languages such as Python, PHP, Node.js, C/C++, Ruby, and Perl are supported. SQL*Net access has to be enabled as a prerequisite for all clients and drivers connecting over SQL*Net.

The Service Administrator must do the following to enable SQL*Net:

- Navigate to the Service Console for **Exadata** Express and open the service console.
- Click **Enable Client Access**.
- Download the client credentials.
- Now, depending on the client-side application and driver being used, configure the application connection string for that application.

For Instructor Use Only.
This document should not be distributed.

Downloading Client Credentials

The screenshot shows the Oracle Cloud Service Console interface. In the top navigation bar, there are three main sections: 'Client-Server Access', 'Download Client Credentials', and 'Download Client Developer Tools'. The 'Download Client Credentials' section is highlighted with a red box. Below this, there is a 'Disable Client Access' link. A red circle with the number '1' is placed over the 'Download Client Credentials' link. The main content area shows a 'Download Client Credentials' dialog box. The dialog has a title bar, a close button, and a message: 'Provide a password for your client credentials, then click Download below.' There are two input fields: 'Password' and 'Confirm Password'. The 'Confirm Password' field has a green checkmark and the text 'Passwords match!'. Below the input fields, there is a paragraph of text explaining the purpose of the client credentials file. At the bottom of the dialog, there are 'Cancel' and 'Download' buttons. A red circle with the number '2' is placed over the 'Download' button. The Oracle logo is visible in the bottom left corner of the console.

You can easily download the zip files for client credentials from service console. Its contents include Oracle Wallet and Java Keystore as well as essential client configuration files. While downloading the zip file, you are prompted to enter a password.

Note that client credential zip files should be carefully managed. Please remember to keep the file secure to avoid unauthorized database access. If you believe the security of this file has been compromised, then immediately disable client access using the cloud service console.

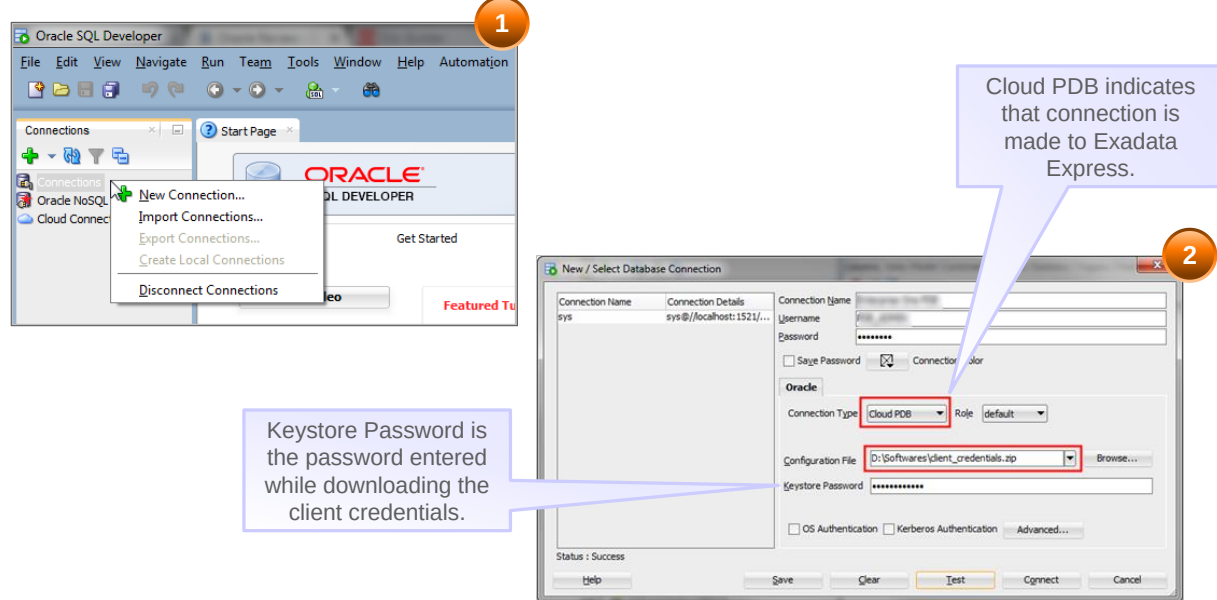
The steps to download the client credentials are as follows:

1. Navigate to Exadata Express and open the service console.
2. Click Download Client Credentials to download a zip file containing your security credentials and network configuration files.
3. Enter a password to create a password-protected Oracle Wallet and Java Keystore files for the service.
4. Click Download and save the downloaded zip file to a secure location that is accessible by your database clients.

In the following slides, you will learn how to use these credentials to connect to the cloud database.

For Instructor Use Only.
This document should not be distributed.

Connecting Oracle SQL Developer



To connect to Oracle Database Exadata Express Cloud Service, you need to download and install Oracle SQL Developer Release 4.1.5 or later. You should have also downloaded the Client Credentials from the Oracle Exadata Express service console. You should then configure an Oracle Cloud connection in Oracle SQL Developer.

The connection can be created as follows:

1. Run Oracle SQL Developer locally.
2. Under Connections, right-click Connections and select **New Connection**.
3. Enter the following details:
 - **Connection Name:** Enter a name for this cloud connection.
 - **Username:** Enter the username required to sign in to Exadata Express.
 - **Password:** Enter the password required to sign in to Exadata Express.
 - **Connection type:** Select **Cloud PDB**.
 - **Configuration File:** Click Browse and select the **Client Credentials** zip file that you previously downloaded from the Exadata Express service console.
 - **Keystore Password:** Enter the password provided while downloading the Client Credentials from the Exadata Express service console.
4. Click Test. If the status is Success, click Connect.

If you have connected successfully, the tables and other objects from Exadata Express display under the new connection.

Connecting Oracle SQLcl

```
D:\PDB Service\SQL CL\sqlcl-no-jre-latest\sqlcl\bin>sql /nolog
```

```
SQLcl: Release 20.2.0.174.1557 RC on Tue Sep 08 12:18:07 2020
```

```
Copyright (c) 1982, 2016, Oracle. All rights reserved.  
SQL>
```

1

```
SQL> set cloudconfig client_credentials.zip
```

```
Wallet Password: *****
```

```
Using temp directory:C:\Users\APOTHU~1.ORA\AppData\Local\Temp\  
oracle_cloud_config6707346342028726502
```

2

```
SQL> conn pdb_admin/welcome1@dbaccess
```

```
Connected.
```

```
SQL>
```

3

ORACLE

You can use Oracle SQLcl, which is a powerful command-line interface (CLI) to connect to the cloud database. To connect, you need to first download and set up Oracle SQLcl locally. You should have also downloaded the Client Credentials from Oracle Exadata Express service console.

To create an Oracle SQLcl cloud connection:

- Navigate to the `sqlcl/bin` directory from where you unzipped the SQLcl installation files, and run `sql /nolog`, to startup Oracle SQLcl. Oracle SQLcl starts displaying the date and time, the SQLcl version, and copyright information, before the SQLcl prompt appears.
- At the SQLcl prompt, type `set cloudconfig <name of your wallet zip file>`, and press the Enter key.
- Enter the Password provided for downloading the Client Credentials from the Exadata Express service console, and press the Enter key.
- To connect to the Exadata Express, type `conn <username>/<password>@<servicename>`, and press the Enter key. `username` and `password` are the credentials of your database account.

You should now be connected to Exadata Express.

For Instructor Use Only.
This document should not be distributed.

Summary

In this lesson, you should have learned about:

- Oracle Database Exadata Express Cloud Service
- The features of Exadata Express Cloud Service
- The process to connect to Database Cloud using SQL Workshop
- The different database clients used to connect to Exadata Express Cloud Service



ORACLE

This lesson provided an overview of Oracle Database Exadata Express Cloud Service. If you want to learn more about Oracle Database Exadata Express Cloud Service, visit <http://docs.oracle.com/cloud/latest/exadataexpress-cloud/index.html>.

If you want to learn more about Oracle Database Cloud Service and get a free trial of Oracle Database Cloud Service, visit <https://cloud.oracle.com/database>.

For Instructor Use Only.
This document should not be distributed.

Practice 2: Overview

There is no practice for lesson 2.



ORACLE

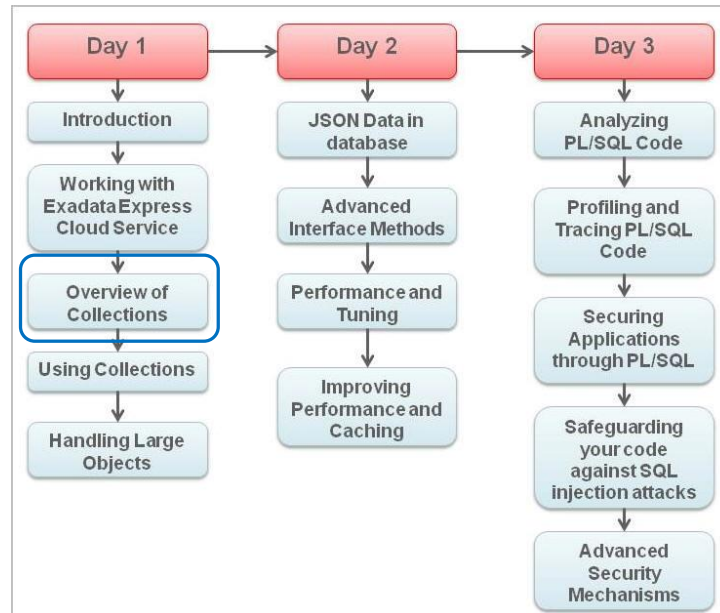
For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

Overview of Collections

ORACLE

Course Agenda



ORACLE

In this lesson, you are introduced to Collections. Collections are composite data types; you can store values of data which can in turn have various internal components. In an earlier course (*Oracle Database 19c: PL/SQL Workshop*) you must've learned about records. Records are also composite data types.

A collection stores a set of values of same data type, whereas a record stores a set of values of different data types. In this lesson, we will discuss different variants of collections.

For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to create and traverse:

- Associative arrays
- Varrays
- Nested tables



ORACLE

On completing this lesson, you'll understand what collections are and the different types of collections. You can create a collection and traverse through the elements of the collection after completing this lesson.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Understanding Collections
- Creating Associative Arrays
- Creating Nested Tables
- Creating Varrays

ORACLE



For Instructor Use Only.
This document should not be distributed.

Collections

- A Collection is a group of data values.
- All the values are of same type.
- Each value in the collection is accessed through an index.
- Types of collections in PL/SQL:
 - Associative arrays
 - String-indexed collections
 - INDEX BY PLS_INTEGER or BINARY_INTEGER
 - Nested tables
 - Varrays



ORACLE

A collection is a group of values, all of the same type. Each value is accessed through a unique subscript that determines its position in the collection. You can group a set of objects (document objects such as reports) as a collection. You can store information of a subset of employees (all managers in the organization) in a collection for various purposes in your application. Collections work like arrays found in most third-generation programming languages.

Collections are declared and used within a PL/SQL block. You can retrieve data from database tables into collections and write the data back into tables from PL/SQL blocks.

Collections can also be passed as parameters to PL/SQL blocks. You can use them to move columns of data into and out of database tables, or between client-side applications and stored subprograms.

There are three categories of collections:

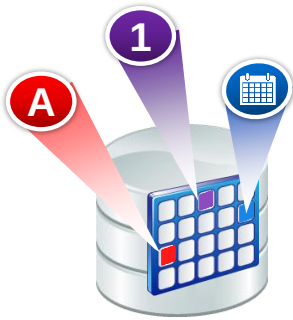
- Associative arrays (known as “index by tables” in previous Oracle releases) are sets of key-value pairs, where each key is unique and is used to locate a corresponding value in the array. The key can be an integer or a string.
- Nested tables can have any number of elements.
- A varray is an ordered collection of elements with an upper limit on the number of elements.

Note: Associative arrays indexed by `PLS_INTEGER` are covered in the prerequisite course, *Oracle Database 19c: Workshop*, and are not emphasized in this course.

For Instructor Use Only.
This document should not be distributed.

Why Collections?

We have variables.



We have cursors.



Then why collections?

ORACLE

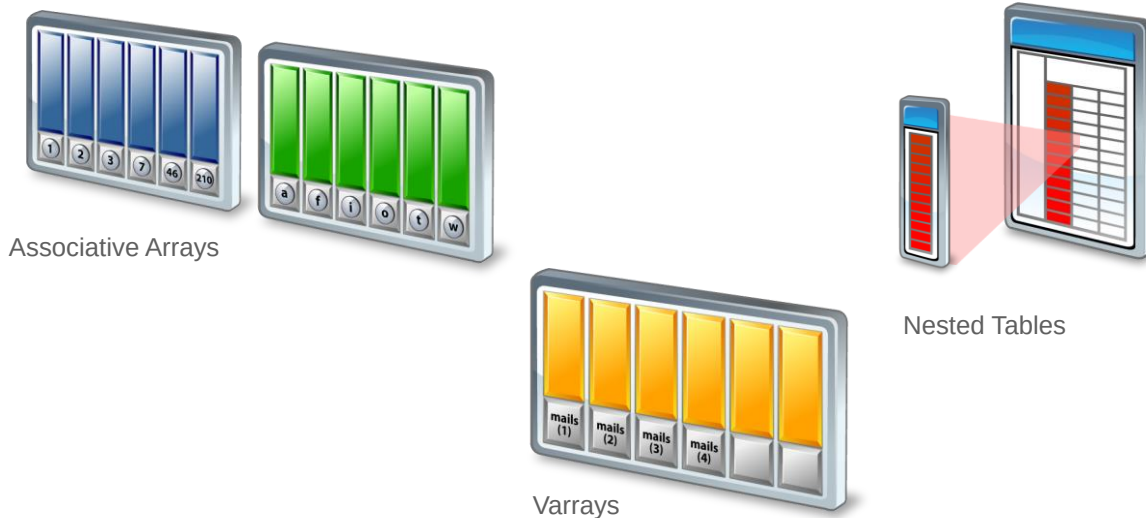
You use variables in PL/SQL blocks to store and operate on data. Variables are used to store single values of certain data type. Whenever you want to retrieve a set of values from the database and operate upon it in PL/SQL block, you can use cursors.

A cursor retrieves data from the database based on a SQL query. The data retrieved by the cursor is known as an active set. You can traverse through each row of data retrieved by the cursor and perform operations on it. However, data retrieved by the cursor is not stored anywhere. Every time you open a cursor, the data is fetched into memory from the database and operated upon.

When you want to store and operate on a group of data values in PL/SQL block, then using Collections would be the right option. Instead of retrieving the data from the database each time you need it using a cursor, you can have the data in the collection. Using a collection will reduce the I/O and improve the application performance.

For Instructor Use Only.
This document should not be distributed.

Collection Types



ORACLE

Associative Arrays

Associative arrays are sets of key-value pairs, where each key is unique and is used to locate a corresponding value in the array. The key can be either integer (`PLS_INTEGER` or `BINARY_INTEGER`) or a character string (`VARCHAR2`). Associative arrays may be sparse.

When you assign a value by using a key for the first time, it adds that key to the associative array. Subsequent assignments using the same key update the same entry. However, it is important to choose a key that is unique. For example, the key values may come from the primary key of a database table, from a numeric hash function, or from concatenating strings to form a unique string value.

Associative arrays are generally used for storing temporary data. Therefore, you can't use them with SQL statements, such as `INSERT` and `SELECT INTO`. You can make them persistent for the life of a database session by declaring the type in a package and assigning the values in a package body. They are typically populated with a `SELECT BULK COLLECT` statement unless they are `VARCHAR2` indexed. `BULK COLLECT` prevents context switching between the SQL and PL/SQL engines, and is much more efficient on large data sets.

For Instructor Use Only.
This document should not be distributed.

Nested Tables

A nested table holds a set of values, you can see it as a single column table. It is a column type and can be stored into the database. Each value in such column can hold a set of rows which can be seen as equivalent to a single column table. You can access each row in the nested table through a subscript value starting with 1.

Nested tables are unbounded; that is, the single column table can have any number of rows. In a PL/SQL block, nested tables are like one-dimensional arrays whose size can increase dynamically. In the database, nested tables are column types where each value of this column will in turn hold a single column table. The Oracle database stores the rows of a nested table in no particular order. When you retrieve a nested table from the database into a PL/SQL variable, the rows are given consecutive subscripts starting at 1. This gives you an array-like access to individual rows. Nested tables are initially dense, but they can become sparse through deletions and, therefore, have nonconsecutive subscripts.

Varrays

A **varray (variable-size array)** is an array whose number of elements can vary from zero (empty) to the declared maximum size. You can access each element of the varray through an index. The lower bound of *index* is 1; the upper bound is the current number of elements. The upper bound changes as you add or delete elements, but it cannot exceed the maximum size. When you store and retrieve a varray from the database, its indexes and element order remain stable.

Associative Arrays	Nested Tables	Varrays
These are key-value pairs where each key is a unique index associated with a value.	These store an unspecified number of elements in no particular order.	These store a specified number of elements.
The values in an associative array are indexed according to the key value provided during initialization.	The elements in a nested table are indexed starting from 1.	Varrays are indexed from 1 to the upper limit specified during declaration in the PL/SQL block.
They cannot be persisted to a database, and can be used only in PL/SQL blocks.	They can be a table column and persisted to the database.	They can be a table column and persisted to the database.

Lesson Agenda

- Understanding Collections
- **Creating Associative Arrays**
- Creating Nested Tables
- Creating Varrays



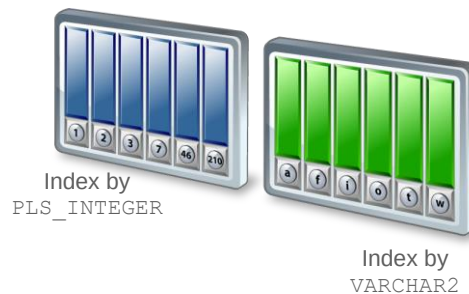
ORACLE

For Instructor Use Only.
This document should not be distributed.

Using Associative Arrays

Associative arrays:

- That are indexed by strings or integers can improve performance
- Are pure memory structures that are much faster than schema-level tables
- Provide significant additional flexibility



ORACLE

An associative array (formerly called PL/SQL table or index-by table) is a set of key-value pairs. Each key is a unique index, you can access the corresponding value through the index.

The data type of index can be either a string type (`VARCHAR2`, `VARCHAR`, `STRING`, or `LONG`) or `PLS_INTEGER`. Indexes are stored in sort order, not creation order.

An associative array can hold unspecified number of elements.

For Instructor Use Only.
This document should not be distributed.

Creating an Associative Array

Syntax:

```
TYPE type_name IS TABLE OF element_type  
INDEX BY VARCHAR2(size)
```

Example:

```
CREATE OR REPLACE PROCEDURE report_credit  
(p_last_name customers.cust_last_name%TYPE,  
 p_credit_limit customers.credit_limit%TYPE)  
IS  
  TYPE typ_name IS TABLE OF customers%ROWTYPE  
    INDEX BY customers.cust_email%TYPE;  
  v_by_cust_email typ_name;  
  i VARCHAR2(50);  
  
  PROCEDURE load_arrays IS  
  BEGIN  
    FOR rec IN (SELECT * FROM customers WHERE cust_email IS NOT NULL)  
    LOOP  
      -- Load up the array in single pass to database table.  
      v_by_cust_email (rec.cust_email) := rec;  
    END LOOP;  
  END;  
END;
```

← Create the string-indexed associative array type.

← Create the string-indexed associative array variable.

← Populate the string-indexed associative array variable.

ORACLE

You can see the syntax of creating an associative array or index by tables. You can index the values of array either through an integer (PLS_INTEGER) or through a character string (VARCHAR2, VARCHAR2, VARCHAR, STRING, or LONG).

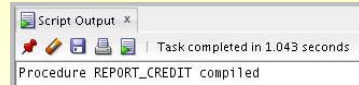
The REPORT_CREDIT procedure shown in the slide accepts two parameters – the customer last name and a credit limit value. The procedure is expected to display the email IDs and credit limit of the customers, whose last name is the same as the parameter passed to the procedure and credit limit is greater than the value passed as parameter.

The string-indexed collection is loaded with the customer information in the load_arrays procedure. In the main body of the program, the collection is traversed to find the credit information. The associative array is indexed through the email ID because it is unique. The last name of the customer can repeat but not the email ID.

For Instructor Use Only.
This document should not be distributed.

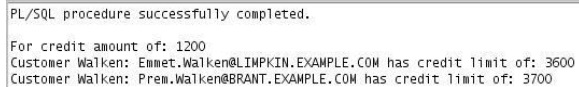
Traversing an Associative Array

```
...  
BEGIN  
  load_arrays;  
  i:= v_by_cust_email.FIRST;  
  dbms_output.put_line ('For credit amount of: ' || p_credit_limit);  
  WHILE i IS NOT NULL LOOP  
    IF v_by_cust_email(i).cust_last_name = p_last_name  
    AND v_by_cust_email(i).credit_limit > p_credit_limit  
    THEN dbms_output.put_line ( 'Customer ' ||  
      v_by_cust_email(i).cust_last_name || ': ' ||  
      v_by_cust_email(i).cust_email || ' has credit limit of: ' ||  
      v_by_cust_email(i).credit_limit);  
    END IF;  
    i := v_by_cust_email.NEXT(i);  
  END LOOP;  
END report_credit;  
/
```



Script Output x
Task completed in 1.043 seconds
Procedure REPORT_CREDIT compiled

```
EXECUTE report_credit('Walken', 1200)
```



PL/SQL procedure successfully completed.
For credit amount of: 1200
Customer Walken: Emmet.Walken@LIMPKIN.EXAMPLE.COM has credit limit of: 3600
Customer Walken: Prem.Walken@BRANT.EXAMPLE.COM has credit limit of: 3700

ORACLE

You can see that in the given code, you are traversing the collection through a loop. The loop exits when there is a null index while traversing the collection. You can see the usage of `v_by_email.FIRST` and `v_by_cust_email.NEXT(i)`. These are predefined Collection methods.

For Instructor Use Only.
This document should not be distributed.

Here is the modified code:

```
CREATE OR REPLACE PROCEDURE report_credit
  (p_email      customers.cust_email%TYPE,
   p_credit_limit customers.credit_limit%TYPE)
IS
  TYPE  typ_name IS TABLE OF customers%ROWTYPE
        INDEX BY customers.cust_email%TYPE;
  v_by_cust_email  typ_name;
  i VARCHAR2(50);

  PROCEDURE load_arrays IS
  BEGIN
    FOR rec IN (SELECT * FROM customers
                WHERE cust_email IS NOT NULL) LOOP
      v_by_cust_email (rec.cust_email) := rec;
    END LOOP;
  END;

BEGIN
  load_arrays;
  dbms_output.put_line
    ('For credit amount of: ' || p_credit_limit);
  IF v_by_cust_email(p_email).credit_limit > p_credit_limit
    THEN dbms_output.put_line ( 'Customer ' ||
      v_by_cust_email(p_email).cust_last_name ||
      ': ' || v_by_cust_email(p_email).cust_email ||
      ' has credit limit of: ' ||
      v_by_cust_email(p_email).credit_limit);
  END IF;
END report_credit;
/

EXECUTE report_credit('Prem.Walken@BRANT.EXAMPLE.COM', 100)
```

```
PL/SQL procedure successfully completed.
```

```
For credit amount of: 100
```

```
Customer Walken: Prem.Walken@BRANT.EXAMPLE.COM has credit limit of: 3700
```

Collection Methods

- A collection method is a PL/SQL subprogram.
- It can be a function or a procedure.
- Collection methods simplify the usage of Collections in PL/SQL block.
- You can use the collection methods along with the Collection variable.

Usage:

```
collection_variable.method
```

ORACLE

A collection method is a PL/SQL subprogram – procedure or function. You can use the collection method in the PL/SQL block just as you use any other procedure or function.

You invoke the collection methods along with the collection variables. Following is a list of commonly used collection methods:

Method	Type	Description
DELETE	Procedure	Deletes elements from a collection
TRIM	Procedure	Deletes elements from the end of varray or nested table
EXTEND	Procedure	Adds elements to the end of varray or nested table
EXISTS	Function	Returns TRUE if and only if a specified element of varray or nested table exists
FIRST	Function	Returns the first index in a collection
LAST	Function	Returns the last index in a collection
COUNT	Function	Returns the number of elements in a collection
LIMIT	Function	Returns the maximum number of elements that a collection can have
PRIOR	Function	Returns index that precedes a specified index
NEXT	Function	Returns index that succeeds a specified index

Lesson Agenda

- Understanding Collections
- Creating Associative Arrays
- **Creating Nested Tables**
- Creating Varrays



ORACLE

For Instructor Use Only.
This document should not be distributed.

Nested Tables

- A collection can hold a set of values or rows.
- All the values are of the same type.
- There is no upper limit on the number of values in the collection.
- Values can be stored as a column type in a table.
- Each value in a nested table is accessed through an index starting from 1.

ORACLE

A nested table holds a set of values. There is no upper limit on the number of values that can be stored in a nested table. The size of the nested table can vary dynamically.

When used in a PL/SQL block, each value in the collection is referenced through a numeric index starting from 1.

In the database, nested tables are column types that hold a set of values. The Oracle database stores the rows of a nested table in no particular order. Each row in the nested table can be a set of scalar values or a set of records.

Nested tables are initially dense, but they can become sparse through deletions and, therefore, have nonconsecutive subscripts.

For Instructor Use Only.
This document should not be distributed.

Creating Nested Table Types

- To create a nested table type in the database:

```
CREATE [OR REPLACE] TYPE type_name AS TABLE OF  
Element_datatype [NOT NULL];
```

- To create a nested table type in PL/SQL:

```
TYPE type_name IS TABLE OF element_datatype  
[NOT NULL];
```

ORACLE

To create a collection, you first define a collection type, and then declare collections of that type. The slide shows the syntax for defining the nested table collection type in both the database (persistent) and in PL/SQL (transient).

Creating Collections in the Database

You can create a nested table data type in the database, which makes the data type available to use in places such as columns in database tables, variables in PL/SQL programs, and attributes of object types.

Before you can define a database table containing a nested table, you must first create the data type for the collection in the database.

Use the syntax shown in the slide to create collection types in the database.

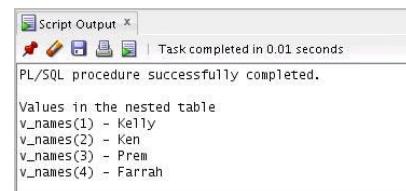
Creating Collections in PL/SQL

You can also create a nested table in PL/SQL. Use the syntax shown in the slide to create collection types in PL/SQL.

For Instructor Use Only.
This document should not be distributed.

Nested Tables: Example

```
DECLARE
    TYPE names IS TABLE OF VARCHAR2(15);
    v_names names := names('Kelly','Ken','Prem','Farrah');
BEGIN
    DBMS_OUTPUT.PUT_LINE('Values in the nested table');
    FOR i IN v_names.FIRST . . v_names.LAST LOOP
        DBMS_OUTPUT.PUT_LINE('v_names('||i||') - '||v_names(i));
    END LOOP;
END;
```



Script Output x
Task completed in 0.01 seconds
PL/SQL procedure successfully completed.
Values in the nested table
v_names(1) - Kelly
v_names(2) - Ken
v_names(3) - Prem
v_names(4) - Farrah

ORACLE

The slide shows a simple nested table created in a PL/SQL block.

You create a data type `names`, which is the nested table. You can store nested tables as column types in the database. Therefore, before attempting to store the nested table in the database, you have to create a type of the nested table. In the slide, we create a data type `names` and create a variable `v_names` of type `names`.

An uninitialized nested table is a null collection. You are initializing the nested table in the code through a collection constructor. You will learn about collection constructors in the following slide.

You traverse the nested table using a `for` loop. The first value of the collection `v_names` is referred through `v_names.first` and the last value is referred through `v_names.last`.

For Instructor Use Only.
This document should not be distributed.

Collection Constructors

A **collection constructor (constructor)** is a system-defined function with the same name as a collection type, which returns a collection of that type.

Example:

```
v_names names := names('Kelly', 'Ken', 'Prem', 'Farrah');
```

ORACLE

A collection constructor is used to initialize the collection. In the example, the collection `v_names` of type `names` is initialized with a list of four values.

You can also create null collections by having an empty initialization list.

For Instructor Use Only.
This document should not be distributed.

Declaring Collections: Nested Table

- First, define an object type:

```
CREATE TYPE typ_item AS OBJECT --create object
(
  prodid NUMBER(5),
  price  NUMBER(7,2) )
/
CREATE TYPE typ_item_nst -- define nested table type
AS TABLE OF typ_item
/
```

1

2

- Then, declare a column of that collection type:

```
CREATE TABLE pOrder ( -- create database table
  ordid NUMBER(5),
  supplier NUMBER(5),
  requester NUMBER(4),
  ordered DATE,
  items typ_item_nst)
NESTED TABLE items STORE AS item_stor_tab
/
```

3

ORACLE

You have seen creating a collection of scalar data values in the earlier example. You can also create a collection of objects. The slide shows creation of the object type and then creating a collection of the objects defined.

To create a table based on a nested table, perform the following steps:

1. Create the `typ_item` type, which holds the information for a single line item.
2. Create the `typ_item_nst` type, which is created as a table of the `typ_item` type.
Note: You must create the `typ_item_nst` nested table type based on the previously declared type.
3. Create the `pOrder` table and use the nested table type in a column declaration, which includes an arbitrary number of items based on the `typ_item_nst` type. Thus, each row of `pOrder` may contain a table of items.

The `NESTED TABLE STORE AS` clause is required to indicate the name of the storage table in which the rows of all values of the nested table reside. The storage table is created in the same schema and the same tablespace as the parent table.

Note: The `USER_COLL_TYPES` dictionary view holds information about collections.

For Instructor Use Only.
This document should not be distributed.

Using Nested Tables

- Add data to the nested table:

```
INSERT INTO pOrder
VALUES (500, 50, 5000, sysdate, typ_item_nst(
    typ_item(55, 555),
    typ_item(56, 566),
    typ_item(57, 577)));
```

```
INSERT INTO pOrder
VALUES (800, 80, 8000, sysdate,
    typ_item_nst (typ_item (88, 888)));
```

pOrder nested table

ORDID	SUPPLIER	REQUESTER	ORDERED	ITEMS	PRODID	PRICE
500	50	5000	30-OCT-07		55	555
					56	566
					57	577
800	80	8000	31-OCT-07		88	888

ORACLE

To insert data into the nested table, you use the `INSERT` statement. You are using a collection constructor to add values to the nested table.

In the example in the slide, the constructors are `TYP_ITEM_NST()` and `TYP_ITEM()`. You pass two elements to the `TYP_ITEM()` constructor, and then pass the results to the `TYP_ITEM_NST()` constructor to build the nested table structure.

The first `INSERT` statement builds the nested table with three subelement rows.

The second `INSERT` statement builds the nested table with one subelement row.

Using Nested Tables

- Querying the results:

```
select * from porder;
```

Query Result x

SQL | All Rows Fetched: 2 in 0.002 seconds

	ORDID	SUPPLIER	REQUESTER	ORDERED	ITEMS
1	500	50	5000 09-MAR-17	0E.TYP_ITEM_NST([0E.TYP_ITEM], [0E.TYP_ITEM], [0E.TYP_ITEM])	
2	800	80	8000 09-MAR-17	0E.TYP_ITEM_NST([0E.TYP_ITEM])	

- Querying the results with the TABLE function:

```
select * from p2.ordid,p1.*  
from porder p2, TABLE(p2.items) p1;
```

Query Result x

SQL | All Rows Fetched: 4 in 0.002 seconds

	ORDID	PRODID	PRICE
1	500	55	555
2	500	56	566
3	500	57	577
4	800	88	888

ORACLE

To see the data in the nested table, you should use the `TABLE` function. When you execute a simple `SELECT` statement to see the data in the table, the data in the nested table column would not appear.

To display the output, you should unnest the collection such that each collection element appears on a row by itself. You can use the `TABLE` expression in the `FROM` clause to unnest a collection.

A `TABLE` expression enables you to query a collection in the `FROM` clause like a table. In effect, you join the nested table with the row that contains the nested table without writing a `JOIN` statement.

The collection column in the `TABLE` expression uses a table alias to identify the containing table.

Referencing Collection Elements

Use the collection name and a subscript to reference a collection element:

- Syntax:

```
collection_name(subscript)
```

- Example:

```
v_with_discount(i)
```

- To reference a field in a collection:

```
p_new_items(i).prodid
```

ORACLE

Every element reference includes a collection name and a subscript enclosed in parentheses. The subscript determines which element is processed. To reference an element, you can specify its subscript by using the following syntax:

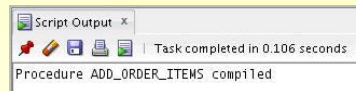
```
collection_name(subscript)
```

In the preceding syntax, `subscript` is an expression that yields a positive integer. For nested tables, the integer must lie in the range 1 through 2147483647. For varrays, the integer must lie in the range 1 through `maximum_size`.

For Instructor Use Only.
This document should not be distributed.

Using Nested Tables in PL/SQL

```
CREATE OR REPLACE PROCEDURE add_order_items
(p_ordid NUMBER, p_new_items typ_item_nst)
IS
    v_num_items      NUMBER;
    v_with_discount  typ_item_nst;
BEGIN
    v_num_items := p_new_items.COUNT;
    v_with_discount := p_new_items;
    IF v_num_items > 2 THEN
        --ordering more than 2 items gives a 5% discount
        FOR i IN 1..v_num_items LOOP
            v_with_discount(i) :=
                typ_item(p_new_items(i).prodid,
                    p_new_items(i).price*.95);
        END LOOP;
    END IF;
    UPDATE pOrder
        SET items = v_with_discount
        WHERE ordid = p_ordid;
END;
```



ORACLE

When you define a variable of a collection type in a PL/SQL block, it is transient and available only for the scope of the PL/SQL block.

In the example shown in the slide:

- The nested table `P_NEW_ITEMS` parameter is passed into the block.
- A local variable `V_WITH_DISCOUNT` is defined with the nested table data type `TYP_ITEM_NST`.
- A collection method, called `COUNT`, is used to determine the number of items in the nested table.
- If more than two items are counted in the collection, the local nested table variable `V_WITH_DISCOUNT` is updated with the product ID and a 5% discount on the price.
- To reference an element in the collection, the subscript `i`, representing an integer from the current loop iteration, is used with the constructor method to identify the row of the nested table.
- Update the `pOrder` table with the discount values.

For Instructor Use Only.
This document should not be distributed.

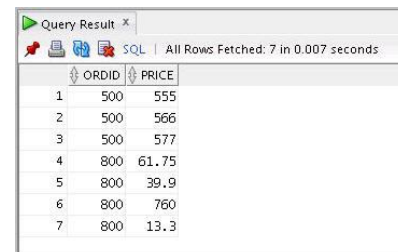
Using Nested Tables in PL/SQL

```
-- caller pgm:
DECLARE
  v_form_items  typ_item_nst:= typ_item_nst();
BEGIN
  -- let's say the form holds 4 items
  v_form_items.EXTEND(4);
  v_form_items(1) := typ_item(1804, 65);
  v_form_items(2) := typ_item(3172, 42);
  v_form_items(3) := typ_item(3337, 800);
  v_form_items(4) := typ_item(2144, 14);
  add_order_items(800, v_form_items);
END;
```

```
SELECT p2.ordid, p1.price
FROM porder p2, TABLE(p2.items) p1;
```

v_form_items variable

PRODID	PRICE
1804	65
3172	42
3337	800
2144	14



ORDID	PRICE	
1	500	555
2	500	566
3	500	577
4	800	61.75
5	800	39.9
6	800	760
7	800	13.3

ORACLE

In the example code shown in the slide:

- A local PL/SQL variable of nested table type is declared and instantiated with the `TYP_ITEM_NST()` collection method.
- The nested table variable is extended to hold four rows of elements with the `EXTEND(4)` method.
- The nested table variable is populated with four rows of elements by constructing a row of the nested table with the `TYP_ITEM` constructor.
- The nested table variable is passed as a parameter to the `ADD_ORDER_ITEMS` procedure shown on the previous page.
- The `ADD_ORDER_ITEMS` procedure updates the `ITEMS` nested table column in the `pOrder` table with the contents of the nested table parameter passed into the routine.

Lesson Agenda

- Understanding Collections
- Creating Associative Arrays
- Creating Nested Tables
- Creating Varrays

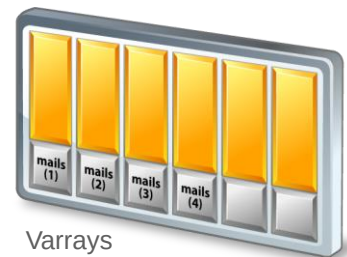


ORACLE

For Instructor Use Only.
This document should not be distributed.

Varrays

- Are variable sized arrays
- Can hold data values from zero to a declared maximum size
- Can be stored in the database as a table column



Varrays

ORACLE

Varrays are also collections of homogeneous elements that hold a fixed number of elements (although you can change the number of elements using the `EXTEND` method). They use sequential numbers as subscripts.

You can define varrays as a SQL type, thereby allowing varrays to be stored in database tables. They can be stored and retrieved through SQL, but with less flexibility than nested tables. You can reference individual elements for array operations, or manipulate the collection as a whole.

You can define varrays in PL/SQL to be used during PL/SQL program execution.

Varrays are always bounded and never sparse. You can specify the maximum size of the varray in its type definition. Its index has a fixed lower bound of 1 and an extensible upper bound. A varray can contain a varying number of elements, from zero (when empty) to the maximum specified in its type definition.

To reference an element, you can use the standard subscripting syntax.

For Instructor Use Only.
This document should not be distributed.

Varrays

- To create a varray in the database:

```
CREATE [OR REPLACE] TYPE type_name AS VARRAY  
    (max_elements) OF element_datatype [NOT NULL];
```

- To create a varray in PL/SQL:

```
TYPE type_name IS VARRAY (max_elements) OF  
element_datatype [NOT NULL];
```

ORACLE

You can see the syntax of creating a varray in the slide. Varrays like nested tables can be stored as database columns. You can choose to use a varray when you know the maximum number of values that have to be accommodated in the collection.

You can increase the capacity of nested tables or varrays by using the `EXTEND` method.

For Instructor Use Only.
This document should not be distributed.

Declaring Collections: Varray

- First, define a collection type:

```
CREATE TYPE typ_Project AS OBJECT( --create object
  project_no NUMBER(4),
  title      VARCHAR2(35),
  cost       NUMBER(12,2))
/
CREATE TYPE typ_ProjectList AS VARRAY (50) OF typ_Project
-- define VARRAY type
/
```

- Then, declare a collection of that type:

```
CREATE TABLE department ( -- create database table
  dept_id  NUMBER(2),
  name     VARCHAR2(25),
  budget   NUMBER(12,2),
  projects typ_ProjectList) -- declare varray as column
/
```

ORACLE

The example in the slide shows how to create a table based on a varray.

1. Create the `TYP_PROJECT` type, which holds the information for a project.
2. Create the `TYP_PROJECTLIST` type, which is created as a varray of the project type. The varray contains a maximum of 50 elements.
3. Create the `DEPARTMENT` table and use the varray type in a column declaration. Each element of the varray will store a project object.

This example demonstrates how to create a varray of phone numbers, and then use it in a `CUSTOMERS` table: (The OE sample schema uses this definition.)

```
CREATE TYPE phone_list_typ
AS VARRAY(5) OF VARCHAR2(25);
/
CREATE TABLE customers
(customer_id NUMBER(6)
,cust_first_name VARCHAR2(50)
,cust_last_name VARCHAR2(50)
,cust_address cust_address_typ(100)
,phone_numbers phone_list_typ
...
);
```

Using Varrays

Add data to the table containing a varray column:

```
INSERT INTO department
VALUES (10, 'Executive Administration', 30000000,
       typ_ProjectList(
         typ_Project(1001, 'Travel Monitor', 400000),
         typ_Project(1002, 'Open World', 10000000)));
```

1

```
INSERT INTO department
VALUES (20, 'Information Technology', 5000000,
       typ_ProjectList(
         typ_Project(2001, 'DB11gR2', 900000)));
```

2

DEPARTMENT table

DEPT_ID	NAME	BUDGET	PROJECTS		
			PROJECT_NO	TITLE	COSTS
10	Executive Administration	30000000	1001	Travel Monitor	400000
			1002	Open World	10000000
20	Information Technology	5000000	2001	DB11gR2	900000

1

2

ORACLE

To add rows to the `DEPARTMENT` table that contains the `PROJECTS` varray column, you use the `INSERT` statement. The structure of the varray column is identified with the constructor methods.

- `TYP_PROJECTLIST()` constructor constructs the varray data type.
- `TYP_PROJECT()` constructs the elements for the rows of the varray data type.

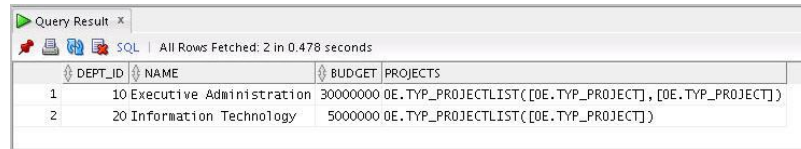
The first `INSERT` statement adds three rows to the `PROJECTS` varray for department 10.

The second `INSERT` statement adds one row to the `PROJECTS` varray for department 20.

Using Varrays

- Querying the results:

```
SELECT * FROM department;
```

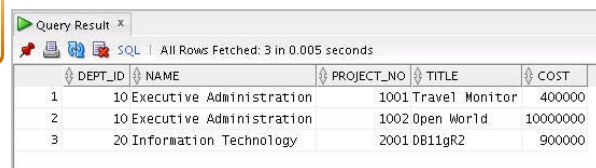


Query Result x
SQL | All Rows Fetched: 2 in 0.478 seconds

DEPT_ID	NAME	BUDGET	PROJECTS
1	10 Executive Administration	300000000	OE.TYP_PROJECTLIST([OE.TYP_PROJECT], [OE.TYP_PROJECT])
2	20 Information Technology	50000000	OE.TYP_PROJECTLIST([OE.TYP_PROJECT])

- Querying the results with the TABLE function:

```
SELECT d2.dept_id, d2.name, d1.*  
FROM department d2, TABLE(d2.projects) d1;
```



Query Result x
SQL | All Rows Fetched: 3 in 0.005 seconds

DEPT_ID	NAME	PROJECT_NO	TITLE	COST
1	10 Executive Administration	1001	Travel Monitor	400000
2	10 Executive Administration	1002	Open World	10000000
3	20 Information Technology	2001	DB11gR2	900000

ORACLE

You query a varray column in the same way that you query a nested table column.

In the first example in the slide, the collections are nested in the result rows that contain them. By including the collection column in the `SELECT` list, the output shows as a row associated with the other row output in the `SELECT` list.

In the second example, the output is unnested such that each collection element appears on a row by itself. You can use the `TABLE` expression in the `FROM` clause to unnest a collection.

Quiz

Q

Which of the following collections is a set of key-value pairs, where each key is unique and is used to locate a corresponding value in the collection?

- a. Associative arrays
- b. Nested Table
- c. Varray
- d. Semsegs

?

ORACLE

Answer: a

For Instructor Use Only.
This document should not be distributed.

Quiz

Which of the following collections can be stored in the database?

- a. Associative arrays
- b. Nested table
- c. Varray

ORACLE

Answer: b, c

For Instructor Use Only.
This document should not be distributed.

Summary

In this lesson, you should have learned how to:

- Identify types of collections:
 - Nested tables
 - Varrays
 - Associative arrays
- Define nested tables and varrays in the database



ORACLE

Collections are a grouping of elements, all of the same type. The types of collections are nested tables, varrays, and associative arrays. You can define nested tables and varrays in the database. Nested tables, varrays, and associative arrays can be used in a PL/SQL program.

There are guidelines for using collections effectively and for determining which collection type is appropriate under specific circumstances.

For Instructor Use Only.
This document should not be distributed.

Practice 3: Overview

In this practice, you analyze collections for common errors, and create a collection.



ORACLE

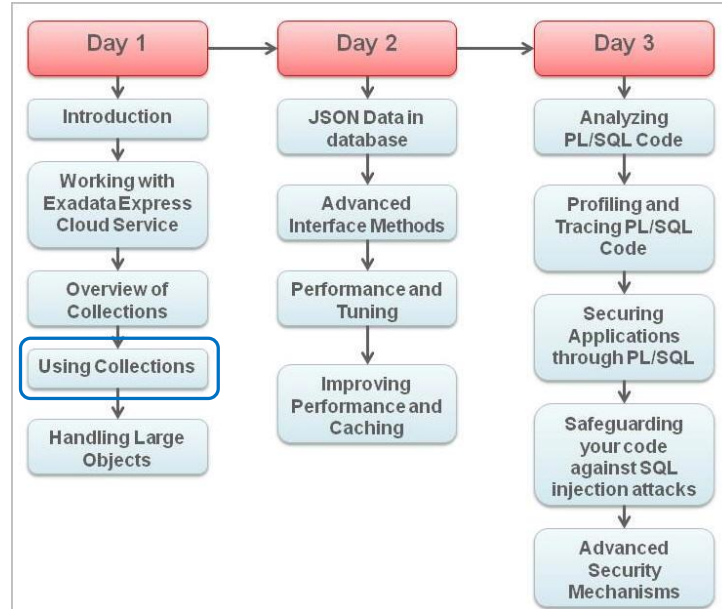
For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

Using Collections

ORACLE

Course Agenda



ORACLE

For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to do the following:

- Use collection methods
- Manipulate collections
- Distinguish between the different types of collections and when to use them
- Use PL/SQL bind types



ORACLE

In this lesson, you learn about using collections.

Collection is a structure used to store a group of values of the same type. You can access each element in the collection through an index. You can use collections to work with a group of data in your PL/SQL block that may or may not be stored in the database tables.

There are three types of collections – associative arrays, nested tables, and varrays. Associative arrays are used only in PL/SQL blocks; you cannot run SQL statements on them. You can have Nested tables and Varrays as column types in database tables and, therefore, run SQL queries on these collections.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Working with Collections
- Collection exceptions
- Summarizing Collections
- PL/SQL Bind types

ORACLE



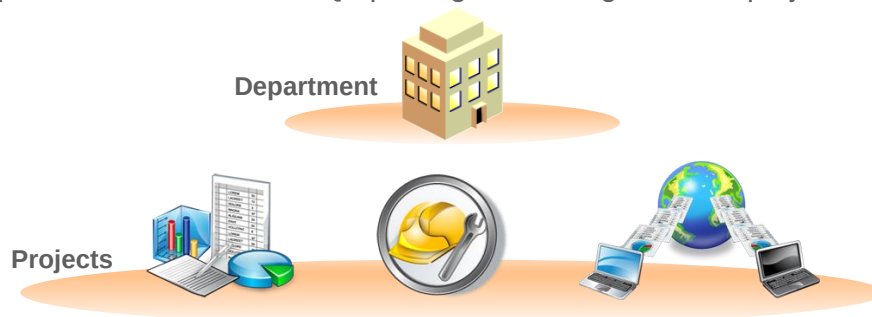
For Instructor Use Only.
This document should not be distributed.

Usage of Collections in Applications

Consider a scenario where you have to keep track of various projects run by a company.

- The company has multiple departments.
- Each department runs multiple projects.
- There is a budget allocated to each project.

The developer has to create a PL/SQL package to manage various projects.



ORACLE

Consider a scenario where the developer has to create mechanisms to manage various projects of a department.

You will create a package `manage_dept_proj` to maintain the data of various projects handled by different departments. The package will implement the following functionality:

1. Assign new projects to a department (Procedure `allocate_new_proj_list`).
2. Retrieve data of a project from the database (Function `get_dept_project`).
3. Update project data in the database (Procedure `update_a_project`).
4. Perform other operations on the project data (`manipulate_project`).
5. Check the costs of the projects incurred by the department (Function `check_costs`).

Working with Collections in PL/SQL

- You can declare collections as the formal parameters of procedures and functions.
- You can specify a collection type in the RETURN clause of a function specification.
- The code of `manage_dept_proj` uses collection both as parameters and return values.

```
CREATE OR REPLACE PACKAGE manage_dept_proj
AS
    PROCEDURE allocate_new_proj_list
        (p_dept_id NUMBER, p_name VARCHAR2, p_budget NUMBER);
    FUNCTION get_dept_project (p_dept_id NUMBER)
        RETURN typ_projectlist;
    PROCEDURE update_a_project
        (p_deptno NUMBER, p_new_project typ_Project,
         p_position NUMBER);
    FUNCTION manipulate_project (p_dept_id NUMBER)
        RETURN typ_projectlist;
    FUNCTION check_costs (p_project_list typ_projectlist)
        RETURN boolean;
END manage_dept_proj;
```

ORACLE

There are several points about collections that you must know when working with them:

- You can declare collections as formal parameters of functions and procedures.
- A function's RETURN clause can be a collection type.
- Collections follow the usual scoping and instantiation rules. In a block or subprogram, collections are instantiated when you enter the block or subprogram and cease to exist when you exit. In a package, collections are instantiated when you first reference the package and cease to exist when you end the database session.

Note: The code on the following page has references `TYP_PROJECT` and `TYP_PROJECTLIST` types and a `department` table. The SQL for the type definition is as follows:

```
CREATE OR REPLACE TYPE typ_project AS OBJECT (
    project_no NUMBER(4),
    title VARCHAR2(35),
    cost NUMBER(12,2));
CREATE OR REPLACE typ_projectlist AS VARRAY (50) OF typ_project;
CREATE TABLE department ( -- create database table
    dept_id NUMBER(2),
    name VARCHAR2(25),
    budget NUMBER(12,2),
    projects typ_ProjectList) -- declare varray as column
```

This is the package body for the varray examples shown on the following pages:

```
CREATE OR REPLACE PACKAGE BODY manage_dept_proj
AS
    PROCEDURE allocate_new_proj_list
        (p_dept_id NUMBER, p_name VARCHAR2, p_budget NUMBER)
    IS
        v_accounting_project typ_projectlist;
    BEGIN
        -- this example uses a constructor
        v_accounting_project :=
            typ_ProjectList
            (typ_Project (1, 'Dsgn New Expense Rpt', 3250),
             typ_Project (2, 'Outsource Payroll', 12350),
             typ_Project (3, 'Audit Accounts Payable', 1425));
        INSERT INTO department VALUES
            (p_dept_id, p_name, p_budget, v_accounting_project);
    END allocate_new_proj_list;

    FUNCTION get_dept_project (p_dept_id NUMBER)
        RETURN typ_projectlist
    IS
        v_accounting_project typ_projectlist;
    BEGIN
        -- this example uses a fetch from the database
        SELECT projects
            INTO v_accounting_project
            FROM department
            WHERE dept_id = p_dept_id;
        RETURN v_accounting_project;
    END get_dept_project;

    PROCEDURE update_a_project
        (p_deptno NUMBER, p_new_project typ_Project,
         p_position NUMBER)
    IS
        v_my_projects typ_ProjectList;
    BEGIN
        v_my_projects := get_dept_project (p_deptno);
        v_my_projects.EXTEND;    --make room for new project
        /* Move varray elements forward */
        FOR i IN REVERSE p_position..v_my_projects.LAST - 1 LOOP
            v_my_projects(i + 1) := v_my_projects(i);
        END LOOP;
        v_my_projects(p_position) := p_new_project; -- add new
                                                    -- project
        UPDATE department SET projects = v_my_projects
            WHERE dept_id = p_deptno;
    END update_a_project;
```

```

FUNCTION manipulate_project (p_dept_id NUMBER)
    RETURN typ_projectlist
IS
    v_accounting_project typ_projectlist;
    v_changed_list typ_projectlist;
BEGIN
    SELECT  projects
        INTO  v_accounting_project
        FROM  department
        WHERE dept_id = p_dept_id;
    -- this example assigns one collection to another
    v_changed_list := v_accounting_project;
    RETURN v_changed_list;
END manipulate_project;

FUNCTION check_costs (p_project_list typ_projectlist)
    RETURN boolean
IS
    c_max_allowed      NUMBER := 10000000;
    i                  INTEGER;
    v_flag              BOOLEAN := FALSE;
BEGIN
    i := p_project_list.FIRST ;
    WHILE i IS NOT NULL LOOP
        IF p_project_list(i).cost > c_max_allowed then
            v_flag := TRUE;
            dbms_output.put_line (p_project_list(i).title ||
                                   ' exceeded allowable budget. ');
            RETURN TRUE;
        END IF;
        i := p_project_list.NEXT(i);
    END LOOP;
    RETURN null;
END check_costs;

END manage_dept_proj;

```

Assigning Values to Collection Variables

You can initialize a constructor by using one of the following mechanisms:

- Invoke a constructor.
- Use an assignment statement.
- Pass it to a subprogram as a parameter and assign value in the subprogram.
- Fetch from the database into the collection.

ORACLE

Until you initialize it, a collection is null (that is, the collection itself is null, not its elements). To initialize a collection, you can use one of the following methods:

- Use a constructor, which is a system-defined function with the same name as the collection type. A constructor allows the creation of an object from an object type. Invoking a constructor is a way to instantiate (create) an object. This function “constructs” collections from the elements passed to it.
- Assign another collection variable directly. You can copy the entire contents of one collection to another as long as both are built from the same data type.
- Pass collection as a parameter to a subprogram and then assign values within the subprogram.
- Fetch data from the database through a `SELECT...INTO` statement.

For Instructor Use Only.
This document should not be distributed.

Assigning Values to Collection Variables

```
CREATE OR REPLACE PROCEDURE allocate_new_proj_list
(p_dept_id NUMBER, p_name VARCHAR2, p_budget NUMBER)
IS
  v_accounting_project typ_projectlist;
BEGIN
  -- this example uses a constructor
  v_accounting_project :=
    typ_ProjectList
      (typ_Project (1, 'Dsgn New Expense Rpt', 3250),
       typ_Project (2, 'Outsource Payroll', 12350),
       typ_Project (3, 'Audit Accounts Payable', 1425));
  INSERT INTO department
    VALUES(p_dept_id, p_name, p_budget, v_accounting_project);
END allocate_new_proj_list;
```

ORACLE

In the procedure, the collection is assigned values through a constructor.

For Instructor Use Only.
This document should not be distributed.

Assigning Values to Collection Variables

```
FUNCTION get_dept_project (p_dept_id NUMBER)
RETURN typ_projectlist
IS
    v_accounting_project typ_projectlist;
BEGIN -- this example uses a fetch from the database
    SELECT projects INTO v_accounting_project
    FROM department WHERE dept_id = p_dept_id;
RETURN v_accounting_project;
END get_dept_project;
```

1

```
FUNCTION manipulate_project (p_dept_id NUMBER)
RETURN typ_projectlist
IS
    v_accounting_project typ_projectlist;
    v_changed_list typ_projectlist;
BEGIN
    SELECT projects INTO v_accounting_project
    FROM department WHERE dept_id = p_dept_id;
    -- this example assigns one collection to another
    v_changed_list := v_accounting_project;
RETURN v_changed_list;
END manipulate_project;
```

2

ORACLE

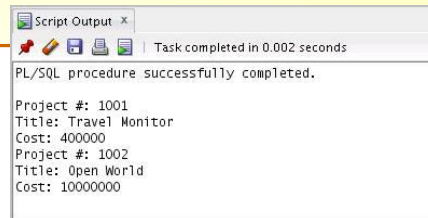
In the first example in the slide, the data from the database is fetched into the local PL/SQL collection variable.

In the second example, one collection variable is assigned to another collection variable.

For Instructor Use Only.
This document should not be distributed.

Accessing Values in the Collection

```
-- sample caller program to the manipulate_project function
DECLARE
    v_result_list typ_projectlist;
BEGIN
    v_result_list := manage_dept_proj.manipulate_project(10);
    FOR i IN 1..v_result_list.COUNT LOOP
        dbms_output.put_line('Project #: ' || v_result_list(i).project_no);
        dbms_output.put_line('Title: ' || v_result_list(i).title);
        dbms_output.put_line('Cost: ' || v_result_list(i).cost);
    END LOOP;
END;
```



ORACLE

The PL/SQL code in the example calls the `manipulate_project` function in the `MANAGE_DEPT_PROJ` package. Department 10 is passed in as the parameter. The output shows the varray element values for the `PROJECTS` column in the `DEPARTMENT` table for department 10.

Although the value of 10 is hard-coded, you can have a form interface to query the user for a department value that can then be passed into the routine.

For Instructor Use Only.
This document should not be distributed.

Working with Collection Methods

Collection methods:

- Are predefined PL/SQL programs
- Help users to operate on collections

```
collection_name.method_name [(parameters)]
```

ORACLE

Function or Procedure	Description
EXISTS	Returns TRUE if and only if a specified element of varray or nested table exists
COUNT	Returns the number of elements that a collection contains
LIMIT	Returns the maximum number of elements that a collection can have
FIRST and LAST	Returns the first and last (smallest and largest) indexes in a collection
PRIOR and NEXT	PRIOR (n) returns the index that precedes index n in a collection; NEXT (n) returns the index that follows index n
EXTEND	Appends one null element. EXTEND (n) appends n elements; EXTEND (n, i) appends n copies of the i th element
TRIM	Removes one element from the end; TRIM (n) removes n elements from the end of a collection
DELETE	Removes all elements from a nested or associative array table. DELETE (n) removes the nth element; DELETE (m, n) removes a range. Note: This does not work on varrays.

Using Collection Methods

Traverse collections with the following methods:

```
FUNCTION check_costs (p_project_list typ_projectlist)
RETURN boolean
IS
  c_max_allowed      NUMBER := 10000000;
  i                  INTEGER;
  v_flag             BOOLEAN := FALSE;
BEGIN
  i := p_project_list.FIRST ;
  WHILE i IS NOT NULL LOOP
    IF p_project_list(i).cost > c_max_allowed then
      v_flag := TRUE;
      dbms_output.put_line (p_project_list(i).title || '
                           exceeded allowable budget. ');

      RETURN TRUE;
    END IF;
    i := p_project_list.NEXT(i);
  END LOOP;
  RETURN null;
END check_costs;
```

ORACLE

In the example in the slide, the `FIRST` method finds the smallest index number, and the `NEXT` method traverses the collection starting at the first index.

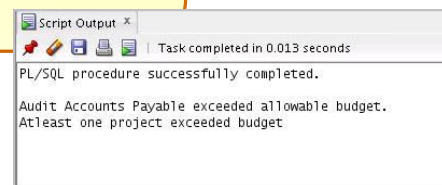
The `check_costs` function checks the collection values and returns a `TRUE` if it encounters a project, which has exceeded the budget constraint. The function returns a `NULL` if none of the projects have exceeded the budget constraint. The maximum budget allowed for a project element is defined by the `C_MAX_ALLOWED` constant in the function.

For Instructor Use Only.
This document should not be distributed.

Using Collection Methods

```
-- sample caller program to check_costs
set serveroutput on
DECLARE
  v_project_list typ_projectlist;
BEGIN
  v_project_list := typ_ProjectList(
    typ_Project (1, 'Dsgn New Expense Rpt', 3250),
    typ_Project (2, 'Outsource Payroll', 120000),
    typ_Project (3, 'Audit Accounts Payable', 14250000));
  IF manage_dept_proj.check_costs(v_project_list) THEN
    dbms_output.put_line('Atleast one project exceeded budget');
  ELSE
    dbms_output.put_line('All Projects accepted, fill out forms.');
```

```
END IF;
END;
```



ORACLE

The code shown in the slide calls the `CHECK_COSTS` function (shown on the previous page). The `CHECK_COSTS` function accepts a `varray` parameter and returns a Boolean value.

A project with three elements is constructed and passed to the `CHECK_COSTS` function. The `CHECK_COSTS` function returns `true`, because the third element of `varray` exceeds the value of the maximum allowed costs.

Although the sample caller program has the `varray` values hard-coded, you could have some sort of form interface where the user enters the values for projects and the form calls the `CHECK_COSTS` function.

For Instructor Use Only.
This document should not be distributed.

Manipulating Individual Elements

```
PROCEDURE update_a_project
(p_deptno NUMBER, p_new_project typ_Project, p_position NUMBER)
IS
    v_my_projects typ_ProjectList;
BEGIN
    v_my_projects := get_dept_project (p_deptno);
    v_my_projects.EXTEND; --make room for new project
    /* Move varray elements by one position */
    FOR i IN REVERSE p_position..v_my_projects.LAST - 1
    LOOP
        v_my_projects(i + 1) := v_my_projects(i);
    END LOOP;
    v_my_projects(p_position) := p_new_project; -- insert new one
    UPDATE department SET projects = v_my_projects
    WHERE dept_id = p_deptno;
END update_a_project;
```

ORACLE

You must use PL/SQL procedural statements to reference the individual elements of a varray in an INSERT, an UPDATE, or a DELETE statement. In the example shown in the slide, the `UPDATE_A_PROJECT` procedure inserts a new project into a department's project list at a given position, and then updates the `PROJECTS` column with the newly entered value that is placed within the old collection values.

This code essentially shuffles the elements of a project so that you can insert a new element in a particular position.

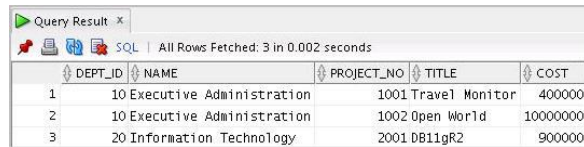
For Instructor Use Only.
This document should not be distributed.

Manipulating Individual Elements

```
-- check the table prior to the update:
SELECT d2.dept_id, d2.name, d1.*
FROM department d2, TABLE(d2.projects) d1;
```

```
-- caller program to update_a_project
BEGIN
  manage_dept_proj.update_a_project(20,
    typ_Project(2002, 'AQM', 80000), 2);
END;
```

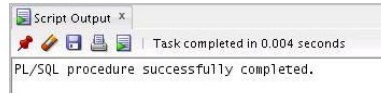
```
-- check the table after the update:
SELECT d2.dept_id, d2.name, d1.*
FROM department d2, TABLE(d2.projects) d1;
```



Query Result x

SQL | All Rows Fetched: 3 in 0.002 seconds

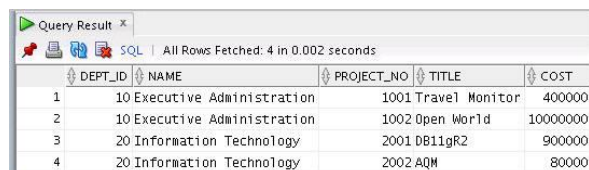
DEPT_ID	NAME	PROJECT_NO	TITLE	COST
1	10 Executive Administration	1001	Travel Monitor	400000
2	10 Executive Administration	1002	Open World	10000000
3	20 Information Technology	2001	DB11gR2	900000



Script Output x

Task completed in 0.004 seconds

PL/SQL procedure successfully completed.



Query Result x

SQL | All Rows Fetched: 4 in 0.002 seconds

DEPT_ID	NAME	PROJECT_NO	TITLE	COST
1	10 Executive Administration	1001	Travel Monitor	400000
2	10 Executive Administration	1002	Open World	10000000
3	20 Information Technology	2001	DB11gR2	900000
4	20 Information Technology	2002	AQM	80000

ORACLE

To execute the procedure, pass the department number to which you want to add a project, the project information, and the position where the project information is to be inserted.

The third code box shown in the slide identifies that a project element should be added to the second position for project 2002 in department 20.

If you execute the following code, the AQM project element is shuffled to position 3 and the CQN project element is inserted at position 2:

```
BEGIN
  manage_dept_proj.update_a_project(20,
    typ_Project(2003, 'CQN', 85000), 2);
END;
```

Querying a Collection by Using the `TABLE` Operator

A collection can be queried if the following are true:

- The collection type was created at the schema level.
- The collection type was declared in a package specification.
- Using a `TABLE` function:
 - Accepts a collection parameter
 - Enables developers to query the collection like a table



ORACLE

A collection can be queried if the following are true:

- The data type of the collection was either created at the schema level or declared in a package specification.
- In the query `FROM` clause, the collection appears in `table_collection_expression` as the argument of the `TABLE` operator. The data type of the collection element is either a scalar data type that SQL supports or a record type in which every field has a data type that SQL supports.
- `TABLE` function – A `TABLE` function accepts a Collection type parameter and returns a collection instance which can be queried like table. You can use the `TABLE` function in the `FROM` clause of the select query.

For Instructor Use Only.
This document should not be distributed.

Querying a Collection with the TABLE Operator

```
CREATE OR REPLACE function GET_EMPS(P_JOB_ID JOBS.JOB_ID%TYPE)
RETURN EMP_TYPE_LIST
IS
...
CURSOR C_EMP(C_JOB_ID JOBS.JOB_ID%TYPE) IS
SELECT DEPARTMENT_ID, LAST_NAME, SALARY, J.JOB_ID, JOB_TITLE
FROM EMPLOYEES E, JOBS J
WHERE E.JOB_ID=J.JOB_ID AND J.JOB_ID=C_JOB_ID;
BEGIN
OPEN C_EMP(P_JOB_ID);
...
RETURN EMPS;
END;
/

SELECT * FROM TABLE(GET_EMPS('IT_PROG'))
/
SELECT D.DEPARTMENT_NAME, E.*
FROM
TABLE(get_emps('IT_PROG')) E, DEPARTMENTS D
WHERE E.DEPARTMENT_ID=D.DEPARTMENT_ID
/
```

ORACLE

The code example in the slide shows the technique of querying a collection returned by a PL/SQL function by using the `TABLE` operator.

The `GET_EMPS` function returns a nested table of type `EMP_TYPE_LIST`. The SQL queries use the `TABLE` operator to read data from the nested table.

The complete example is illustrated on the following page.

For Instructor Use Only.
This document should not be distributed.

Execute the following statements using your HR connection:

```
DROP TYPE EMP_TYPE;

DROP TYPE EMP_TYPE_LIST ;

CREATE OR REPLACE TYPE EMP_TYPE AS OBJECT
(
  DEPARTMENT_ID          NUMBER(6) ,
  LAST_NAME              VARCHAR2(25) ,
  SALARY                 NUMBER(8,2) ,
  JOB_ID                 VARCHAR2(30) ,
  JOB_TITLE               VARCHAR2(30)) ;
/

CREATE OR REPLACE TYPE EMP_TYPE_LIST AS TABLE OF EMP_TYPE;
/

CREATE OR REPLACE function GET_EMPS(P_JOB_ID  JOBS.JOB_ID%TYPE)
RETURN EMP_TYPE_LIST
IS
  EMPS EMP_TYPE_LIST:=EMP_TYPE_LIST();
  R EMP_TYPE:=EMP_TYPE(NULL,NULL,NULL,NULL,NULL);
  i pls_integer:=0;
  CURSOR C_EMP(C_JOB_ID  JOBS.JOB_ID%TYPE) IS
  SELECT DEPARTMENT_ID, LAST_NAME, SALARY, J.JOB_ID, JOB_TITLE
  FROM EMPLOYEES E, JOBS J
  WHERE E.JOB_ID=J.JOB_ID AND J.JOB_ID=C_JOB_ID;
  BEGIN
  OPEN C_EMP(P_JOB_ID);
  LOOP
  FETCH C_EMP INTO
  R.DEPARTMENT_ID, R.LAST_NAME, R.SALARY, R.JOB_ID, R.JOB_TITLE;
  EXIT WHEN C_EMP%NOTFOUND;
  i:=i+1;
  emps.extend;
  EMPS(i):=R;
  END LOOP;
  RETURN EMPS;
END;
```

Check the result:

```
SELECT * FROM TABLE(GET_EMPS('IT_PROG'));
SELECT  D.DEPARTMENT_NAME, E.*
FROM
TABLE(get_emps('IT_PROG')) E, DEPARTMENTS D
WHERE E.DEPARTMENT_ID=D.DEPARTMENT_ID;
```

	DEPARTMENT_ID	LAST_NAME	SALARY	JOB_ID	JOB_TITLE
1	60	Hunold	9000	IT_PROG	Programmer
2	60	Ernst	6000	IT_PROG	Programmer
3	60	Austin	4800	IT_PROG	Programmer
4	60	Pataballa	4800	IT_PROG	Programmer
5	60	Lorentz	4200	IT_PROG	Programmer

Lesson Agenda

- Working with Collections
- Collection exceptions
- Summarizing Collections
- PL/SQL Bind types

ORACLE



For Instructor Use Only.
This document should not be distributed.

Collection Exceptions

Common exceptions with collections:

- `COLLECTION_IS_NULL`
- `NO_DATA_FOUND`
- `SUBSCRIPT_BEYOND_COUNT`
- `SUBSCRIPT_OUTSIDE_LIMIT`
- `VALUE_ERROR`

ORACLE

In most cases, if you reference a nonexistent collection element, PL/SQL raises a predefined exception. The following table has a list of commonly raised exceptions, while using collections:

Exception	Raised when:
<code>COLLECTION_IS_NULL</code>	You try to operate on an atomically null collection.
<code>NO_DATA_FOUND</code>	A subscript designates an element that was deleted.
<code>SUBSCRIPT_BEYOND_COUNT</code>	A subscript exceeds the number of elements in a collection.
<code>SUBSCRIPT_OUTSIDE_LIMIT</code>	A subscript is outside the legal range.
<code>VALUE_ERROR</code>	A subscript is null or not convertible to an integer.

Collection Exceptions: Example

Common exceptions with collections:

```
DECLARE
  TYPE NumList IS TABLE OF NUMBER;
  nums NumList;          -- atomically null
BEGIN
  /* Assume execution continues despite the raised exceptions.
  */
  nums(1) := 1;           -- raises COLLECTION_IS_NULL
  nums := NumList(1,2);   -- initialize table
  nums(NULL) := 3         -- raises VALUE_ERROR
  nums(0) := 3;           -- raises SUBSCRIPT_OUTSIDE_LIMIT
  nums(3) := 3;           -- raises SUBSCRIPT_BEYOND_COUNT
  nums.DELETE(1);         -- delete element 1
  IF nums(1) = 1 THEN     -- raises NO_DATA_FOUND
  ...
```

ORACLE

The partial code in the slide illustrates various situations when an exception can be raised.

- `COLLECTION_IS_NULL` is raised when you are assigning a value to be stored at a given index. The collection here is `NULL` because in the declare section, you have not initialized the collection. Before assigning or updating the value, you have to initialize the collection. We initialize the collection correctly in the following statement using a constructor: **`nums := NumList(1,2);`**.
- A `VALUE_ERROR` exception is raised when you are trying to access a `NULL` index of the collection
- `SUBSCRIPT_OUTSIDE_LIMIT` exception is raised when you try to access a subscript that is outside the legal range.
- `SUBSCRIPT_BEYOND_COUNT` exception is raised because you are trying to access an index 3, whereas you have initialized the collection with only two values. Based on the initialization, you can have an index value of 2.
You can increase the size of the collection by using the `EXTEND` method.
- `NO_DATA_FOUND` exception is raised when you are trying to access an element that was deleted.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Working with Collections
- Collection exceptions
- Summarizing Collections
- PL/SQL Bind types



ORACLE

For Instructor Use Only.
This document should not be distributed.

Listing Characteristics for Collections

Collection Type	Number of Elements	Index Type	Dense or Sparse	Uninitialized status	Where defined	Can be ADT attribute data type
Associative array (index-by table)	Unspecified	String or PLS_INTEGER	Either	Empty	In PL/SQL block or package	No
VARRAY(variable-sized array)	Specified	Integer	Always dense	Null	In PL/SQL block or package or schema level	Only if defined at schema level
Nested table	Unspecified	Integer	Starts dense can become sparse	Null	In PL/SQL block or package or schema level	Only if defined at schema level

ORACLE

- Use associative arrays when you need:
 - To collect information of unknown volume
 - Flexible subscripts (negative, non sequential, or string based)
 - To pass the collection to and from the database server (Use associative arrays with bulk constructs.)
- Use nested tables when you need:
 - Persistence
 - To pass the collection as a parameter

Choosing Between Nested Tables and Varrays

- Use varrays when:
 - The number of elements is known in advance
 - The elements are usually all accessed in sequence
- Use nested tables when:
 - The index values are not consecutive
 - There is no predefined upper bound for the index values
 - You need to delete or update some, not all, elements simultaneously
 - You would usually create a separate lookup table with multiple entries for each row of the main table and access it through join queries

Lesson Agenda

- Working with Collections
- Collection exceptions
- Summarizing Collections
- PL/SQL Bind types



ORACLE

For Instructor Use Only.
This document should not be distributed.

PL/SQL Bind Types

- PL/SQL bind types refer to the data types that can be passed as parameters to PL/SQL program units while invoking them from dynamic SQL or clients such as JDBC or OCI.
- In 19c, you can use the following types as parameters while invoking PL/SQL program units:
 - Boolean
 - Records
 - Collections

ORACLE

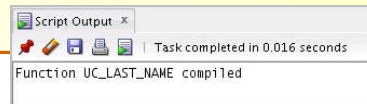
In 12.1, Oracle Database extended its support for PL/SQL-specific data types in SQL statements. Before Oracle Database 12c, values with PL/SQL-only data types (for example, `BOOLEAN`, associative array, and record) could not be bound from client programs (OCI or JDBC) or from static and native dynamic SQL issued from PL/SQL in the server. Whenever you had to bind a PL/SQL expression by using `EXECUTE IMMEDIATE` or `DBMS_SQL`, the data type of that expression had to be a known SQL data type. Specifically, you could not bind Boolean and user-defined types declared in a package specification, including records and collections.

Oracle Database 12c and later make it possible to bind values with PL/SQL-only data types to anonymous blocks (which are SQL statements), PL/SQL function calls in SQL queries and `CALL` statements, and the `TABLE` operator in SQL queries.

For Instructor Use Only.
This document should not be distributed.

Subprogram with a BOOLEAN Parameter

```
CREATE OR REPLACE FUNCTION uc_last_name (  
    employee_id_in    IN employees.employee_id%TYPE,  
    upper_in          IN BOOLEAN)  
RETURN employees.last_name%TYPE  
IS  
    l_return    employees.last_name%TYPE;  
BEGIN  
    SELECT last_name  
    INTO l_return  
    FROM employees  
    WHERE employee_id = employee_id_in;  
  
    RETURN CASE WHEN upper_in THEN UPPER (l_return) ELSE  
        l_return END;  
END;  
/
```



ORACLE

The slide shows a subprogram, which accepts a boolean parameter along with a number parameter.

The function returns the last name of the employee whose `employee_id` is passed as parameter to the function `uc_last_name`.

Based on the boolean value passed as parameter, the `last_name` value is returned either in uppercase or lowercase.

For Instructor Use Only.
This document should not be distributed.

Subprogram with a BOOLEAN Parameter

```
DECLARE
  b BOOLEAN := TRUE;
BEGIN
  FOR rec IN (SELECT uc_last_name(employee_id, b) lname
              FROM employees
              WHERE department_id = 10)
  LOOP
    DBMS_OUTPUT.PUT_LINE (rec.lname);
  END LOOP;
END;
/
```

```
PL/SQL procedure successfully completed.
WHALEN
```

ORACLE

The code shown in the slide invokes the `uc_last_name` function in a SQL query. The boolean value, which is passed as parameter to the function, is initialized in the PL/SQL block.

This was not possible in earlier versions of Oracle Database. Oracle Database 12c and later allow more PL/SQL data types to cross PL/SQL to SQL interface.

Following is the code that allows you to pass records as parameters to the PL/SQL Program units:

```
CREATE OR REPLACE PACKAGE names_pkg AUTHID CURRENT_USER
AS
  TYPE names_t
  IS TABLE OF VARCHAR2 (100)
  INDEX BY PLS_INTEGER;

  PROCEDURE display_names (
    names_in  IN names_t);
END names_pkg;
/
```

Subprogram with a RECORD Parameter (continued)

```
CREATE OR REPLACE PACKAGE BODY names_pkg
AS
    PROCEDURE display_names (
        names_in    IN names_t)
    IS
    BEGIN
        FOR indx IN 1 .. names_in.COUNT
        LOOP
            DBMS_OUTPUT.put_line (
                names_in (indx));
        END LOOP;
    END;
END names_pkg;
/
DECLARE
    l_names    names_pkg.names_t;
BEGIN
    l_names (1) := 'Loey';
    l_names (2) := 'Dylan';
    l_names (3) := 'Indigo';
    l_names (4) := 'Saul';
    l_names (5) := 'Sally';
    EXECUTE IMMEDIATE
        'BEGIN names_pkg.display_names (:names); END;'
        USING l_names;
    FOR rec
        IN (SELECT * FROM TABLE (l_names))
    LOOP
        DBMS_OUTPUT.put_line (
            rec.COLUMN_VALUE);
    END LOOP;
END;
```

For Instructor Use Only.
This document should not be distributed.

Quiz

Q

Which of the following collection method is used for traversing a collection?

- a. EXISTS
- b. COUNT
- c. LIMIT
- d. FIRST

?

ORACLE

Answer: d

For Instructor Use Only.
This document should not be distributed.

Quiz

Q

A PL/SQL anonymous block, a SQL CALL statement, or a SQL query can invoke a PL/SQL function that has parameters of the type:

- a. Boolean
- b. Record declared in the package specification
- c. Collection declared in the package specification
- d. All of the above

?

ORACLE

Answer: d

For Instructor Use Only.
This document should not be distributed.

Summary

In this lesson, you should have learned how to:

- Access collection elements
- Use collection methods in PL/SQL
- Identify raised exceptions with collections
- Decide which collection type is appropriate for each scenario



ORACLE

Collections are a grouping of elements, all of the same type. The types of collections are nested tables, varrays, and associative arrays. You can define nested tables and varrays in the database. Nested tables, varrays, and associative arrays can be used in a PL/SQL program.

When using collections in PL/SQL programs, you can access the collection elements, use predefined collection methods, and use the exceptions that are commonly encountered with collections.

There are guidelines for using collections effectively and for determining which collection type is appropriate under specific circumstances.

For Instructor Use Only.
This document should not be distributed.

Practice 4: Overview

This practice covers using collections.

The Oracle logo, consisting of the word "ORACLE" in white capital letters on a red rectangular background.

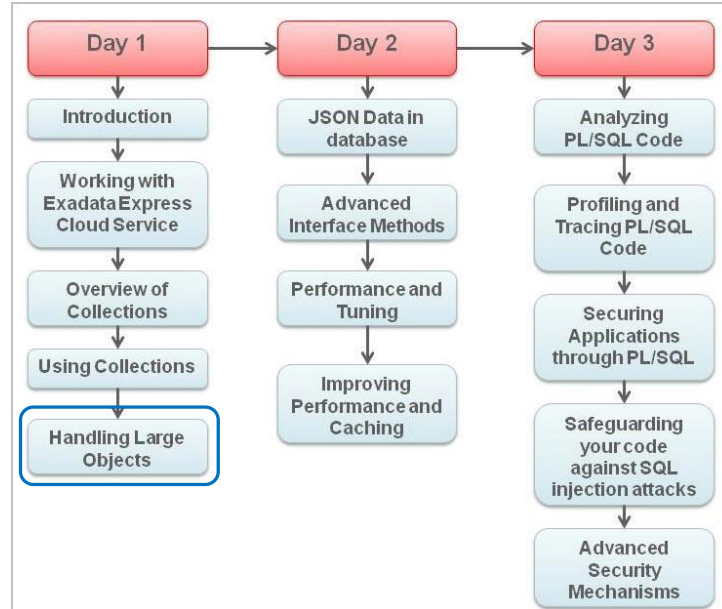
In this practice, you write a PL/SQL package to manipulate the collection.
Use the `OE` schema for this practice.

For Instructor Use Only.
This document should not be distributed.

Handling Large Objects

ORACLE

Course Agenda



ORACLE

For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to do the following:

- Create and maintain `LOB` data types
- Differentiate between internal and external `LOB`s
- Use the `DBMS_LOB` package
- Describe the use of temporary `LOB`s
- Describe SecureFile `LOB`



ORACLE

This lesson describes the characteristics of the `LOB` data types. Examples, syntax, and issues regarding the `LOB` types are also presented.

You will learn about various types of `LOB` data types and packages, which support manipulating `LOB`s.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Introduction to LOBs
- Using DBMS_LOB package
- Working with BFILEs
- Working with CLOBs
- Reading LOBs from the database
- Using temporary LOBs
- Using SecureFile LOBs

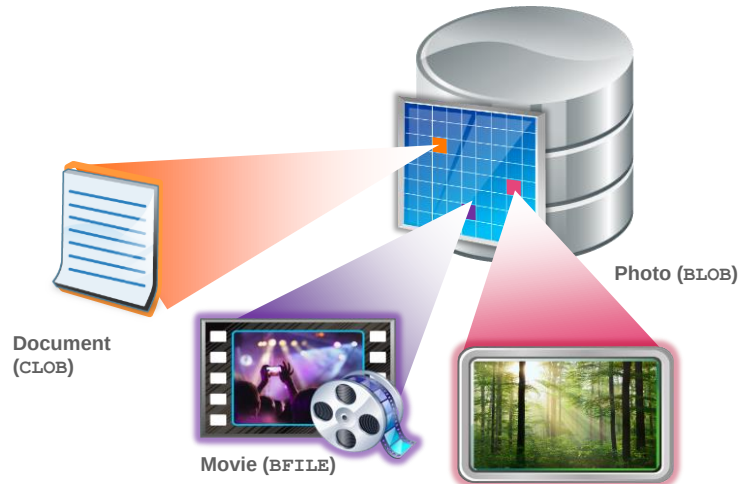


ORACLE

For Instructor Use Only.
This document should not be distributed.

What Is a LOB?

- Large Objects (LOBs) refer to a set of data types, which are designed to hold large amounts of data.
- There are four LOB data types:
 - BLOB
 - CLOB
 - NCLOB
 - BFILE



ORACLE

A LOB data type is used to store large volumes of data. You can store structured, semi-structured, or unstructured data.

You can use LOB data types to define column types, which can store documents, videos, and so on.

There are four large object data types:

- **BLOB** represents a binary large object, such as a video clip or a picture.
- **CLOB** represents a character large object.
- **NCLOB** represents a multiple-byte character large object.
- **BFILE** represents a binary file stored in an OS binary file outside the database. The **BFILE** column or attribute stores a file locator that points to the external file.

For Instructor Use Only.
This document should not be distributed.

Types of LOBs

Based on the physical location of the large object, there are two types of LOBs:

- Internal LOBs
- External LOBs



ORACLE

You can store LOBs either in the tablespace of the database or external to the database in the operating system's file system.

Based on the actual location of the large object, there are two types of large objects – Internal large objects and External large objects.

Internal LOBs are stored in tablespaces within the database. The SQL data types `BLOB`, `CLOB`, and `NCLOB` are used for supporting internal large objects.

External LOBs are stored in the operating system files, outside the database table spaces. `BFILE` is the data type used to support external large objects.

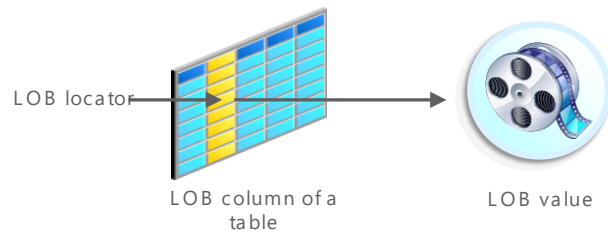
You can define a column type of LOBs in tables for both internal and external LOBs.

Differences between Internal and External LOBs:

Internal LOBs	External LOBs
LOB values are stored in the tablespace of the database.	LOB values are stored as operating system files outside the database.
CLOB, BLOB, and NCLOB types are used for internal LOBs.	BFILE type is used for external LOBs.
Internal LOBs can be part of transactions.	External LOBs cannot be part of transactions.
In case of an internal LOB, there is a distinct locator for each value.	In case of an external LOB, multiple LOB locators can refer to the same LOB.

LOB Locators and LOB Values

- There are two components for every LOB instance:
 - LOB locator
 - LOB value
- LOB locator is a reference to the LOB value.
- A LOB type column in a table has LOB locators.
- LOB value is the actual LOB data, which can be stored externally or internally to the database.



ORACLE

There are two parts to a LOB:

- **LOB value:** The data that constitutes the real object being stored
- **LOB locator:** A pointer to the location of the LOB value that is stored in the database

Irrespective of the type of the LOB, a LOB locator is stored for value in a LOB data type column. You can think of a LOB locator as a pointer to the actual location of the LOB value.

A LOB column does not contain the data; it contains the locator of the LOB value.

When a user creates an internal LOB, the value is stored in the LOB segment and a locator to the out-of-line LOB value is placed in the LOB column of the corresponding row in the table. External LOBs store data outside the database, so only a locator to the LOB value is stored in the table.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Introduction to LOBs
- **Using the DBMS_LOB package**
- Working with BFILES
- Working with CLOBs
- Reading LOBs from the database
- Using temporary LOBs
- Using SecureFile LOBs

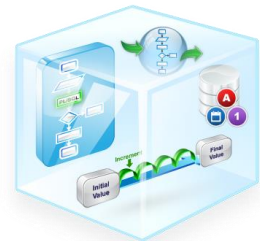


ORACLE

For Instructor Use Only.
This document should not be distributed.

DBMS_LOB Package

- DBMS_LOB is an Oracle supplied package.
- It has subprograms, which can read and modify LOBs.
- DBMS_LOB subprograms perform operations based on LOB locators.



ORACLE

The DBMS_LOB package has subprograms for working with LOBs. You can read and modify the BLOB, CLOB, and NCLOB types of LOBs through the subprograms defined in the DBMS_LOB package. It has read-only operations for BFILES.

The DBMS_LOB functions and procedures can be broadly classified into two types: mutators and observers:

- **The mutators can modify LOB values:** APPEND, COPY, ERASE, TRIM, WRITE, FILECLOSE, FILECLOSEALL, and FILEOPEN.
- **The observers can read LOB values:** COMPARE, FILEGETNAME, INSTR, GETLENGTH, READ, SUBSTR, FILEEXISTS, and FILEISOPEN.

DBMS_LOB provides routines to access and manipulate internal and external LOBs:

- **Modify LOB values:** APPEND, COPY, ERASE, TRIM, WRITE, LOADFROMFILE
- **Read or examine LOB values:** GETLENGTH, INSTR, READ, SUBSTR
- **Specific to BFILES:** FILECLOSE, FILECLOSEALL, FILEEXISTS, FILEGETNAME, FILEISOPEN, FILEOPEN

For Instructor Use Only.
This document should not be distributed.

Security Model of the DBMS_LOB Package

- The DBMS_LOB package is created as a SYS user.
- You can use the subprograms as a non-SYS user with applicable access rights.
- You can modify BLOBs, CLOBs, and NCLOBs, but not BFILES.
- You can access BFILES through a DIRECTORY object.

ORACLE

A DBMS_LOB subprogram called from an anonymous PL/SQL block is executed with the privileges of the current user.

A DBMS_LOB subprogram invoked from a stored procedure is executed with the privileges of the owner of the stored procedure.

For Instructor Use Only.
This document should not be distributed.

What Is a DIRECTORY Object?

- Enables secure access to the BFILES
- Specifies a logical alias name for a physical directory on the operating system's file system
- Provides the flexibility to manage the locations of the BFILES without hard-coding the absolute path
- Is created by administrators or users with CREATE DIRECTORY privileges

ORACLE

A DIRECTORY is a nonschema database object that enables the administration of access and usage of BFILES in Oracle Database. To associate an OS file with a BFILE, you should first create a DIRECTORY object that is an alias for the full path name to the OS file.

The DIRECTORY object provides the flexibility to manage the locations of the files, instead of forcing you to hard-code the absolute path names of physical files in your applications.

The DIRECTORY object is created by the DBA (or a user with the CREATE ANY DIRECTORY privilege). The privileges may differ from those defined for the DIRECTORY object and could change after creation of the DIRECTORY object. You can create DIRECTORY objects by using the following guidelines:

- Directories should point to paths that do not contain database files, because tampering with these files can corrupt the database.
- The CREATE ANY DIRECTORY and DROP ANY DIRECTORY system privileges should be used carefully and not granted to users indiscriminately.
- DIRECTORY objects are not schema objects; all are owned by SYS.
- Create the directory paths with appropriate permissions on the OS before creating the DIRECTORY object. Oracle does not create the OS path.
- If you migrate the database to a different OS, you may have to change the path value of the DIRECTORY object.

Information about the DIRECTORY object that you create by using the CREATE DIRECTORY command is stored in the DBA_DIRECTORIES and ALL_DIRECTORIES data dictionary views.

For Instructor Use Only.
This document should not be distributed.

Managing BFILES: Role of a DBA

The DBA or the system administrator:

1. Creates an OS directory and supplies files
2. Creates a `DIRECTORY` object in the database
3. Grants the `READ` privilege on the `DIRECTORY` object to appropriate database users



ORACLE

Managing BFILES requires coordination between the database administrator and the system administrator, and then between the developer and the user of the files.

The database or system administrator must perform the following privileged tasks:

1. Create the operating system (OS) directory (as an Oracle user), and set permissions so that the Oracle server can read the contents of the OS directory. Load files into the OS directory.
2. Create a database `DIRECTORY` object that references the OS directory.
3. Grant the `READ` privilege on the database `DIRECTORY` object to the database users that require access to it.

For Instructor Use Only.
This document should not be distributed.

Managing BFILES: Role of a Developer

The developer or the user:

1. Creates an Oracle table with a column that is defined as a `BFILE` data type
2. Inserts rows into the table by using the `BFILENAME` function to populate the `BFILE` column
3. Writes a PL/SQL subprogram that declares and initializes a `LOB` locator, and reads `BFILE`



ORACLE

The designer, application developer, or user must perform the following tasks:

1. Create a database table containing a column that is defined as the `BFILE` data type.
2. Insert rows into the table by using the `BFILENAME` function to populate the `BFILE` column, associating the `BFILE` field to an OS file in the named `DIRECTORY`.
3. Write PL/SQL subprograms that:
 - a. Declare and initialize the `BFILE` `LOB` locator
 - b. Select the row and column containing the `BFILE` in the `LOB` locator
 - c. Read the `BFILE` with a `DBMS_LOB` function by using the locator file reference

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Introduction to LOBs
- Using DBMS_LOB package
- **Working with BFILEs**
- Working with CLOBs
- Reading LOBs from the database
- Using temporary LOBs
- Using SecureFile LOBs



ORACLE

For Instructor Use Only.
This document should not be distributed.

Working on BFILES

To use BFILES in your application, you usually follow this workflow:

1. Create a DIRECTORY object as the SYSDBA user.
2. Grant access to the application user to access the DIRECTORY object.
3. Create and initialize a BFILE column in the table as the application user (non-SYSDBA user).
4. Populate and access the data in BFILE column.

ORACLE

To use BFILES in your application, you have to perform certain administrator tasks.

1. You have to create a DIRECTORY object in the database. You create the DIRECTORY object as a SYSDBA user or any other user who has CREATE DIRECTORY privileges.
2. After you create the DIRECTORY object, you have to grant the application user (OE in this case) access to the DIRECTORY object.
3. Connect as the application user (OE) and add BFILE column to table.
4. You populate the BFILE column by loading the files from the directory. You can use the subprograms defined in the DBMS_LOB package for loading data into the BFILE column.

You can now use the BFILE column for various operations in the application. We will look into the implementation of these steps in the following slides.

For Instructor Use Only.
This document should not be distributed.

Preparing to Use BFILES

1. Create an OS directory to store the physical data files:

```
mkdir /home/oracle/labs/DATA_FILES/MEDIA_FILES
```

2. Create a DIRECTORY object by using the CREATE DIRECTORY command:

```
CREATE OR REPLACE DIRECTORY data_files AS  
'/home/oracle/labs/DATA_FILES/MEDIA_FILES';
```

3. Grant the READ privilege on the DIRECTORY object to appropriate users:

```
GRANT READ ON DIRECTORY data_files TO OE;
```

ORACLE

To use a BFILE within an Oracle table, you must have a table with a column of the BFILE data type. For the Oracle server to access an external file, the server must know the physical location of the file in the OS directory structure.

The database DIRECTORY object provides the means to specify the location of the BFILES. Use the CREATE DIRECTORY command to specify the pointer to the location where your BFILES are stored. You must have the CREATE ANY DIRECTORY privilege.

Syntax definition: CREATE DIRECTORY *dir_name* AS *os_path*;

In this syntax, *dir_name* is the name of the directory database object, and *os_path* specifies the location of the BFILES.

The slide examples show the commands to set up:

- The physical directory (for example, /temp/data_files) in the OS
- A named DIRECTORY object, called data_files, that points to the physical directory in the OS
- The READ access right on the directory to be granted to users in the database that provides the privilege to read the BFILES from the directory

Note: The value of the SESSION_MAX_OPEN_FILES database initialization parameter, which is set to 10 by default, limits the number of BFILES that can be opened in a session.

For Instructor Use Only.
This document should not be distributed.

Creating BFILE Columns in the Table

- Use the `BFILENAME` function to initialize a `BFILE` column. The function syntax is:

```
FUNCTION BFILENAME(directory_alias IN VARCHAR2,  
                  filename IN VARCHAR2)  
RETURN BFILE;
```

- Example:

- Add a `BFILE` column to a table:

```
ALTER TABLE customers ADD video BFILE;
```

- Update the column by using the `BFILENAME` function:

```
UPDATE customers  
SET video = BFILENAME('DATA_FILES', 'Winters.avi')  
WHERE customer_id = 448;
```

ORACLE

The `BFILENAME` function is a built-in function that you use to initialize a `BFILE` column, by using the following two parameters:

- *directory_alias* for the name of the database `DIRECTORY` object that references the OS directory containing the files
- *filename* for the name of the `BFILE` to be read

It creates a pointer (or `LOB` locator) to the external file stored in a physical directory and assigns it to *directory_alias*.

You can populate the `BFILE` column by using the `BFILENAME` function in either of the following:

- The `VALUES` clause of an `INSERT` statement
- The `SET` clause of an `UPDATE` statement

You can use an `UPDATE` operation to change the pointer reference target of the `BFILE`. You can also initialize `BFILE` column to a `NULL` value and update later.

After the `BFILE` columns are associated with a file, subsequent read operations on the `BFILE` can be performed by using the PL/SQL `DBMS_LOB` package and OCI.

For Instructor Use Only.
This document should not be distributed.

Populating a BFILE Column with PL/SQL

```
CREATE OR REPLACE PROCEDURE set_video(  
  dir_alias VARCHAR2, custid NUMBER) IS  
  filename VARCHAR2(40);  
  file_ptr BFILE;  
  CURSOR cust_csr IS  
    SELECT cust_first_name FROM customers  
    WHERE customer_id = custid FOR UPDATE;  
BEGIN  
  FOR rec IN cust_csr LOOP  
    filename := rec.cust_first_name || '.gif';  
    file_ptr := BFILENAME(dir_alias, filename);  
    DBMS_LOB.FILEOPEN(file_ptr);  
    UPDATE customers SET video = file_ptr  
      WHERE CURRENT OF cust_csr;  
    DBMS_OUTPUT.PUT_LINE('FILE: ' || filename ||  
      ' SIZE: ' || DBMS_LOB.GETLENGTH(file_ptr));  
    DBMS_LOB.FILECLOSE(file_ptr);  
  END LOOP;  
END set_video;
```

ORACLE

The slide example demonstrates populating the BFILE column. It is a PL/SQL procedure called `set_video`, which accepts the name of the directory, and a customer ID. The procedure performs the following tasks:

- Uses a cursor `FOR` loop to obtain each customer record
- Sets the `filename` by appending `.gif` to the customer's `first_name`
- Creates a `LOB` locator for the `BFILE` in the `file_ptr` variable
- Calls the `DBMS_LOB.FILEOPEN` procedure to verify whether the file exists, and to determine the size of the file by using the `DBMS_LOB.GETLENGTH` function
- Executes an `UPDATE` statement to write the `BFILE` locator value to the `video` `BFILE` column
- Displays the file size returned from the `DBMS_LOB.GETLENGTH` function
- Closes the file by using the `DBMS_LOB.FILECLOSE` procedure

Suppose that you execute the following call:

```
EXECUTE set_video('DATA_FILES', 844)
```

The sample result is:

```
FILE: Alice.gif SIZE: 2619802
```

For Instructor Use Only.
This document should not be distributed.

Using Data in the BFILE Column

The `DBMS_LOB.FILEEXISTS` function can check whether the file exists in the OS. The function returns:

- 0 if the file does not exist
- 1 if the file does exist

```
CREATE OR REPLACE FUNCTION get_filesize(p_file_ptr IN
OUT BFILE)
RETURN NUMBER IS
    v_file_exists BOOLEAN;
    v_length NUMBER := -1;
BEGIN
    v_file_exists := DBMS_LOB.FILEEXISTS(p_file_ptr) = 1;
    IF v_file_exists THEN
        DBMS_LOB.FILEOPEN(p_file_ptr);
        v_length := DBMS_LOB.GETLENGTH(p_file_ptr);
        DBMS_LOB.FILECLOSE(p_file_ptr);
    END IF;
    RETURN v_length;
END;
/
```

ORACLE

The `set_video` procedure on the previous page terminates with an exception if a file does not exist. To prevent the loop from prematurely terminating, you could create a function, such as `get_filesize`, to determine whether a given BFILE locator references a file that actually exists on the server's file system. The `DBMS_LOB.FILEEXISTS` function accepts the BFILE locator as a parameter and returns an INTEGER with:

- A value 0 if the physical file does not exist
- A value 1 if the physical file exists

If the BFILE parameter is invalid, one of the following three exceptions may be raised:

- `NOEXIST_DIRECTORY` if the directory does not exist
- `NOPRIV_DIRECTORY` if the database processes do not have privileges for the directory
- `INVALID_DIRECTORY` if the directory was invalidated after the file was opened

In the `get_filesize` function, the output of the `DBMS_LOB.FILEEXISTS` function is compared with the value 1 and the result of the condition sets the BOOLEAN variable `file_exists`. The `DBMS_LOB.FILEOPEN` call is performed only if the file exists, thereby preventing unwanted exceptions from occurring. The `get_filesize` function returns a value of -1 if a file does not exist; otherwise, it returns the size of the file in bytes. The caller can take appropriate action with this information.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Introduction to LOBs
- Using DBMS_LOB package
- Working with BFILEs
- **Working with CLOBs**
- Reading LOBs from the database
- Using temporary LOBs
- Using SecureFile LOBs



ORACLE

For Instructor Use Only.
This document should not be distributed.

Working on CLOBs

To use CLOBs in your application, you usually follow this workflow:

1. Create a tablespace as a SYSDBA user.
2. Create a CLOB column in the table as the application user (non-SYSDBA).
3. Initialize the CLOB columns.
4. Populate and access the data in CLOB columns.
5. Write data to a CLOB column.

ORACLE

You can store BLOBs, CLOBs, and NCLOBs in the database as internal LOBs. Internal LOBs are stored in the tablespace within the database.

1. To have an internal LOB column in a table, you have to define the tablespace first. You can define the tablespace as the SYSDBA user.
2. After you have the tablespace defined, you can create internal LOB columns in the database table.
3. Initialize or populate the columns.
4. Load data into the LOB columns.
5. Perform operations on the B/CLOB columns.

BFILEs are read-only files; therefore, you cannot update the values in the BFILE columns. You can perform write and update operations on the internal LOB columns. We will look into the implementation of these steps in the following slides.

For Instructor Use Only.
This document should not be distributed.

Initializing the LOB Columns Added to a Table

- Add the LOB columns to an existing table by using ALTER TABLE:

```
ALTER TABLE customers
  ADD (resume CLOB, picture BLOB);
```

- Create a tablespace where you will put a new table with the LOB columns:

```
CREATE TABLESPACE lob_tbs1
  DATAFILE 'lob_tbs1.dbf' SIZE 800M REUSE
  EXTENT MANAGEMENT LOCAL
  UNIFORM SIZE 64M
  SEGMENT SPACE MANAGEMENT AUTO;
```

ORACLE

You can create an internal LOB type column in a table either while creating the table or by using the ALTER TABLE command. The example in the slide is adding two LOB columns to the table customers. You can also add LOB columns to a table while creating the table by using the CREATE command.

The internal LOBs are stored in a tablespace within the database. Therefore, while creating columns of internal LOB data types, it is recommended that you create a tablespace first. You have to create the tablespace as a SYSDBA user.

For Instructor Use Only.
This document should not be distributed.

Initializing the LOB Columns Added to a Table

Initialize the column LOB locator value with the `DEFAULT` option or the DML statements by using:

- The `EMPTY_CLOB()` function for a CLOB column
- The `EMPTY_BLOB()` function for a BLOB column

```
CREATE TABLE customer_profiles (  
  id NUMBER,  
  full_name VARCHAR2(45),  
  resume CLOB DEFAULT EMPTY_CLOB(),  
  picture BLOB DEFAULT EMPTY_BLOB()  
  LOB(picture) STORE AS BASICFILE  
  (TABLESPACE lob_tbs1);
```

ORACLE

The contents of a LOB column are stored in the LOB segment, whereas the column in the table contains only a reference to that specific storage area called the LOB locator. In PL/SQL, you can define a variable of the LOB type, which contains only the value of the LOB locator. You can initialize the LOB locators by using the following functions:

- `EMPTY_CLOB()` function to a LOB locator for a CLOB column
- `EMPTY_BLOB()` function to a LOB locator for a BLOB column

These functions create the LOB locator pointing to an empty LOB. You can use the `DBMS_LOB` package subroutines to populate the content.

The example in the slide shows that you can use the `EMPTY_CLOB()` and `EMPTY_BLOB()` functions in the `DEFAULT` option in a `CREATE TABLE` statement. Thus, the LOB locator values are populated in their respective columns when a row is inserted into the table and the LOB columns were not specified in the `INSERT` statement.

The `CUSTOMER_PROFILES` table is created. The `PICTURE` column holds the LOB data in the BasicFile format, because the storage clause identifies the format.

For Instructor Use Only.
This document should not be distributed.

Populating LOB Columns

- Insert a row into a table with LOB columns:

```
INSERT INTO customer_profiles
(id, full_name, resume, picture)
VALUES (164, 'Charlotte Kazan', EMPTY_CLOB(), NULL);
. . .
```

- Initialize a LOB by using the EMPTY_BLOB() function:

```
UPDATE customer_profiles
SET resume = 'Date of Birth: 8 February 1951',
    picture = EMPTY_BLOB()
WHERE id = 164;
```

- Update a CLOB column:

```
UPDATE customer_profiles
SET resume = 'Date of Birth: 1 June 1956'
WHERE id = 150;
```

ORACLE

You can use the special `EMPTY_BLOB()` and `EMPTY_CLOB()` functions in `INSERT` or `UPDATE` statements of SQL DML to initialize a `NULL` or non-`NULL` internal LOB to empty. To populate a LOB column, perform the following steps:

1. Initialize the LOB column to a non-`NULL` value—that is, set a LOB locator pointing to an empty or populated LOB value. This is done by using the `EMPTY_BLOB()` and `EMPTY_CLOB()` functions.
2. Populate the LOB contents by using the `DBMS_LOB` package routines.

However, as shown in the slide examples, the two `UPDATE` statements initialize the `resume` LOB locator value and populate its contents by supplying a literal value. This can also be done in an `INSERT` statement. A LOB column can be updated to:

- Another LOB value
- A `NULL` value
- A LOB locator with empty contents by using the `EMPTY_*` LOB() built-in function

You can update the LOB by using a bind variable in embedded SQL. When assigning one LOB to another, a new copy of the LOB value is created. Use a `SELECT FOR UPDATE` statement to lock the row containing the LOB column before updating a piece of the LOB contents.

For Instructor Use Only.
This document should not be distributed.

Loading Data to a LOB Column

Create the procedure to read MS Word files and load them into the LOB column.

```
CREATE OR REPLACE PROCEDURE loadLOBfromBFILE_proc
(p_dest_loc IN OUT BLOB, p_file_name IN VARCHAR2,
 p_file_dir IN VARCHAR2)
IS
  v_src_loc BFILE := BFILENAME(p_file_dir, p_file_name);
  v_amount  INTEGER := 4000;
  offset    INTEGER := 1;
BEGIN
  DBMS_LOB.OPEN(v_src_loc, DBMS_LOB.LOB_READONLY);
  v_amount := DBMS_LOB.GETLENGTH(v_src_loc);
  DBMS_LOB.LOADBLOBFROMFILE(p_dest_loc, v_src_loc,
    v_amount, offset, offset);
  DBMS_LOB.CLOSE(v_src_loc);
END loadLOBfromBFILE_proc;
```

ORACLE

The procedure shown in the slide is used to load data into the LOB column.

Before running the `LOADLOBFROMBFILE_PROC` procedure, you must set a directory object that identifies where the LOB files are stored externally. In this example, the Microsoft Word documents are stored in the `DATA_FILES` directory that was created earlier in this lesson.

The `LOADBLOBFROMFILE` procedure is used to read the LOB data into the `PICTURE` column in the `CUSTOMER_PROFILES` table.

In this example:

- `DBMS_LOB.OPEN` is used to open an external LOB in read-only mode.
- `DBMS_LOB.GETLENGTH` is used to find the length of the LOB value.
- `DBMS_LOB.LOADBLOBFROMFILE` is used to load the data from a file into an internal LOB.
- `DBMS_LOB.CLOSE` is used to close the external LOB.

For Instructor Use Only.
This document should not be distributed.

Writing Data to a LOB

Create the procedure to insert LOBs into the table:

```
CREATE OR REPLACE PROCEDURE write_lob
(p_file IN VARCHAR2, p_dir IN VARCHAR2)
IS
  i      NUMBER;          v_fn VARCHAR2(15);
  v_ln   VARCHAR2(40);    v_b   BLOB;
BEGIN
  DBMS_OUTPUT.ENABLE;
  DBMS_OUTPUT.PUT_LINE('Begin inserting rows...');
  FOR i IN 1 .. 30 LOOP
    v_fn:=SUBSTR(p_file,1,INSTR(p_file,'.')-1);
    v_ln:=SUBSTR(p_file,INSTR(p_file,'.')+1,LENGTH(p_file)-
                INSTR(p_file,'.')-4);
    INSERT INTO customer_profiles
      VALUES (i, v_fn, v_ln, EMPTY_BLOB())
      RETURNING picture INTO v_b;
    loadLOBFromBFILE_proc(v_b,p_file, p_dir);
    DBMS_OUTPUT.PUT_LINE('Row ' || i || ' inserted.');
```

```
  END LOOP;
  COMMIT;
END write_lob;
```

ORACLE

You can initialize a BLOB column value by using the `EMPTY_BLOB()` function as a default predicate. The code shown in the slide uses the `INSERT` statement to initialize the locator. The `LOADLOBFROMBFILE` routine is then called and the LOB column value is inserted.

For Instructor Use Only.
This document should not be distributed.

Writing Data to a LOB

```
CREATE OR REPLACE DIRECTORY resume_files AS  
'/home/oracle/labs/DATA_FILES/RESUMES';
```

```
set serveroutput on  
set verify on  
set term on
```

```
execute write_lob('karl.brimmer.doc', 'RESUME_FILES')  
execute write_lob('monica.petera.doc', 'RESUME_FILES')  
execute write_lob('david.sloan.doc', 'RESUME_FILES')
```

ORACLE

1. The Microsoft Word files are stored in the `/home/oracle/labs/DATA_FILES/RESUMES` directory.
2. To read them into the `PICTURE` column in the `CUSTOMER_PROFILES` table, the `WRITE_LOB` procedure is called and the name of the `.doc` files is passed as a parameter.

For Instructor Use Only.
This document should not be distributed.

The output is similar to the following:

```
execute write_lob('karl.brimmer.doc', 'RESUME_FILES');
Begin inserting rows...
Row 1 inserted.
...
PL/SQL procedure successfully completed.

execute write_lob('monica.petera.doc', 'RESUME_FILES');
Begin inserting rows...
Row 1 inserted.
...
PL/SQL procedure successfully completed.

execute write_lob('david.sloan.doc', 'RESUME_FILES');
Begin inserting rows...
Row 1 inserted.
...
PL/SQL procedure successfully completed.
```

Lesson Agenda

- Introduction to LOBs
- Using DBMS_LOB package
- Working with BFILEs
- Working with CLOBs
- Reading LOBs from the database
- Using temporary LOBs
- Using SecureFile LOBs



ORACLE

For Instructor Use Only.
This document should not be distributed.

Reading LOBs from the Table

```
CREATE OR REPLACE PROCEDURE lob_txt(file_name VARCHAR2,p_dir VARCHAR2 DEFAULT 'PLSQL_DIR')
IS
c CLOB:=null;
byte_count pls_integer;
fil BFILE:=BFILENAME(p_DIR,file_name);
v_dest_offset integer:=1;
v_src_offset integer:=1;
v_lang_context integer:=0;
v_warning integer;
BEGIN
c:=TO_CLOB(' ');
IF DBMS_LOB.FILEEXISTS(FIL)=1 then
    DBMS_LOB.FILEOPEN(fil,DBMS_LOB.FILE_READONLY);
    byte_count:=DBMS_LOB.GETLENGTH(fil);
    DBMS_OUTPUT.PUT_LINE('The length of the file:'||byte_count);
    DBMS_LOB.LOADCLOBFROMFILE
    (dest_lob => c,src_bfile => fil,amount => byte_count,dest_offset => v_dest_offset
    ,src_offset => v_src_offset,bfile_csid => 0,lang_context => v_lang_context
    ,warning => v_warning);
    DBMS_LOB.FILECLOSEALL;
    INSERT INTO lob_text VALUES (lob_seq.nextval,c);
    COMMIT;
ELSE
    DBMS_OUTPUT.PUT_LINE('The file does not exist ');
END IF;
END;
/
```

ORACLE

The procedure given in the slide uploads a text file into a CLOB type column in a table.

To retrieve the records that were inserted, you can call the `LOB_TXT` procedure, For example to retrieve the record for the file 'java.sql':

```
SET LONG 10000
set serveroutput on
exec lob_txt('java.sql')
select KEY,txt from lob_text;
```

For Instructor Use Only.
This document should not be distributed.

Updating LOB by Using DBMS_LOB in PL/SQL

```
DECLARE
  v_lobloc CLOB;      -- serves as the LOB locator
  v_text   VARCHAR2(50) := 'Resigned = 5 June 2000';
  v_amount NUMBER;    -- amount to be written
  v_offset INTEGER;   -- where to start writing
BEGIN
  SELECT resume INTO v_lobloc FROM customer_profiles
  WHERE id = 164 FOR UPDATE;
  v_offset := DBMS_LOB.GETLENGTH(v_lobloc) + 2;
  v_amount := length(v_text);
  DBMS_LOB.WRITE (v_lobloc, v_amount, v_offset, v_text);
  v_text := ' Resigned = 30 September 2000';
  SELECT resume INTO v_lobloc FROM customer_profiles
  WHERE id = 150 FOR UPDATE;
  v_amount := length(v_text);
  DBMS_LOB.WRITEAPPEND(v_lobloc, v_amount, v_text);
  COMMIT;
END;
```

ORACLE

In the example in the slide, the `v_lobloc` variable serves as the LOB locator, and the `amount` variable is set to the length of the text that you want to add. The `SELECT FOR UPDATE` statement locks the row and returns the LOB locator for the `RESUME LOB` column. Finally, the PL/SQL `WRITE` package procedure is called to write the text into the LOB value at the specified offset. `WRITEAPPEND` procedure in the `DBMS_LOB` appends to the existing LOB value.

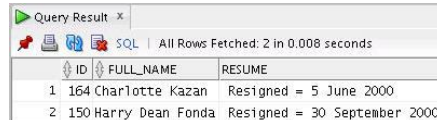
For Instructor Use Only.
This document should not be distributed.

Selecting CLOB Values by Using SQL

- Query:

```
SELECT id, full_name , resume -- CLOB
FROM customer_profiles
WHERE id IN (164, 150);
```

- Output in SQL Developer:



The screenshot shows a 'Query Result' window in SQL Developer. It displays the results of the SQL query. The window title is 'Query Result x'. Below the title bar, there are icons for saving, refreshing, and other actions, followed by the text 'SQL | All Rows Fetched: 2 in 0.008 seconds'. The main area shows a table with three columns: 'ID', 'FULL_NAME', and 'RESUME'. There are two rows of data.

	ID	FULL_NAME	RESUME
1	164	Charlotte Kazan	Resigned = 5 June 2000
2	150	Harry Dean Fonda	Resigned = 30 September 2000

ORACLE

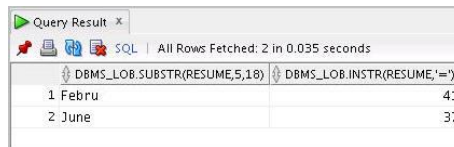
It is possible to see the data in a CLOB column by using a SELECT statement. It is not possible to see the data in a BLOB or BFILE column by using a SELECT statement in SQL*Plus. You must use a tool that can display the binary information for a BLOB, as well as the relevant software for a BFILE—for example, you can use Oracle Forms.

Selecting CLOB Values by Using DBMS_LOB

- `DBMS_LOB.SUBSTR(lob, amount, start_pos)`
- `DBMS_LOB.INSTR(lob, pattern)`

```
SELECT DBMS_LOB.SUBSTR (resume, 5, 18),  
       DBMS_LOB.INSTR (resume, ' = ')  
FROM   customer_profiles  
WHERE  id IN (150, 164);
```

- SQL Developer



Query Result x

SQL | All Rows Fetched: 2 in 0.035 seconds

DBMS_LOB.SUBSTR(RESUME,5,18)	DBMS_LOB.INSTR(RESUME,' = ')
1 Febru	41
2 June	37

ORACLE

DBMS_LOB.SUBSTR

Use `DBMS_LOB.SUBSTR` to display part of a `LOB`. It is similar in functionality to the `SUBSTR` SQL function.

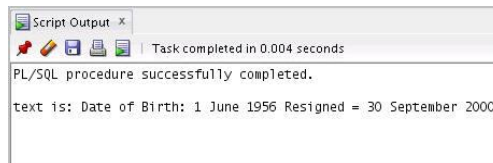
DBMS_LOB.INSTR

Use `DBMS_LOB.INSTR` to search for information within the `LOB`. This function returns the numerical position of the information.

Selecting CLOB Values in PL/SQL

```
SET LINESIZE 50 SERVEROUTPUT ON FORMAT WORD_WRAP

DECLARE
    text VARCHAR2(4001);
BEGIN
    SELECT resume INTO text
    FROM customer_profiles
    WHERE id = 150;
    DBMS_OUTPUT.PUT_LINE('text is: ' || text);
END;
/
```



ORACLE

The slide shows the code for accessing CLOB values that can be implicitly converted to VARCHAR2. When selected, the RESUME column value is implicitly converted from a CLOB to a VARCHAR2 to be stored in the TEXT variable.

For Instructor Use Only.
This document should not be distributed.

Removing LOBS

- Delete a row containing LOBS:

```
DELETE
FROM customer_profiles
WHERE id = 164;
```

- Disassociate a LOB value from a row:

```
UPDATE customer_profiles
SET resume = EMPTY_CLOB()
WHERE id = 150;
```

ORACLE

A LOB instance can be deleted (destroyed) by using the appropriate SQL DML statements. The `DELETE` SQL statement deletes a row and its associated internal LOB value. To preserve the row and destroy only the reference to the LOB, you must update the row by replacing the LOB column value with `NULL` or an empty string, or by using the `EMPTY_B/CLOB()` function.

Note: Replacing a column value with `NULL` and using `EMPTY_B/CLOB` are not the same. Using `NULL` sets the value to null; using `EMPTY_B/CLOB` ensures that nothing is in the column.

A LOB is destroyed when the row containing the LOB column is deleted, when the table is dropped or truncated, or when all LOB data is updated.

You must explicitly remove the file associated with a `BFILE` by using the OS commands.

For Instructor Use Only.
This document should not be distributed.

Quiz

The `BFILE` data type stores a locator to the physical file.

- a. True
- b. False

ORACLE

Answer: a

For Instructor Use Only.
This document should not be distributed.

Quiz

You use the `BFILENAME` function to:

- a. Create a `BFILE` column
- b. Initialize a `BFILE` column
- c. Update a `BFILE` column

ORACLE

Answer: b, c

For Instructor Use Only.
This document should not be distributed.

Quiz

Q

Which of the following statements are true?

- a. You should initialize the `LOB` column to a non-NULL value by using the `EMPTY_BLOB()` and `EMPTY_CLOB()` functions.
- b. You can populate the `LOB` contents by using the `DBMS_LOB` package routines.
- c. It is possible to see the data in a `CLOB` column by using a `SELECT` statement.
- d. It is not possible to see the data in a `BLOB` or `BFILE` column by using a `SELECT` statement in SQL*Plus.

?

ORACLE

Answer: a, b, c, d

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Introduction to LOBs
- Using DBMS_LOB package
- Working with BFILEs
- Working with CLOBs
- Reading LOBs from the database
- Using temporary LOBs
- Using SecureFile LOBs

ORACLE

For Instructor Use Only.
This document should not be distributed.

Temporary LOBS

- Temporary LOBS:
 - Provide an interface to support creation of LOBS that act like local variables
 - Can be BLOBS, CLOBS, or NCLOBS
 - Are not associated with a specific table
 - Are created by using the `DBMS_LOB.CREATETEMPORARY` procedure
 - Use `DBMS_LOB` routines
- The lifetime of a temporary LOB is a session.
- Temporary LOBS are useful for transforming data in permanent internal LOBS.

ORACLE

Temporary LOBS provide an interface to support the creation and deletion of LOBS that act like local variables. Temporary LOBS can be BLOBS, CLOBS, or NCLOBS.

The following are the features of temporary LOBS:

- Data is stored in your temporary tablespace, not in tables.
- Temporary LOBS are faster than persistent LOBS, because they do not generate redo or rollback information.
- Temporary LOBS lookup is localized to each user's own session. Only the user who creates a temporary LOB can access it, and all temporary LOBS are deleted at the end of the session in which they were created.
- You can create a temporary LOB by using `DBMS_LOB.CREATETEMPORARY`.

Temporary LOBS are useful when you want to perform a transformational operation on a LOB (for example, changing an image type from GIF to JPEG). A temporary LOB is empty when created and does not support the `EMPTY_B/CLOB` functions.

Use the `DBMS_LOB` package to use and manipulate temporary LOBS.

For Instructor Use Only.
This document should not be distributed.

Creating a Temporary LOB

The PL/SQL procedure to create and test a temporary LOB:

```
CREATE OR REPLACE PROCEDURE is_templob_open(  
  p_lob IN OUT BLOB, p_retval OUT INTEGER) IS  
BEGIN  
  -- create a temporary LOB  
  DBMS_LOB.CREATETEMPORARY(p_lob, TRUE);  
  -- see if the LOB is open: returns 1 if open  
  p_retval := DBMS_LOB.ISTEMPORARY (p_lob);  
  DBMS_OUTPUT.PUT_LINE('You have created a  
    temporary LOB in the PL/SQL block');  
  -- free the temporary LOB  
  DBMS_LOB.FREETEMPORARY(p_lob);  
END;  
/
```

ORACLE

The example in the slide shows a user-defined PL/SQL procedure, `is_templob_open`, which creates a temporary LOB. This procedure accepts a LOB locator as input, creates a temporary LOB, opens it, and tests whether the LOB is open.

The `is_templob_open` procedure uses the procedures and functions from the `DBMS_LOB` package as follows:

- The `CREATETEMPORARY` procedure is used to create the temporary LOB.
- The `ISOPEN` function is used to test whether a LOB is open: This function returns the value 1 if the LOB is open.
- The `FREETEMPORARY` procedure is used to free the temporary LOB. Memory increases incrementally as the number of temporary LOBs grows, and you can reuse the temporary LOB space in your session by explicitly freeing temporary LOBs.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Introduction to LOBs
- Using DBMS_LOB package
- Working with BFILEs
- Working with CLOBs
- Reading LOBs from the database
- Using temporary LOBs
- Using SecureFile LOBs

ORACLE

For Instructor Use Only.
This document should not be distributed.

SecureFile LOBs

Oracle Database offers a reengineered large object (LOB) data type that:

- Improves performance
- Simplifies application development
- Offers advanced, next-generation functionality, such as intelligent compression and transparent encryption



ORACLE

With SecureFile LOBs, the LOB data type is completely reengineered with dramatically improved performance, manageability, and ease of application development. This implementation also offers advanced, next-generation functionality such as intelligent compression and transparent encryption. This feature significantly strengthens the native content management capabilities of Oracle Database.

SecureFiles improve many aspects of managing LOBs, including disk format, caching, locking and space management algorithms. SecureFiles allow the LOBs to be deduplicated, compressed and encrypted using parameter settings.

For Instructor Use Only.
This document should not be distributed.

Storage of SecureFile LOBs

An efficient new storage paradigm is available in Oracle Database for LOB storage.

- If the `SECUREFILE` storage keyword appears in the `CREATE TABLE` statement, the new storage is used.
- If the `BASICFILE` storage keyword appears in the `CREATE TABLE` statement, the old storage paradigm is used.
- By default, the storage is `BASICFILE`, unless you modify the setting for the `DB_SECUREFILE` parameter in the `init.ora` file.

ORACLE

Starting with Oracle Database 11g, you have the option of using the new SecureFile storage paradigm for LOBs. You can specify the use of the new paradigm by using the `SECUREFILE` keyword in the `CREATE TABLE` statement. If that keyword is left out, the old storage paradigm for basic file LOBs is used. This is the default behavior.

You can modify the `init.ora` file and change the default behavior for the storage of LOBs by setting the `DB_SECUREFILE` initialization parameter. The following values are allowed:

- **ALWAYS:** Attempts to create all LOB files as `SECUREFILES` but creates any LOBs not in `ASSM` tablespaces as `BASICFILE` LOBs
- **FORCE:** Creates all LOBs in the system as `SECUREFILE` LOBs
- **PERMITTED:** The default; allows `SECUREFILES` to be created when specified with the `SECUREFILE` keyword in the `CREATE TABLE` statement
- **NEVER:** Creates LOBs that are specified as `SECUREFILE` LOBs as `BASICFILE` LOBs
- **IGNORE:** Ignores the `SECUREFILE` keyword and all `SECUREFILE` options

Creating a SecureFile LOB

- Create a tablespace for the LOB data:

```
-- have your dba do this:  
CREATE TABLESPACE sf_tbs1  
  DATAFILE 'sf_tbs1.dbf' SIZE 1500M REUSE  
  AUTOEXTEND ON NEXT 200M  
  MAXSIZE 3000M  
  SEGMENT SPACE MANAGEMENT AUTO;
```

1

- Create a table to hold the LOB data:

```
CONNECT oe/oe  
CREATE TABLE customer_profiles_sf  
(id NUMBER,  
  first_name VARCHAR2 (40),  
  last_name  VARCHAR2 (80),  
  profile_info BLOB)  
LOB(profile_info) STORE AS SECUREFILE  
  (TABLESPACE sf_tbs1);
```

2

ORACLE

To create a column to hold a LOB that is a SecureFile, you:

- Create a tablespace to hold the data
- Define a table that contains a LOB column data type that is used to store the data in the SecureFile format

In the example shown in the slide:

1. The `sf_tbs1` tablespace is defined. This tablespace stores the LOB data in the SecureFile format. When you define a column to hold SecureFile data, you must have Automatic Segment Space Management (ASSM) enabled for the tablespace to support SecureFiles.
2. The `CUSTOMER_PROFILES_SF` table is created. The `PROFILE_INFO` column holds the LOB data in the SecureFile format, because the storage clause identifies the format.

For Instructor Use Only.
This document should not be distributed.

Quiz

Q

Which of the following statements are true about temporary LOBS?

- a. Data is stored in your temporary tablespace, not in tables.
- b. Temporary LOBS are faster than persistent LOBS, because they do not generate redo or roll back information.
- c. Only the user who creates a temporary LOB can access it.
- d. All temporary LOBS are deleted at the end of the session in which they were created.
- e. You can create a temporary LOB by using `DBMS_LOB.CREATETEMPORARY`.

?

ORACLE

Answer: a, b, c, d, e

For Instructor Use Only.
This document should not be distributed.

Summary

In this lesson, you should have learned how to:

- Identify four built-in types for large objects: `BLOB`, `CLOB`, `NCLOB`, and `BFILE`
- Describe two storage options for `LOBs`:
 - Oracle server (internal `LOBs`)
 - External host files (external `LOBs`)
- Use the `DBMS_LOB` PL/SQL package to provide routines for `LOB` management
- Describe SecureFile `LOB`



ORACLE

There are four `LOB` data types:

- A `BLOB` is a binary large object.
- A `CLOB` is a character large object.
- An `NCLOB` stores multiple-byte national character set data.
- A `BFILE` is a large object stored in a binary file outside the database.

`LOBs` can be stored internally (in the database) or externally (in an OS file). You can manage `LOBs` by using the `DBMS_LOB` package and its procedure.

Temporary `LOBs` provide an interface to support the creation and deletion of `LOBs` that act like local variables.

You learned about SecureFile format, and also learned that the performance of SecureFile format `LOBs` is faster than the BasicFile format `LOBs`.

For Instructor Use Only.
This document should not be distributed.

Practice 5: Overview

This practice covers the following topics:

- Creating object types of the `CLOB` and `BLOB` data types
- Creating a table with the `LOB` data types as columns
- Using the `DBMS_LOB` package to populate and interact with the `LOB` data
- Setting up the environment for `LOBs`

ORACLE

In this practice, you create a table with both the `BLOB` and `CLOB` columns. Then, you use the `DBMS_LOB` package to populate the table and manipulate the data.

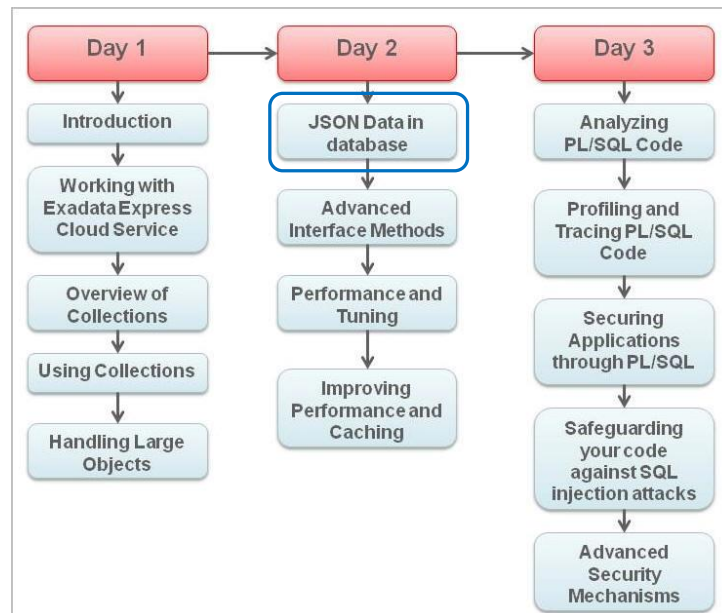
Use the `OE` schema for this practice.

For Instructor Use Only.
This document should not be distributed.

JSON Data in Database

ORACLE

Course Agenda



ORACLE

For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to:

- Describe JSON data
- Store JSON Data in database tables
- Retrieve and perform operations on JSON data
- Manipulate JSON data from a PL/SQL block



ORACLE

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- JSON Data
 - JSON data in Oracle Database
 - SQL/JSON generation functions
 - JSON Data in PL/SQL blocks
 - PL/SQL object types



ORACLE

For Instructor Use Only.
This document should not be distributed.

What Is JSON?

JavaScript Object Notation (JSON) is:

- Programming language used by web browsers and web servers to communicate
- A lightweight data interchange format
- A readable format for data structuring
- Easy for humans to read and write
- Easy for software to parse and generate



ORACLE

JavaScript Object Notation (JSON) is a general programming language widely used by web browsers and web servers for data interchange. It is an object notation that stores the objects in human-readable format.

The JavaScript-based notation of the objects is inherently compatible with JavaScript programs, eliminating the need of parsing and serializing while working with JavaScript programs. Apart from JavaScript a variety of programming languages can parse and generate JSON data.

JSON is defined in standards ECMA-404 (JSON Data Interchange Format) and ECMA-262 (ECMA Script Language Specification).

For Instructor Use Only.
This document should not be distributed.

Structure of JSON Data

JSON represents an object as a set of property-value pairs.

- A JSON object is enclosed in a pair of braces { }.
- The property name and property value pairs are enclosed in braces.
- The property name and property value is separated by a colon.

```
{ property 1: value 1,  
  property 2:value 2,  
  property 3: value 3 ..}
```

ORACLE

JSON object is stored in text format. Each object is represented as a set of property and value pairs. The values assumed by each property can be an object, array, number, string, Boolean type or a JSON object.

Each property value pair is separated by a comma.

When a property value is an array, then the array elements are enclosed in square brackets [].

For Instructor Use Only.
This document should not be distributed.

JSON Data: Example

```
{
  CUSTOMER_ID : 120,
  CUST_FIRST_NAME: 'Diane',
  CUST_LAST_NAME: 'Higgins',
  CUST_ADDRESS: {STREET: '113 Washington Sq N',
                  POSTAL_CODE: '48933',
                  CITY: 'Lansing',
                  STATE_PROVINCE: 'MI',
                  COUNTRY_ID: 'US'}
  PHONE_NUMBERS: {'+1 517 123 4199'}
  NLS_LANGUAGE: 'US',
  NLS_TERRITORY: 'AMERICA',
  CREDIT_LIMIT: 200,
  ...}
```

ORACLE

The object notation shows the name-value pairs of properties of the customer object.
Here is a JSON of a customer object:

```
{
  CUSTOMER_ID : 120
  CUST_FIRST_NAME: 'Diane'
  CUST_LAST_NAME: 'Higgins'
  CUST_ADDRESS: {STREET: '113 Washington Sq N',
                  POSTAL_CODE: '48933',
                  CITY: 'Lansing',
                  STATE_PROVINCE: 'MI',
                  COUNTRY_ID: 'US'},
  PHONE_NUMBERS: {'+1 517 123 4199'}
  NLS_LANGUAGE: 'US'
  NLS_TERRITORY: 'AMERICA'
  CREDIT_LIMIT: 200
  CUST_EMAIL: Diane.Higgins@TANAGER.EXAMPLE.COM
  ACCOUNT_MGR_ID: 145
  DATE_OF_BIRTH: '26-DEC-84'
  MARITAL_STATUS: 'SINGLE'
  GENDER: 'M'
  INCOME_LEVEL: '150,000 - 169,999'
  CREDIT_CARDS: [ {CARD_TYPE: MC, CARD_NUM: 11111111}, {CARD_TYPE: VISA, CARD_NUM:
                  2323232323}
}
```

For Instructor Use Only.
This document should not be distributed.

The customer object data has number fields, character strings, and also some composite fields such as `customer_address`, `phone numbers`, and `credit cards`.

Customer address is a composite data type which in turn have other fields such as street, postal code, and so on. You can represent it as a JSON object in itself.

Phone numbers is also a composite data type where the number of phone numbers each customer might have is not known. You may define it as a varray or a nested table as per your requirement. Here you are representing it as a JSON object.

Credit cards is also a composite data type where you are saving a pair of values, credit card type, and card number. You may use an appropriate composite data type for storing credit card information in the table.

In the JSON object representation of data, you can see how each of these composite data types are represented.

Why JSON?

- A simple way of data interchange
- Written in text format
- Interoperable
- Inherently compatible with JavaScript
- Easy for machines to parse and generate

ORACLE

JSON's popularity as data interchange language among web components is due to its simple human-readable object notation. Most of the applications today have components, which interact over the web. JSON being simple makes the interaction over web simpler. Although JSON is language independent, it uses conventions that are used in languages such as C, C++, C#, Java, JavaScript, Perl, Python, and so on. These properties make it an ideal data interchange language.

JSON is written in text format with a simple syntax structure. Parsing a JSON object is fairly simple. The text-only data format of JSON makes it compatible with any programming language.

A JavaScript program can convert JSON data into native JavaScript objects, without any parsers. Therefore, it is inherently compatible with JavaScript.

The text-only format of JSON makes it simpler for the machines to parse and generate JSON objects.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- JSON Data
- JSON data in Oracle Database
- SQL/JSON generation functions
- JSON Data in PL/SQL blocks
- PL/SQL object types



ORACLE

For Instructor Use Only.
This document should not be distributed.

JSON Data in Oracle Database: Scenario

- Consider a scenario in the Order Entry schema:
 - You want to collate the order history of each customer in your database. You can use the order history of the customer to predict the future requirements of the customer. Based on the predictions, you can make product suggestions to the customer and enhance customer experience.
- However, the order history of each customer may not be identical. The structure of data in the order history may change from customer to customer.
- You need something, which allows flexibility in structure.

Storing order history as a JSON object would be a good design choice in this scenario.



ORACLE

For Instructor Use Only.
This document should not be distributed.

JSON Data in Oracle Database

Oracle Database:

- Declarative querying language
- Reliable ACID transactions
- Complex data processing

JSON:

- Flexible schema structure for data
- Efficient data interchange language for web applications

Oracle Database supports JSON natively with relational database features including transactions, indexing, declarative querying, and views.

ORACLE

JSON data is generally stored in NoSQL databases, such as Oracle NoSQL Database and Oracle Berkeley DB, because they allow storage and retrieval of data that is not based on a schema. NoSQL databases, however, don't offer the reliability and consistency models of relational databases. Applications that require flexibility and consistency end up using relational databases and NoSQL databases in parallel.

Native support for JSON data by Oracle database obviates such workarounds. It provides all the benefits of the relational database features for use with JSON, including transactions, indexing, declarative querying and views.

For Instructor Use Only.
This document should not be distributed.

JSON Data in Oracle Database

- Oracle Database provides various functions to interpret and generate JSON data.
- You can store JSON data as table columns.
- JSON data columns can coexist with non-JSON columns.
- You can use SQL to join JSON data with relational data.
- You can project JSON data to make it available for relational processes and tools.

ORACLE

In Oracle Database, JSON data is stored in table columns like non-JSON data. JSON is stored using common SQL data types `VARCHAR2`, `CLOB`, and `BLOB`. JSON columns can co-exist with non-JSON columns.

You can write SQL queries on JSON columns just as you write on non-JSON columns.

Note: Oracle recommends that you always use an `is_json` check constraint to ensure that the column values are valid JSON instances.

For Instructor Use Only.
This document should not be distributed.

Creating a Table with JSON Column

- JSON column can assume any one of the following standard SQL data types in a table:
 - VARCHAR2
 - CLOB
 - BLOB
- Decide on the data type to be used based on the volume of data you would store in the column.
- Storing JSON data using standard SQL data types allows all the features of Oracle Database to be applied to it.

ORACLE

You can use standard SQL data types to store JSON objects. You can use one of the two variants of VARCHAR2.

VARCHAR2 (4000) where there is an upper limit of 4000 on the number of characters that can be stored in each value.

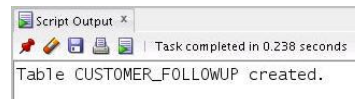
When your JSON object has more than 4000 characters, you can use VARCHAR2 (32767). If there is a probability that the JSON object can have more than 32767 characters, then you can use a CLOB or BLOB type of column.

When you store JSON data using standard SQL types, you can manipulate JSON data just like any other type of data used in SQL. All the Oracle Database features, such as advanced replication, are applicable to the JSON objects as well.

For Instructor Use Only.
This document should not be distributed.

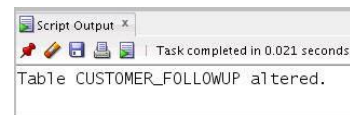
Creating a Table with JSON Column

```
CREATE TABLE customer_followup AS (SELECT customer_id, cust_first_name,  
                                         cust_last_name, cust_address,  
                                         cust_email, account_mgr_id  
                                         FROM customers);
```



Script Output x
Task completed in 0.238 seconds
Table CUSTOMER_FOLLOWUP created.

```
ALTER TABLE customer_followup ADD (order_history VARCHAR2(4000));
```



Script Output x
Task completed in 0.021 seconds
Table CUSTOMER_FOLLOWUP altered.

ORACLE

The code in the slide creates a `customer_followup` table based on the data in the `customers` table. You are copying a subset of the `customers` table information into the `customer_followup` table.

You can define a JSON data column in a table either while creating the table or later using an `ALTER TABLE` command. Here in the slide, a JSON column is added to the table using the `ALTER TABLE` command. You can see that the new column is also a `VARCHAR2` column.

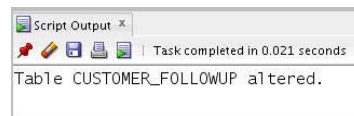
For Instructor Use Only.
This document should not be distributed.

JSON or Not?

When we store JSON as `VARCHAR2` column, how do we differentiate a JSON and a non-JSON column?

- JSON objects should have the right structure.
- You can use `is_json` SQL/JSON condition to check the structure of JSON data.
- Oracle recommends that you define an `is_json` constraint on every JSON column.

```
ALTER TABLE customer_followup  
ADD CONSTRAINT json_check CHECK(order_history IS JSON);
```



ORACLE

You've used `VARCHAR2` data type for creating the JSON data column. The difference between a JSON `VARCHAR2` column and non-JSON `VARCHAR2` column is the structure of the text data. To store valid JSON data in the column, you have to ensure that the data is structured according to JSON format.

You can check whether the data stored in a column is valid JSON or not by using the `is_json` condition.

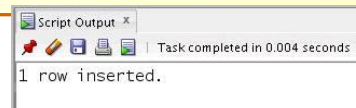
Oracle recommends that you define a `is_json` constraint on the JSON column to ensure that the data is structured according to the JSON format.

For Instructor Use Only.
This document should not be distributed.

Inserting Data into JSON Columns

- You can insert data into JSON column by using the `INSERT` statement.

```
insert into customer_followup (customer_id, cust_first_name, cust_last_name,
                             cust_address, cust_email, account_mgr_id, order_history)
select customer_id, cust_first_name,
       cust_last_name, cust_address, cust_email, account_mgr_id,
       (select JSON_OBJECT('id' VALUE o.order_id,
                           'orderTotal' VALUE o.order_total,
                           'sales_rep_id' VALUE o.sales_rep_id)
        FROM orders o
        where o.customer_id = co.customer_id)
from customers co
where co.customer_id = 120;
```



ORACLE

You are using `INSERT` statement to add values to the `customer_followup` table. The JSON data is being inserted into the `order_history` column with appropriate format.

You are retrieving the data from the `orders` table and generating a JSON object from the data retrieved from the `orders` table. This JSON is then inserted into the `order_history` column of the `customer_followup` table.

You might have noticed that we are using a `JSON_OBJECT` function to generate JSON before adding the object to the column. `JSON_OBJECT` is one of the SQL/JSON generation functions. We will discuss these functions in the following slides.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- JSON Data
- JSON data in Oracle Database
- **SQL/JSON generation functions**
- JSON Data in PL/SQL blocks
- PL/SQL object types



ORACLE

For Instructor Use Only.
This document should not be distributed.

SQL/JSON Generation Functions

You can use the following SQL/JSON functions to generate JSON data from non-JSON data:

- `JSON_OBJECT`
- `JSON_ARRAY`
- `JSON_OBJECTAGG`
- `JSON_ARRAYAGG`

ORACLE

You can use SQL/JSON generation functions to generate JSON data from non-JSON data. These functions make it simple to construct JSON data from SQL query. They return JSON objects and JSON arrays. You can also generate complex, hierarchical JSON documents by nesting calls to these functions.

It is always recommended to generate JSON data through these functions for the following reasons:

- Using string concatenation to generate JSON documents is error-prone. These functions always return well-formed JSON documents reducing the probability of error.
- By using subqueries, you can generate an entire set of JSON documents using a single SQL statement. The JSON generation operation is therefore optimized.

JSON values within the returned data are derived from SQL values in the input as follows:

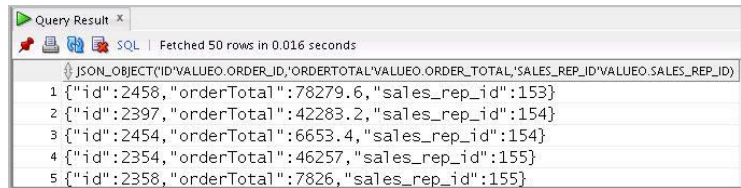
- A SQL number is converted into a JSON number.
- A non-NULL and non-number SQL value is converted to a JSON string.
- A SQL NULL value is handled by the optional NULL-handling clause.

For Instructor Use Only.
This document should not be distributed.

JSON_OBJECT Function

- JSON_OBJECT constructs JSON objects from name-value pairs.
- The name-value pairs are explicitly provided as arguments to the function.

```
SELECT JSON_OBJECT('id' VALUE o.order_id,  
                  'orderTotal' VALUE o.order_total,  
                  'sales_rep_id' VALUE o.sales_rep_id)  
FROM orders o ;
```



The screenshot shows a SQL query result window titled "Query Result". It displays the results of the query: `SELECT JSON_OBJECT('id' VALUE o.order_id, 'orderTotal' VALUE o.order_total, 'sales_rep_id' VALUE o.sales_rep_id) FROM orders o ;`. The results are 50 rows of JSON objects, with the first five rows shown. Each row contains a JSON object with three properties: `id`, `orderTotal`, and `sales_rep_id`.

	JSON_OBJECT('ID'VALUEO.ORDER_ID,'ORDERTOTAL'VALUEO.ORDER_TOTAL,'SALES_REP_ID'VALUEO.SALES_REP_ID)
1	{"id":2458,"orderTotal":78279.6,"sales_rep_id":153}
2	{"id":2397,"orderTotal":42283.2,"sales_rep_id":154}
3	{"id":2454,"orderTotal":6653.4,"sales_rep_id":154}
4	{"id":2354,"orderTotal":46257,"sales_rep_id":155}
5	{"id":2358,"orderTotal":7826,"sales_rep_id":155}

ORACLE

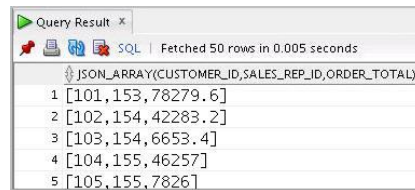
The JSON_OBJECT function returns a JSON object where data is stored in name-value pairs. The query creates a JSON object with three properties – `id`, `order_total` and `sales_rep_id`.

The number of JSON objects created in this query is 50. There is a JSON object created for each row in the `orders` table.

JSON_ARRAY Function

- The `JSON_ARRAY` function generates a JSON array based on the parameters passed.

```
SELECT JSON_ARRAY(customer_id, sales_rep_id, order_total)
FROM orders o ;
```



The screenshot shows a 'Query Result' window with the following data:

	JSON_ARRAY(CUSTOMER_ID,SALES_REP_ID,ORDER_TOTAL)
1	[101,153,78279.6]
2	[102,154,42283.2]
3	[103,154,6653.4]
4	[104,155,46257]
5	[105,155,7826]

ORACLE

The `JSON_ARRAY` function accepts the array elements as parameters and returns a JSON array. The query shown in the slide has the `JSON_ARRAY` function accepting three parameters – `customer_id`, `sales_rep_id`, and `order_total`. Therefore, a JSON array is created for each row in the `orders` table.

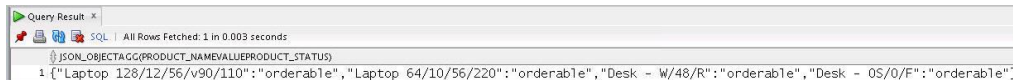
Array elements are enclosed in square brackets [], whereas JSON objects are enclosed in braces {}. The query here returns 50 JSON arrays based on the number of rows in the `orders` table.

For Instructor Use Only.
This document should not be distributed.

JSON_OBJECTAGG Function

- JSON_OBJECTAGG constructs a JSON object by aggregating data from multiple rows in the table.

```
SELECT JSON_OBJECTAGG(product_name VALUE product_status)
FROM product_information where min_price>2000 ;
```



The screenshot shows a 'Query Result' window with the following content:

SQL	All Rows Fetched: 1 in 0.003 seconds
1	JSON_OBJECTAGG(PRODUCT_NAMEVALUEPRODUCT_STATUS)
1	{ "Laptop 128/12/56/v90/110": "orderable", "Laptop 64/10/56/220": "orderable", "Desk - W/48/R": "orderable", "Desk - OS/0/F": "orderable" }

ORACLE

JSON_OBJECTAGG creates a JSON object by aggregating data from multiple rows in a table.

The query in the slide creates an object from multiple rows in the `product_information` table. You know that a JSON object is a set of name-value pairs. Note that in the query, the name field of the object is also populated from one of the column names. The name field is always expected to be a character string. Make sure that you provide a column with character string data for name. Here we use the `product_name` field of the table for the name.

The value field can be a string, number or `null`. In the query here, we provide `product_status` for value.

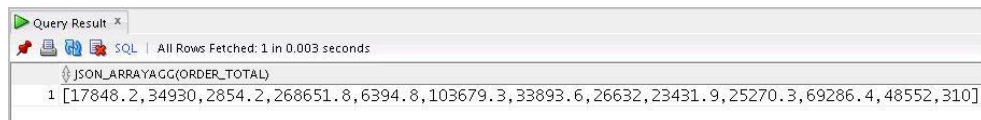
The resulting JSON object here has four name value pairs as you see on the Query Result tab. These four products satisfy the condition `min_price>2000` in the `product_information` table.

For Instructor Use Only.
This document should not be distributed.

JSON_ARRAYAGG Function

- JSON_ARRAYAGG function constructs a JSON array from multiple rows in the table.

```
SELECT JSON_ARRAYAGG(order_total)
FROM orders
WHERE sales_rep_id = 161 ;
```



The screenshot shows a 'Query Result' window with the following content:

JSON_ARRAYAGG(ORDER_TOTAL)
1 [17848.2,34930,2854.2,268651.8,6394.8,103679.3,33893.6,26632,23431.9,25270.3,69286.4,48552,310]

ORACLE

The JSON_ARRAYAGG function creates a JSON array by aggregating data from multiple rows in a table. The query in the slide identifies order values of certain sales representative with sales_rep_id value equal to 161.

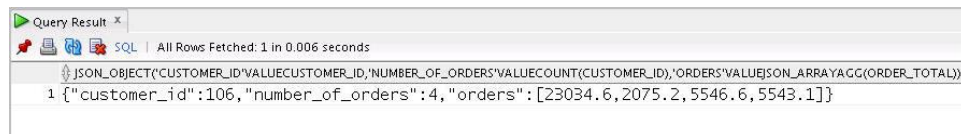
A JSON array is created with 12 order_total values put together.

For Instructor Use Only.
This document should not be distributed.

SQL/JSON Functions

You can nest multiple SQL/JSON functions to create complicated JSON objects.

```
SELECT JSON_OBJECT('customer_id' VALUE customer_id,  
                  'number_of_orders' VALUE count(customer_id),  
                  'orders' VALUE JSON_ARRAYAGG(order_total))  
FROM orders  
WHERE customer_id = 106 group by customer_id;
```



The screenshot shows a 'Query Result' window with the following content:

Query Result	
SQL All Rows Fetched: 1 in 0.006 seconds	
	<code>JSON_OBJECT('CUSTOMER_ID'VALUECUSTOMER_ID,'NUMBER_OF_ORDERS'VALUECOUNT(CUSTOMER_ID),'ORDERS'VALUEJSON_ARRAYAGG(ORDER_TOTAL))</code>
1	<code>{"customer_id":106,"number_of_orders":4,"orders":[23034.6,2075.2,5546.6,5543.1]}</code>

ORACLE

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- JSON Data
- JSON data in Oracle Database
- SQL/JSON generation functions
- SQL data from JSON objects
- PL/SQL object types



ORACLE

For Instructor Use Only.
This document should not be distributed.

Retrieving SQL Data from JSON Object

- Simple dot notation access
- SQL/JSON path expression
- SQL/JSON query functions

ORACLE

You can access JSON data in a table column in multiple ways.

Simple dot notation is the simplest and straightforward method of accessing JSON data in table column.

SQL/JSON path expressions are used to access JSON data by providing the access path to it. You can provide an absolute path expression or a relative path expression.

These techniques can be used when the volume of JSON data is minimal, to handle huge volumes of JSON data, you may have to write PL/SQL blocks, which can retrieve multiple values of JSON data according to your application logic.

You can use SQL/JSON query functions, such as `JSON_VALUE` and `JSON_QUERY` in such cases.

For Instructor Use Only.
This document should not be distributed.

Accessing JSON Data

- You can access certain values in the JSON object by using simple dot notation.
- To access the `order_id` value in the JSON column `order_history` in the `customer_followup` table, you can write a simple SQL query:

```
SELECT cf.order_history.id FROM customer_followup  
WHERE customer_id = 120;
```



The screenshot shows a 'Query Result' window with a table containing one row. The column is labeled 'ID' and the value is '2373'.

ID
2373

ORACLE

For Instructor Use Only.
This document should not be distributed.

Accessing JSON Data

- You can define a path expression to query JSON data in the database.
- Each path expression can select zero or more values that match and satisfy the expression.
- Path expressions can use wildcards, and matching is case-sensitive.
- We pass the path expressions as an argument to the SQL/JSON function.

ORACLE

When JSON data is stored in the database, you can query it using path expressions that are somewhat analogous to XQuery or XPath expressions for XML data. Similar to the way that SQL/XML allows SQL access to XML data using XQuery expressions, Oracle Database provides SQL access to JSON data using SQL/JSON path expressions.

An absolute simple path expression begins with a dollar sign (\$), which represents the path-expression context item, that is, the JSON data to be matched.

For Instructor Use Only.
This document should not be distributed.

JSON_VALUE Function

- The **JSON_VALUE** function is used to retrieve data from an object in a JSON column.
- It returns a scalar SQL value.
- The default return value is **VARCHAR2**.
- You can change it using a **RETURNING** clause.
- These functions are useful when you work on JSON data in PL/SQL blocks.

```
SELECT JSON_VALUE(order_history, '$.orderTotal')  
FROM customer_followup  
WHERE customer_id =120;
```



The screenshot shows a 'Query Result' window with the following details:

- SQL icon | All Rows Fetched: 1 in 0.002 secon
- Query: `JSON_VALUE(ORDER_HISTORY,$.ORDERTOTAL')`
- Result: 1 416

ORACLE

Similar to **JSON_VALUE**, you have **JSON_QUERY**, which selects one or more values from JSON data and returns a string (**VARCHAR2**) value.

You can use different clauses with the **JSON_VALUE** or **JSON_QUERY** function, such as **RETURNING**, **Wrapper**, **Error**, and **Empty-Field**.

For more information about JSON Data in Oracle Database, refer to:

<https://docs.oracle.com/database/122/ADJSN/toc.htm>

Using SQL/JSON Functions

```
CREATE OR REPLACE FUNCTION get_order_value(ord_his VARCHAR2)
RETURN NUMBER AS
BEGIN
    RETURN JSON_VALUE(ord_his,'$.orderTotal' RETURNING NUMBER);
END;
```

Script Output x
Task completed in 0.017 seconds
Function GET_ORDER_VALUE compiled

```
DECLARE
    jsonData VARCHAR2(4000);
    ord_val NUMBER;
BEGIN
    SELECT order_history INTO jsonData FROM customer_followup
    WHERE customer_id = 120;
    ord_val := get_order_value(jsonData);
    DBMS_OUTPUT.PUT_LINE('The old order value is '||ord_val);
    ord_val := ord_val + (0.1*ord_val);
    DBMS_OUTPUT.PUT_LINE('The new order value is '||ord_val);
END;
```

Script Output x
Task completed in 0.004 seconds
PL/SQL procedure successfully completed.
The old order value was: 416
The new order value will be: 457.6

ORACLE

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- JSON Data
- JSON data in Oracle Database
- SQL/JSON generation functions
- JSON Data in PL/SQL blocks
- PL/SQL object types



ORACLE

For Instructor Use Only.
This document should not be distributed.

PL/SQL Objects for JSON

- PL/SQL object types:
 - Allow fine-grained programmatic construction and manipulation of in-memory JSON data
 - Are transient
- To persist the data in PL/SQL objects, you must serialize them into `VARCHAR2` or `LOB` and store it in the database table.

ORACLE

Now that you have learned how to generate JSON data from relational data, you will see how you can accept JSON data and work on it in PL/SQL.

For Instructor Use Only.
This document should not be distributed.

JSON Object Types in PL/SQL

The following are the object types in PL/SQL. You can use them to work on JSON in PL/SQL program units:

- `JSON_ELEMENT_T`
- `JSON_OBJECT_T`
- `JSON_ARRAY_T`
- `JSON_SCALAR_T`
- `JSON_KEY_LIST`

ORACLE

JSON Object types are transient and are used to manipulate JSON data in-memory. You can perform the following operations on JSON data:

- Check the structure, types, or values of existing JSON data. For example, you can check `minimum_price` of a JSON object `product`.
- Transform existing JSON data.

The `JSON_ELEMENT_T` type is the supertype of all other JSON object types. You can construct an object of this type only by parsing JSON data. All other object types are subtypes of `JSON_ELEMENT_T` type.

`JSON_OBJECT_T` and `JSON_ARRAY_T` are used for JSON objects and arrays. `JSON_SCALAR_T` is used for JSON scalar values such as strings, numbers, Boolean values, and `null`.

`JSON_KEY_LIST` is a varray of size `VARCHAR2(4000)`. This object is primarily used as a return type of `get_keys` method. This holds the names of all the properties of a JSON object.

For Instructor Use Only.
This document should not be distributed.

JSON Object Methods

The following methods enable you to perform operations on JSON data:

- Parsing and serializing
- Getter and setter
- Introspection

ORACLE

Parsing is the process of converting a `VARCHAR2`, `CLOB`, or `BLOB` object into JSON object. Static function `parse()` is used to return an instance of `JSON_ELEMENT_T`, `JSON_OBJECT_T` or `JSON_ARRAY_T`.

Serializing is the process of converting PL/SQL object into textual equivalent. `to_string()` is a serialization method, which returns the `VARCHAR2` representation of a JSON object. Serialization of the JSON object is required for storing the JSON object in the database.

`JSON_OBJECT_T` and `JSON_OBJECT_ARRAY_T` have getter and setter methods, which obtain and update the values of an object.

Type `JSON_ELEMENT_T` has introspection methods, which can determine whether an instance is a JSON object, array, scalar, string, number or Boolean. The names of these methods begin with the prefix `'is_'`.

`JSON_OBJECT_T` and `JSON_ARRAY_T` also have a `remove()` and `clone()` methods, which are used for deleting a JSON object or creating a copy of the JSON object.

For Instructor Use Only.
This document should not be distributed.

Getter and Setter Methods

- The getter and setter methods are used with PL/SQL object types `JSON_OBJECT_T` and `JSON_ARRAY_T`.
- They are used to retrieve and update objects.

Getter methods

```
get()  
get_string()  
get_clob()  
get_blob()
```

Setter methods

```
put()  
put_null()  
append() (for  
JSON_ARRAY_T only)
```

ORACLE

The getter and setter methods are used to retrieve and update JSON data, respectively. These methods are used with `JSON_OBJECT_T` and `JSON_ARRAY_T` object types.

The `get()` method returns a reference to the original object as an instance of `JSON_ELEMENT_T`. The modifications you make to the `JSON_ELEMENT_T` instance apply to the original object.

When you pass a JSON object as an argument to the `get_string()` method, it returns a `VARCHAR2` equivalent of the JSON object. Similarly, `get_clob()` and `get_blob()` methods return a `CLOB` and `BLOB` equivalent, respectively.

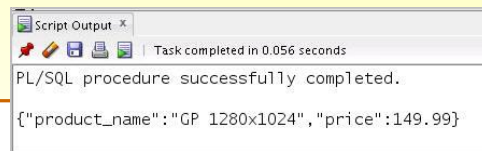
The setter methods are `put()`, `put_null()`, and `append()`. These methods update the object or array instance to which they are applied.

The `put()` method requires a second argument (besides the object field name or array element position), which is the new value to set. For an array, `put()` also accepts an optional third argument, `OVERWRITE`. This is a `BOOLEAN` value (default `FALSE`) that says whether to replace an existing value at the given position.

- If the object already has a field of the same name, then `put()` replaces that value with the new value.
- If the array already has an element at the given position, then, by default, `put()` shifts that element and any successive elements forward (incrementing their positions by one) to make room for the new element, which is placed at the given position. But if optional argument `OVERWRITE` is present and is `TRUE`, then the existing element at the given position is simply replaced by the new element.

JSON Methods in PL/SQL: Example

```
CREATE OR REPLACE PROCEDURE JSON_PUT_METHOD AS
DECLARE
je JSON_ELEMENT_T;
jo JSON_OBJECT_T;
BEGIN
je := JSON_ELEMENT_T.parse('{"product_name": "GP 1280x1024"}');
IF(je.is_Object) THEN
jo := treat(je AS JSON_OBJECT_T);
jo.put('price', 149.99);
END IF;
DBMS_OUTPUT.PUT_LINE(je.to_string);
END JSON_PUT_METHOD;
```



ORACLE

The code in the slide shows the usage of JSON methods in PL/SQL JSON object types.

You create an object of `JSON_ELEMENT_T` and `JSON_OBJECT_T` type.

You use the `parse()` method to create a `JSON_ELEMENT_T` instance from the string, which is passed as a parameter to this method, and then type cast the instance into a `JSON_OBJECT_T` type by using the `treat()` method.

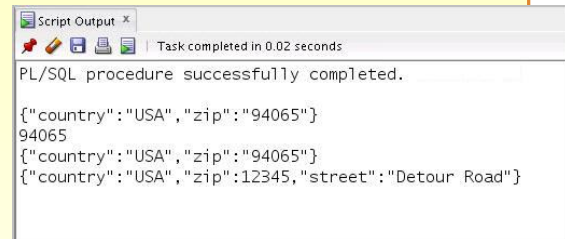
You use the `put()` method to update the JSON object. In the example, you add the price field to the JSON object.

You can display the final JSON object by using the `je.to_string` method.

For Instructor Use Only.
This document should not be distributed.

JSON Methods in PL/SQL: Example

```
CREATE OR REPLACE PROCEDURE JSON_GET_METHODS AS
in_data JSON_OBJECT_T;
address JSON_OBJECT_T;
zip NUMBER;
BEGIN
in_data := new JSON_OBJECT_T('{"first_name" : "John","last_name" : "Doe",
                             "address"   : {"country" : "USA", "zip" : "94065"} }');
address := in_data.get_object('address');
DBMS_OUTPUT.PUT_LINE(address.to_string);
zip := address.get_number('zip');
DBMS_OUTPUT.PUT_LINE(zip);
DBMS_OUTPUT.PUT_LINE(address.to_string);
address.PUT('zip', 12345);
address.PUT('street', 'Detour Road');
DBMS_OUTPUT.PUT_LINE(address.to_string);
END JSON_GET_METHODS;
/
```



```
Script Output X
Task completed in 0.02 seconds
PL/SQL procedure successfully completed.
{"country": "USA", "zip": "94065"}
94065
{"country": "USA", "zip": 12345, "street": "Detour Road"}
```

ORACLE

The example in the slide has nested JSON objects. The JSON object has three fields: `first_name`, `last_name`, and `address`, which in turn is a JSON object.

You have initialized the JSON object through a constructor. The `address` field is also a JSON object, which in turn has `country` and `zip` fields.

The code example uses `get_number` and `put` methods to access and update the address object.

For Instructor Use Only.
This document should not be distributed.

Summary

In this lesson, you should have learned how to:

- Create a table with a JSON column in it
- Insert data into a JSON column
- Generate JSON data from database tables
- Use PL/SQL JSON object types
- Use JSON methods in PL/SQL blocks



ORACLE

For Instructor Use Only.
This document should not be distributed.

Practice 6: Overview

- 6-1: JSON data in tables
 - Create a table with JSON column.
 - Insert data into the column.
 - Generate JSON data from relational data.
- 6-2: JSON data in PL/SQL blocks
 - Usage of various methods on JSON objects in a PL/SQL block.

ORACLE

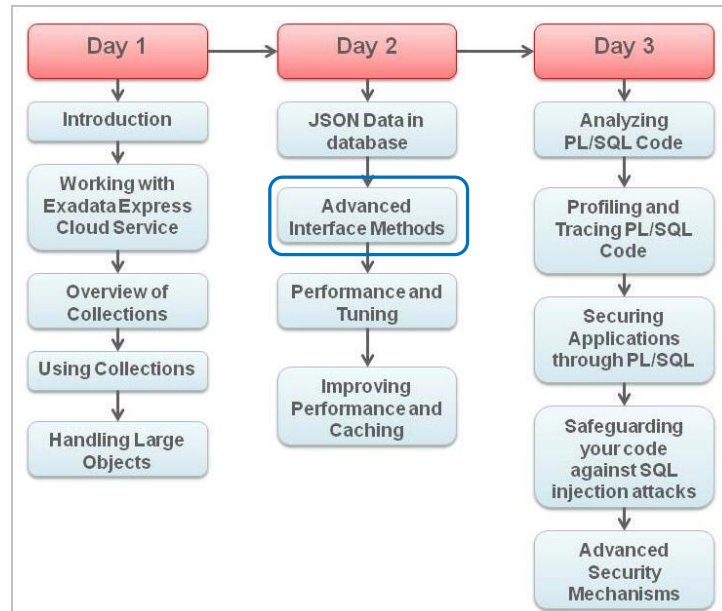
For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

Advanced Interface Methods

ORACLE

Course Agenda



ORACLE

In this lesson, you will learn to interface PL/SQL programs with other language programs such as C and Java.

For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to:

- Explain the architecture of external procedures
- Execute external C programs from PL/SQL
- Execute Java programs from PL/SQL



ORACLE

In this lesson, you learn how to implement an external C routine from PL/SQL code and how to incorporate Java code into your PL/SQL programs.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Understanding External Procedures
- Defining an external C procedure
- Executing Java programs from PL/SQL

ORACLE



For Instructor Use Only.
This document should not be distributed.

PL/SQL External Procedures

- An external procedure is a procedure written in a different programming language.
- You register the procedure with the base language and then call it for special-purpose processing.
- External procedures:
 - Interface the database server with external systems
 - Extend the functionality of the database server

ORACLE

There might be situations where you might want to implement your application in multiple languages. For example, as Java runs within the address space of the server, you might want to import existing Java applications into the database and leverage this technology by calling Java functions from PL/SQL and SQL.

The strengths and capabilities of different languages are available to you through external procedures, regardless of your program environment. You are not limited to one language. External procedures promote reusability and modularity, because you can deploy specific languages for specific purposes.

An external procedure is stored in a dynamic-link library (DLL), which is prototyped in a call specification using PL/SQL. Whenever you invoke a PL/SQL procedure, the language compiler loads the target library at run time and executes the external procedure in the Oracle Database.

For Instructor Use Only.
This document should not be distributed.

Oracle Database with Different Languages

Oracle Database allows programmers to work with different programming languages:

- C, C++
- Java
- COBOL
- Visual Basic
- .NET

ORACLE

Oracle Database allows you to work with different programming languages:

- You can use Oracle Call Interface or Oracle C++ Call Interface or Pro*C/++ compiler to work with C or C++ languages.
- To work with COBOL, you can use Pro*COBOL compiler.
- To work with Java programs, you can use JDBC and SQLJ client-side API or use Java in the database.
- You can work with Visual Basic using Oracle Provider for OLE DB.
- You can work with .NET through Oracle Provider for .NET.

Each of these languages offers different advantages such as ease of use, need for portability, and availability of programmers with specific expertise. The choice of the language depends on factors like:

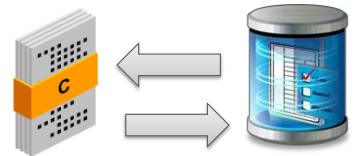
- Computation-intensive tasks are executed efficiently in a lower-level language such as C.
- For security and portability, you might choose Java.
- For familiarity with Microsoft programming languages, you might choose .NET.

Therefore, it is essential for these different programming languages to work together.

For Instructor Use Only.
This document should not be distributed.

Scenario

A company has very complicated statistics programs written in C. The customer wants to access the data stored in an Oracle database and pass the data into C programs. After the execution of the C programs, depending on the result of the evaluations, data is inserted into appropriate Oracle database tables.



ORACLE

The scenario in the slide indicates a situation in which the C program is more efficient in processing the company data and generates insights based on it. Instead of implementing the C programs again through database instructions, it will be efficient if you can reuse the existing C programs.

The benefits of external procedures:

- External procedures integrate the strength and capability of different languages to give transparent access to these routines within the database.
- They provide functionality in the database that is specific to a particular application, company, or technological area.
- They can be shared by all users on a database, and they can be moved to other databases or computers, thereby providing standard functionality with limited cost in development, maintenance, and deployment.

Using an external procedure, you can invoke an external routine through PL/SQL. By using external procedures, you can integrate the powerful programming features of 3GLs with the ease of data access of SQL and PL/SQL commands.

You can extend the database and provide backward compatibility. For example, you can invoke different index or sorting mechanisms as an external procedure to implement data cartridges.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

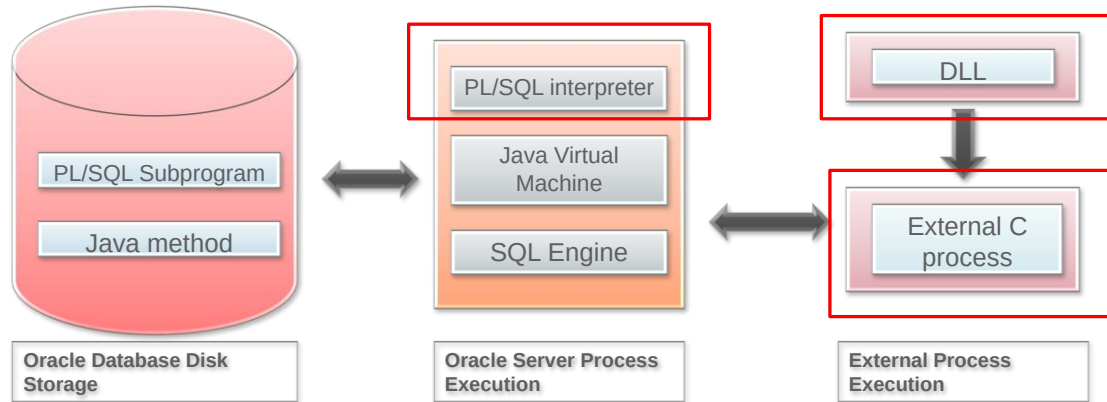
- Understanding External Procedures
- Defining an external C procedure
- Executing Java programs from PL/SQL

ORACLE



For Instructor Use Only.
This document should not be distributed.

External Procedure Execution Architecture



ORACLE

In this lesson, we will look at two variants of external procedures – C external procedure and Java external procedure.

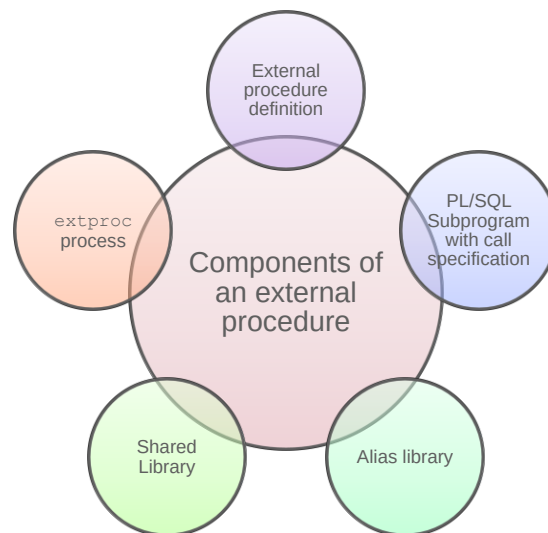
A C external procedure is stored in a DLL, which is invoked. The PL/SQL interpreter while executing a PL/SQL block invokes the external procedure based on the call specification defined in the PL/SQL block.

The components highlighted are active while executing an external C procedure.

For Instructor Use Only.
This document should not be distributed.

Components for External C Procedure Execution

Various components work together to interface a C program with Oracle Database.



ORACLE

- **External procedure:** A unit of code written in C
- **Shared library:** An operating system file that stores the external procedure, usually the dynamic-link library (DLL)
- **Alias library:** A schema object that represents the operating system shared library. You create this object to refer the actual DLL in which the external procedure is present.
- **PL/SQL subprograms:** Packages, procedures, or functions that define the program unit specification and mapping to the PL/SQL library
- **extproc process:** A session-specific process that executes external procedures. It's a process started by the Oracle Database or database listener for each session. It passes the parameters and other relevant information to the external procedure and receives the results from the external procedure and returns it to the PL/SQL unit. This process is started by the listener process of the database instance.

You can learn more about the `extproc` process at:

<https://docs.oracle.com/en/database/oracle/oracle-database/19/ntqrf/registering-an-external-procedure.html>

For Instructor Use Only.
This document should not be distributed.

Defining an External C Procedure

1. Define the procedure in an external language (C, Java, and so on).
2. Link the external procedure in a shared library, such as dynamic-link library (DLL).
3. Create an alias library as a database object.
4. Grant `execute` privileges on the library.
5. Publish the C library procedure by creating a call specification in the PL/SQL program unit.
6. You can now execute the external procedure.

ORACLE

For Instructor Use Only.
This document should not be distributed.

Define a C Function

1. Create a C program. (You generally save it as a `.c` file.)

```
#include <ctype.h>
int calc_tax(int n)
{
    int tax;
    tax = (n*8)/100;
    return(tax);
}
```

2. Link the external procedure by creating a shared object (`.so`) file for the `.c` file, and place it in `$ORACLE_HOME/bin`.

Note: These steps *vary for each operating system; consult the documentation*.

ORACLE

The slide demonstrates the process of creating a C program and then converting it into a shared object file.

You can use different methods for generating the `.so` file. The creation of the `.so` file is dependent on the underlying operating system you use.

The C program is a simple tax calculation program.

For Instructor Use Only.
This document should not be distributed.

Creating an Alias Library

3. Use the `CREATE LIBRARY` statement to create an alias library object.

```
CREATE OR REPLACE LIBRARY library_name IS|AS  
    'file_path';
```

4. Grant the `EXECUTE` privilege on the alias library.

```
GRANT EXECUTE ON library_name TO user|ROLE|PUBLIC;
```

ORACLE

An alias library is a database object that is used to map to an external shared library. An external procedure that you want to use must be stored in a DLL or a shared object library (SO) operating system file. The DBA controls access to the DLL or `.so` files by using the `CREATE LIBRARY` statement to create a schema object called an alias library that represents the external file. The DBA must give you `EXECUTE` privileges on the library object so that you can publish the external procedure, and then call it from a PL/SQL program.

Steps

- 1., 2. Steps 1 and 2 vary for each operating system. Consult your operating system or the compiler documentation.
3. Create an alias library object by using the `CREATE LIBRARY` command:

```
CREATE OR REPLACE LIBRARY c_utility  
AS $ORACLE_HOME/bin/calc_tax.so';  
/
```

The example shows the creation of a database object called `c_utility`, which references the location of the file and the name of the operating system file, `calc_tax.so`.

For Instructor Use Only.
This document should not be distributed.

4. Grant the `EXECUTE` privilege on the library object:

```
GRANT EXECUTE ON c_utility TO OE;
```

5. Publish the external C routine.
6. Call the external C routine from PL/SQL.

Dictionary Information

The alias library definitions are stored in the `USER_LIBRARIES` and `ALL_LIBRARIES` data dictionary views.

Publishing External C Procedures

Publish the external procedure in PL/SQL through call specifications. A call specification performs the following tasks:

- Registers the external procedure.
- Dispatches external procedure call to the right target.
- Performs data type conversions.
- Performs parameter mode mappings.
- Automatic memory allocation and clean up.

Access to the external procedure is controlled through the alias library.

ORACLE

Oracle Database can use only external procedures that are published through a call specification, which maps names, parameter types, and return types for your Java class method or C external procedure to their SQL counterparts. It is written like any other PL/SQL stored procedure except that, in its body, instead of declarations and a `BEGIN END` block, you code the `AS LANGUAGE` clause.

Call specifications can be specified in any of the following locations:

- Stand-alone PL/SQL procedures and functions
- PL/SQL package specifications
- PL/SQL package bodies
- Object type specifications
- Object type bodies

For Instructor Use Only.
This document should not be distributed.

Call Specification Syntax

- Identify the external body within a PL/SQL program to publish the external C procedure.

```
CREATE OR REPLACE FUNCTION function_name
(parameter_list)
RETURN datatype
    regularbody | externalbody
END;
```

- The external body contains the external C procedure information.

```
IS|AS LANGUAGE C
LIBRARY libname
[NAME C_function_name]
[CALLING STANDARD C | PASCAL]
[WITH CONTEXT]
[PARAMETERS (param_1, [param_n]);
```

ORACLE

You create the PL/SQL procedure or function and use the `IS|AS LANGUAGE C` to publish the external C procedure. The external body contains the external routine information.

Syntax Definitions

where:	LANGUAGE	Is the language in which the external routine was written (defaults to C)
	LIBRARY <i>libname</i>	Is the name of the library database object
	NAME "C_function_name"	Represents the name of the C function; if omitted, the external procedure name must match the name of the PL/SQL subprogram
	CALLING STANDARD	Specifies the Windows NT calling standard (C or Pascal) under which the external routine was compiled (defaults to C)
	WITH CONTEXT	Specifies that a context pointer is passed to the external routine for callbacks
	<i>parameters</i>	Identifies arguments passed to the external routine

Call Specification

- The parameter list:

```
parameter_list_element  
[ , parameter_list_element ]
```

- The parameter list element:

```
{ formal_parameter_name [indicator]  
| RETURN INDICATOR  
| CONTEXT }  
[BY REFERENCE]  
[external_datatype]
```

ORACLE

The formal parameter list can be used to specify the position and the types of arguments, as well as to indicate whether they should be passed by value or by reference.

Syntax Definitions

where:	<code>formal_parameter_name</code> [<code>INDICATOR</code>]	Is the name of the PL/SQL parameter that is being passed to the external routine; the <code>INDICATOR</code> keyword is used to map a C parameter whose value indicates whether the PL/SQL parameter is null
	<code>RETURN INDICATOR</code>	Corresponds to the C parameter that returns a null indicator for the function return value
	<code>CONTEXT</code>	Specifies that a context pointer will be passed to the external routine
	<code>BY REFERENCE</code>	In C, you can pass <code>IN</code> scalar parameters by value (the value is passed) or by reference (a pointer to the value is passed). Use <code>BY REFERENCE</code> to pass the parameter by reference.
	<code>External_datatype</code>	Is the external data type that maps to a C data type

Note: The `PARAMETER` clause is optional if the mapping of the parameters is done on a positional basis, and indicators, reference, and context are not needed.

Publishing an External C Routine

Example

- Publish a C function called `calc_tax` from a PL/SQL function:

```
CREATE OR REPLACE FUNCTION tax_amt (  
  x BINARY_INTEGER)  
RETURN BINARY_INTEGER  
AS LANGUAGE C  
  LIBRARY sys.c_utility  
  NAME "calc_tax";
```

- The C prototype:

```
int calc_tax (n);
```

ORACLE

You have an external C function called `calc_tax` that takes in one argument, the total sales amount. The function returns the tax amount calculated at 8%. The prototype for your `calc_tax` function is as follows:

```
int calc_tax (n);
```

To publish the `calc_tax` function in a stored PL/SQL function, use the `AS LANGUAGE C` clause within the function definition. The `NAME` identifies the name of the C function. Double quotation marks are used to preserve the case of the function defined in the C program. `LIBRARY` identifies the library object that locates the C file. The `PARAMETERS` clause is not needed in this example, because the mapping of parameters is done on a positional basis.

Executing an External C Procedure

1. The user process invokes a PL/SQL program.
2. The server process executes a PL/SQL subprogram.
3. PL/SQL subprogram looks up the alias library.
4. Oracle Database starts the external procedure agent, `extproc`.
5. The `extproc` process loads the shared library.
6. The `extproc` process links the server to the external file and executes the external procedure.
7. The data and status are returned to the server.

ORACLE

```
BEGIN
DBMS_OUTPUT.PUT_LINE(tax_amt(1000));
END;
-- Additional code for experimenting
DECLARE
    CURSOR cur_orders IS
        SELECT order_id, order_total
        FROM   orders;
    v_tax  NUMBER(8,2);
BEGIN
    FOR order_record IN cur_orders
    LOOP
        v_tax := tax_amt(order_record.order_total);
        DBMS_OUTPUT.PUT_LINE('Total tax: ' || v_tax);
    END LOOP;
END;
```

Lesson Agenda

- Understanding External Procedures
- Defining an external C procedure
- Executing Java programs from PL/SQL

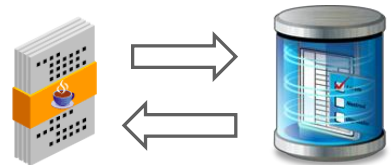


ORACLE

For Instructor Use Only.
This document should not be distributed.

Executing Java Programs from PL/SQL

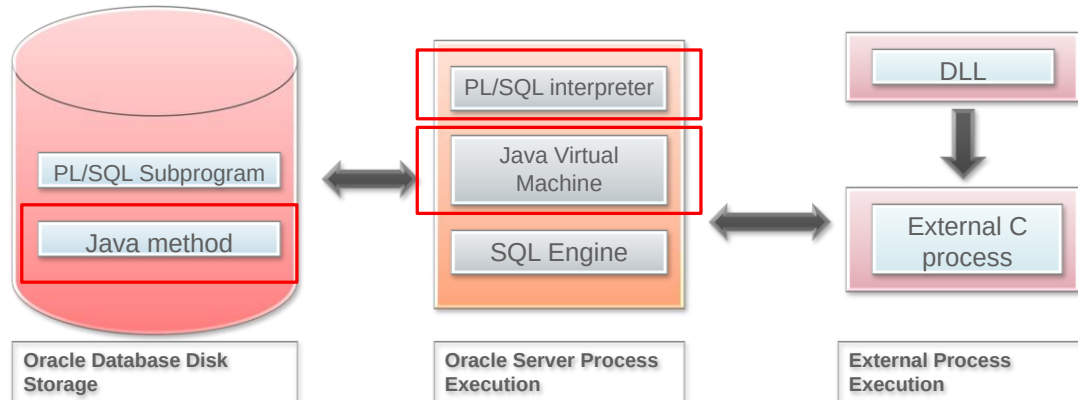
- Java programs can natively execute in Oracle Database like C programs.
- Unlike C programs Java programs are stored as Schema objects.
- Java programs use a Java shared library or libunit for execution.
- Libunits are analogous to DLLs in case of C.
- There is a libunit for each Java class.
- Oracle Database executes these libunits on Java Virtual Machine, which resides natively in the database.



ORACLE

For Instructor Use Only.
This document should not be distributed.

External Procedure Execution Architecture



Java programs are stored as schema objects. The following is the process of executing a Java program from a PL/SQL block.

1. Use the `loadjava` utility from command line or from an application to upload the Java binaries and resources into a system-generated database table, where they are stored as Java schema objects.
 2. The `loadjava` command-line utility uses the SQL `CREATE JAVA` statements to load Java source, class, or resource files into the RDBMS libunits. Libunits can be considered analogous to the DLLs written in C, although they map one to one with Java classes, whereas DLLs can contain multiple routines. Alternatively, you can implicitly call `CREATE JAVA` from SQL*Plus.
 3. When you load a Java class into the database, its methods are not published automatically, because Oracle Database does not know which methods are safe entry points for calls from SQL. To publish the Java class method, create the PL/SQL subprogram unit specification that references the Java class methods.
 4. Execute the PL/SQL subprogram that invokes the Java class method.
- The components highlighted in the architecture diagram are used while executing a Java program.

Development Steps for Java Class Methods

1. Upload the Java file by using the `loadjava` utility.
2. Publish the Java class method by creating the call specification in PL/SQL program unit.
3. Execute the PL/SQL subprogram that invokes the Java class method.

ORACLE

Similar to using external C routines, the following steps are required to complete the setup before executing the Java class method from PL/SQL:

1. Upload the Java file. This takes an external Java binary file and stores the Java code in the database.
2. Publish the Java class method by creating the PL/SQL subprogram unit specification that references the Java class methods.
3. Execute the PL/SQL subprogram that invokes the Java class method.

For Instructor Use Only.
This document should not be distributed.

Loading Java Class Methods

- Upload the Java file.
 - At the operating system level, use the `loadjava` command-line utility to load either the Java class file or the Java source file.
- To load the Java source file, use:

```
>loadjava -user oe/oe@pdborcl Factorial.java
```

ORACLE

Java classes and their methods are stored in RDBMS libunits where the Java sources, binaries, and resources can be loaded.

Use the `loadjava` command-line utility to load and resolve the Java classes. Using the `loadjava` utility, you can upload the Java source, class, or resource files into an Oracle database, where they are stored as Java schema objects. You can run `loadjava` from the command line or from an application.

After the file is loaded, it is visible in the data dictionary views.

```
SELECT object_name, object_type FROM   user_objects
WHERE  object_type like 'J%';
OBJECT_NAME                                OBJECT_TYPE
-----
Factorial                                JAVA SOURCE

SELECT text FROM   user_source WHERE name = 'Factorial';
TEXT
-----
public class Factorial {
    public static int calcFactorial (int n) {
        if (n == 1) return 1;
        else return n * calcFactorial (n - 1) ;    }}
```


Publishing a Java Class Method

- Publish the Java class method by creating the call specification in PL/SQL.
 - Identify the external body within a PL/SQL program to publish the Java class method.
 - The external body contains the name of the Java class method.
- The call specification can be located in PL/SQL program or PL/SQL package or ADT definition.

Example:

```
CREATE OR REPLACE FUNCTION plstojavafac_fun
(N NUMBER)
RETURN NUMBER
AS
LANGUAGE JAVA
NAME 'Factorial.calcFactorial
(int) return int';
```

ORACLE

The publishing of Java class methods is specified in the `AS LANGUAGE` clause. This call specification identifies the appropriate Java target routine, data type conversions, parameter mode mappings, and purity constraints. You can publish value-returning Java methods as functions and void Java methods as procedures.

You want to publish a Java method named `calcFactorial` that returns the factorial of its argument, as shown in the slide:

- The PL/SQL function `plstojavafac_fun` is created to identify the parameters and the Java characteristics.
- The `NAME` clause string uniquely identifies the Java method.
- The parameter named `N` corresponds to the `int` argument.

You can publish the Java method directly from the JVM.

Java `calcFactorial` method definition:

```
public class Factorial {
    public static int calcFactorial (int n) {
        if (n == 1) return 1;
        else return n * calcFactorial (n - 1) ;
    }
}
```

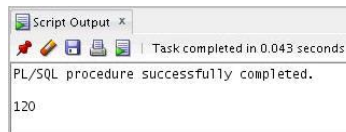
Here is another example:

```
create or replace function dept_sal(p_deptno VARCHAR2)
return VARCHAR2
AS
LANGUAGE JAVA
NAME 'dept.sal(java.lang.String) return java.lang.String';
/
```

Executing the Java Routine

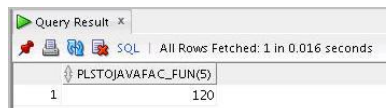
You can call the `calcFactorial` class method by using the following command:

```
EXECUTE DBMS_OUTPUT.PUT_LINE(plstojavafac_fun (5));
```



Alternatively, to execute a `SELECT` statement from the `DUAL` table:

```
SELECT plstojavafac_fun (5) FROM dual;
```



PLSTOJAVAFAC_FUN(5)	
1	120

ORACLE

For Instructor Use Only.
This document should not be distributed.

Creating Call Specifications in Packages

You can create call specifications in PL/SQL packages. Here is an example:

```
CREATE OR REPLACE PACKAGE Demo_pack
AUTHID DEFINER
AS
    PROCEDURE plsToJ_InSpec_proc
        (x BINARY_INTEGER, y VARCHAR2, z DATE)
END;
```

```
CREATE OR REPLACE PACKAGE BODY Demo_pack
AS
    PROCEDURE plsToJ_InSpec_proc
        (x BINARY_INTEGER, y VARCHAR2, z DATE)
    IS LANGUAGE JAVA
    NAME 'pkg1.class4.J_InSpec_meth
        (int, java.lang.String, java.sql.Date)';
```

ORACLE

The examples in the slide create a package specification and body named `Demo_pack`.

The package declaration has the specification of the PL/SQL procedure named `plsToJ_InSpec_proc`.

You have the procedure definition in the package body.

Note that you cannot tell whether this procedure is implemented by PL/SQL or by way of an external procedure. The details of the implementation appear only in the package body in the declaration of the procedure body.

For Instructor Use Only.
This document should not be distributed.

Quiz

Q

Which of the following statements is true about `extproc`?

- a. Oracle Database starts the external procedure agent, `extproc`.
- b. The `extproc` process loads the shared library.
- c. The `extproc` process compiles the external procedure.
- d. All of the above.

?

ORACLE

Answer: a,b

For Instructor Use Only.
This document should not be distributed.

Quiz

Q

Which of the following are true about call specifications?

- a. Link the server to the external file and execute the external procedure.
- b. Dispatch the appropriate C or Java target procedure.
- c. Perform data type conversions.
- d. Perform parameter mode mappings.
- e. Perform automatic memory allocation and cleanup.
- f. Call Java methods or C procedures from database triggers.

?

ORACLE

Answer: b,c,d,e,f

For Instructor Use Only.
This document should not be distributed.

Quiz

Q

Select the correct order of steps required to execute a Java class method from PL/SQL.

- A. Publish the Java class method by creating the PL/SQL subprogram unit specification that references the Java class methods.
- B. Upload the Java file by using the `loadjava` command-line utility.
- C. Execute the PL/SQL subprogram that invokes the Java class method.

- a. A, B, C
- b. B, A, C
- c. C, A, B
- d. C, B, A

?

ORACLE

Answer: b

For Instructor Use Only.
This document should not be distributed.

Summary

In this lesson, you should have learned how to use:

- External C routines and call them from your PL/SQL programs
- Java methods and call them from your PL/SQL programs



ORACLE

You can embed calls to external C programs from your PL/SQL programs by publishing the external routines in a PL/SQL block. You can take external Java programs and store them in the database to be called from PL/SQL functions, procedures, and triggers.

For Instructor Use Only.
This document should not be distributed.

Practice 7: Overview

This practice covers writing programs to interact with:

- C routines
- Java code

ORACLE

In this practice, you write two PL/SQL programs: one program calls an external C routine and the other calls a Java routine.

Use the `OE` schema for this practice.

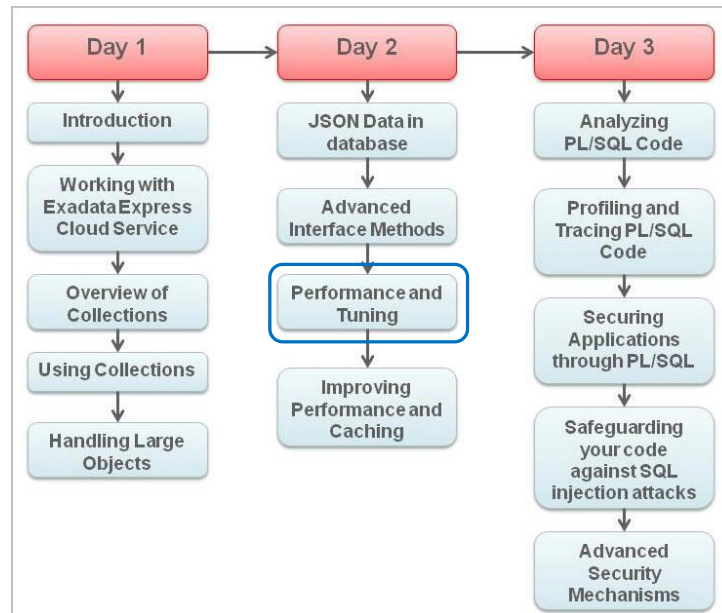
For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

Performance and Tuning

ORACLE

Course Agenda



ORACLE

In this lesson you will learn about various aspects of improving performance of the application. You will learn about different compiler options which can be used for performance tuning.

For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to do the following:

- Configure your compiler to the appropriate compilation mode
- Enable intra unit inlining to improve performance
- Tune PL/SQL code to improve performance



ORACLE

In this lesson, you will learn the various methods of improving performance of a PL/SQL unit. You can broadly identify two ways of improving the performance:

- Configure the compiler right.
- Write efficient code.

In the following slides, you will look into this in detail.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Configuring the compiler
- Enabling intraunit inlining
- Tuning PL/SQL code

ORACLE



For Instructor Use Only.
This document should not be distributed.

Compiling a PL/SQL Unit

- The performance of a PL/SQL unit can vary based on the compilation method you use.
- There are two modes of compilation:
 - Interpreted compilation
 - Default compilation mode
 - Interpreted at run time
 - Native compilation
 - Compiles into native code
 - Stored in the `SYSTEM` tablespace

ORACLE

During execution, the PL/SQL program unit is compiled into machine code — called M-Code. However, the target machine is virtual: PVM — PL/SQL virtual machine. At run time the M-Code is scanned by a subroutine on the PVM. The scanning detects op code and its operands and then calls the subroutine that implements this op code with the appropriate actual arguments. The fetching of the op code and operands happens at run time. This is interpreted compilation.

In native mode, a platform-specific DLL is produced at compile time. This DLL at run time executes on the PVM subroutines with the same arguments as would have been called by scanning the M-Code. The difference is that the scanning effort has been moved from run time to compile time; this improves runtime performance. This is native compilation. The DLL compiled is stored in the `SYSTEM` table space.

You have to choose an appropriate compile mode based on your purpose.

For Instructor Use Only.
This document should not be distributed.

Deciding on a Compilation Method

- Use the interpreted mode when (typically, during development):
 - You use a debugging tool, such as SQL Developer
 - You need the code compiled quickly
- Use the native mode when (typically post development):
 - Your code is heavily PL/SQL based
 - You want increased performance in production

ORACLE

When deciding on a compilation method, you need to examine:

- Where you are in the development cycle
- What the program unit does

If you are debugging and recompiling program units frequently, the interpreted mode has the following advantages:

- You can use PL/SQL debugging tools on program units compiled for interpreted mode (but not for those compiled for native mode).
- Compiling for interpreted mode is faster than compiling for native mode.

After completing the debugging phase of development, consider the following when determining whether to compile a PL/SQL program unit for native mode:

- The native mode provides the greatest performance gains for computation-intensive procedural operations. Examples are data warehouse applications and applications with extensive server-side transformations of data for display.
- The native mode provides the least performance gains for PL/SQL subprograms that spend most of their time executing SQL.
- When many program units (typically over 15,000) are compiled for native execution, and are simultaneously active, the large amount of shared memory required might affect system performance.

For Instructor Use Only.
This document should not be distributed.

Configuring the Compiler

- `PLSQL_CODE_TYPE`: Specifies the compilation mode for the PL/SQL library units

```
PLSQL_CODE_TYPE = { INTERPRETED | NATIVE }
```

- `PLSQL_OPTIMIZE_LEVEL`: Specifies the optimization level to be used to compile the PL/SQL library units

```
PLSQL_OPTIMIZE_LEVEL = { 0 | 1 | 2 | 3 }
```

- In general, for faster performance, use the following setting:

```
PLSQL_CODE_TYPE = NATIVE  
PLSQL_OPTIMIZE_LEVEL = 2
```

ORACLE

The `PLSQL_CODE_TYPE` Parameter

The `PLSQL_CODE_TYPE` compilation parameter determines whether the PL/SQL code is natively compiled or interpreted.

If you choose `INTERPRETED`:

- PL/SQL library units are compiled to PL/SQL bytecode format
- These modules are executed by the PL/SQL interpreter engine

If you choose `NATIVE`:

- PL/SQL library units (with the possible exception of top-level anonymous PL/SQL blocks) are compiled to native (machine) code
- Such modules are executed natively without incurring interpreter overhead

When the value of `PLSQL_CODE_TYPE` parameter is changed, it has no effect on the PL/SQL library units that have already been compiled. The value of this parameter is stored persistently with each library unit. If a PL/SQL library unit is compiled natively, all subsequent automatic recompilations of that library unit use the native compilation. In Oracle Database 19c, native compilation is easier and more integrated, with fewer initialization parameters to set.

For Instructor Use Only.
This document should not be distributed.

The `PLSQL_OPTIMIZE_LEVEL` Parameter

This parameter specifies the optimization level that is used to compile the PL/SQL library units. The higher the setting of this parameter, the more effort the compiler makes to optimize the PL/SQL library units. The available values are 0, 1, 2, and 3:

- **0:** Maintains the evaluation order and, therefore, the pattern of side effects, exceptions, and package initializations of Oracle9i and earlier releases. It also removes the new semantic identity of `BINARY_INTEGER` and `PLS_INTEGER`, and restores the earlier rules for the evaluation of integer expressions.
- **1:** Applies a wide range of optimizations to PL/SQL programs, including the elimination of unnecessary computations and exceptions, but generally does not move source code out of its original source order
- **2:** Applies a wide range of modern optimization techniques beyond those of level 1, including changes that may move source code relatively far from its original location
- **3:** Is available in Oracle Database 12c and later. It applies a wide range of optimization techniques beyond those of level 2, automatically including techniques not specifically requested. This enables subprogram inlining, which is an optimization process that replaces procedure calls with a copy of the body of the procedure to be called. The copied procedure almost always runs faster than the original call.

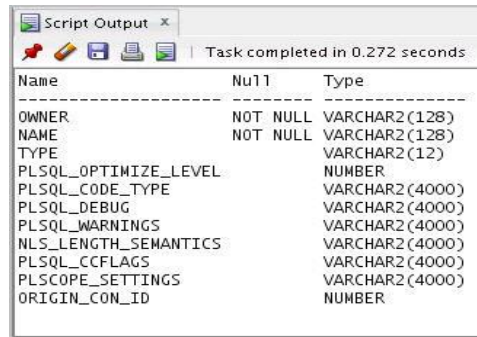
To allow subprogram inlining, either accept the default value of the `PLSQL_OPTIMIZE_LEVEL` initialization parameter (which is 2) or set it to 3. With `PLSQL_OPTIMIZE_LEVEL = 2`, you must specify each subprogram to be inlined. With `PLSQL_OPTIMIZE_LEVEL = 3`, the PL/SQL compiler seeks opportunities to inline subprograms beyond those that you specify.

Generally, setting this parameter to 2 pays off in terms of better execution performance. If the compiler runs slowly on a particular source module or if optimization does not make sense for some reason (for example, during rapid turnaround development), setting this parameter to 1 results in almost as good a compilation with less use of compile-time resources. The value of this parameter is stored persistently with the library unit.

Viewing the Compilation Settings

```
DESCRIBE ALL_PLSQL_OBJECT_SETTINGS
```

↳ Displays the settings for a PL/SQL object



Name	Null	Type
OWNER	NOT NULL	VARCHAR2(128)
NAME	NOT NULL	VARCHAR2(128)
TYPE		VARCHAR2(12)
PLSQL_OPTIMIZE_LEVEL		NUMBER
PLSQL_CODE_TYPE		VARCHAR2(4000)
PLSQL_DEBUG		VARCHAR2(4000)
PLSQL_WARNINGS		VARCHAR2(4000)
NLS_LENGTH_SEMANTICS		VARCHAR2(4000)
PLSQL_CCFLAGS		VARCHAR2(4000)
PLSCOPE_SETTINGS		VARCHAR2(4000)
ORIGIN_CON_ID		NUMBER

ORACLE

The `USER|ALL|DBA_PLSQL_OBJECT_SETTINGS` data dictionary views are used to display the settings for a PL/SQL object.

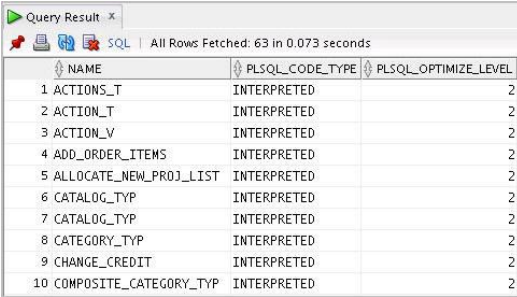
The columns of the `USER_PLSQL_OBJECTS_SETTINGS` data dictionary view include:

- **Owner:** The owner of the object. This column is not displayed in the `USER_PLSQL_OBJECTS_SETTINGS` view.
- **Name:** The name of the object
- **Type:** The available choices are `PROCEDURE`, `FUNCTION`, `PACKAGE`, `PACKAGE BODY`, `TRIGGER`, `TYPE`, or `TYPE BODY`.
- **PLSQL_OPTIMIZE_LEVEL:** The optimization level that was used to compile the object
- **PLSQL_CODE_TYPE:** The compilation mode for the object
- **PLSQL_DEBUG:** Specifies whether or not the object was compiled for debugging
- **PLSQL_WARNINGS:** The compiler warning settings used to compile the object
- **NLS_LENGTH_SEMANTICS:** The national language support (NLS) length semantics used to compile the object
- **PLSQL_CCFLAGS:** The conditional compilation flag used to compile the object
- **PLSCOPE_SETTINGS:** Controls the compile time collection, cross reference, and storage of PL/SQL source code identifier data

Viewing the Compilation Settings

```
SELECT name, plsql_code_type, plsql_optimize_level  
FROM   user_plsql_object_settings;
```

↖ To view the compilation settings



Query Result x

All Rows Fetched: 63 in 0.073 seconds

	NAME	PLSQL_CODE_TYPE	PLSQL_OPTIMIZE_LEVEL
1	ACTIONS_T	INTERPRETED	2
2	ACTION_T	INTERPRETED	2
3	ACTION_V	INTERPRETED	2
4	ADD_ORDER_ITEMS	INTERPRETED	2
5	ALLOCATE_NEW_PROJ_LIST	INTERPRETED	2
6	CATALOG_TYP	INTERPRETED	2
7	CATALOG_TYP	INTERPRETED	2
8	CATEGORY_TYP	INTERPRETED	2
9	CHANGE_CREDIT	INTERPRETED	2
10	COMPOSITE_CATEGORY_TYP	INTERPRETED	2

ORACLE

Set the values of the compiler initialization parameters by using the `ALTER SYSTEM` or `ALTER SESSION` statements.

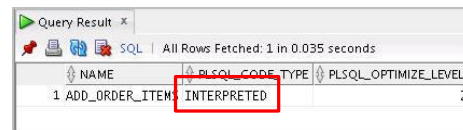
The parameters' values are accessed when the `CREATE` OR `REPLACE` or `ALTER` statements are executed.

For Instructor Use Only.
This document should not be distributed.

Setting Up a Database for Native Compilation

- You can set up native compilation mode for all the PL/SQL packages executing in the database instance.
- To set up a database for native compilation:
 - You require DBA privileges
 - You have to set the `PLSQL_CODE_TYPE` compilation parameter to `NATIVE`

```
SELECT name, plsql_code_type, plsql_optimize_level
FROM   user_plsql_object_settings
WHERE  name = 'ADD_ORDER_ITEMS';
```



The screenshot shows a 'Query Result' window with the following data:

	NAME	PLSQL_CODE_TYPE	PLSQL_OPTIMIZE_LEVEL
1	ADD_ORDER_ITEMS	INTERPRETED	2

The 'INTERPRETED' value in the `PLSQL_CODE_TYPE` column is highlighted with a red box.

ORACLE

If you have DBA privileges, you can set up a new database for PL/SQL native compilation by setting the `PLSQL_CODE_TYPE` compilation parameter to `NATIVE`. The performance benefits apply to all built-in PL/SQL packages that are used for many database operations.

For Instructor Use Only.
This document should not be distributed.

Modifying Compilation Mode of a Program Unit

```
ALTER SYSTEM SET PLSQL_CODE_TYPE = NATIVE;
```

Script Output x
Task completed in 0.041 seconds
System SET altered.

```
ALTER PROCEDURE add_order_items COMPILE;
```

Script Output x
Task completed in 0.29 seconds
Procedure ADD_ORDER_ITEMS altered.

```
SELECT name, plsql_code_type, plsql_optimize_level  
FROM   user_plsql_object_settings  
WHERE  name = 'ADD_ORDER_ITEMS';
```

Query Result x
SQL | All Rows Fetched: 1 in 0.015 seconds

	NAME	PLSQL_CODE_TYPE	PLSQL_OPTIMIZE_LEVEL
1	ADD_ORDER_ITEMS	NATIVE	2

ORACLE

To change a compiled PL/SQL object from interpreted code type to native code type, you must set the `PLSQL_CODE_TYPE` parameter to `NATIVE` (optionally, set the other parameters), and then recompile the program.

In the example in the slide:

1. The compilation type is checked on the `ADD_ORDER_ITEMS` program unit.
2. The compilation method is set to `NATIVE` at the session level.
3. The `ADD_ORDER_ITEMS` program unit is recompiled.
4. The compilation type is checked again on the `ADD_ORDER_ITEMS` program unit to verify that it changed.

If you want to compile an entire database for native or interpreted compilation, scripts are provided to help you do so.

- You must have DBA privileges.
- Set `PLSQL_CODE_TYPE` at the system level.
- Run the `dbmsupgmv.sql`-supplied script that is found in the `$ORACLE_HOME/rdbms/admin` folder.

For detailed information, see the *Oracle Database PL/SQL Language Reference 19c* reference manual.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Configuring the compiler
- Enabling Subprogram inlining
- Tuning PL/SQL code

ORACLE



For Instructor Use Only.
This document should not be distributed.

PL/SQL Optimizer

- PL/SQL optimizer:
 - Rearranges the code for better performance
 - Is enabled by default
- The `PLSQL_OPTIMIZE_LEVEL` compilation parameter determines the extent of optimization.
- Subprogram inlining is one of the optimization techniques implemented by the optimizer.

ORACLE

For Instructor Use Only.
This document should not be distributed.

Subprogram Inlining: Introduction

- Definition:
 - Inlining is the replacement of a call to a subroutine with a copy of the body of the subroutine that is called.
 - The copied procedure generally runs faster than the original.
 - The PL/SQL compiler can automatically find the calls that should be inlined.
- Benefits:
 - When applied judiciously, inlining can provide large performance gains (by a factor of 2–10).



ORACLE

Subprogram inlining replaces a subprogram invocation with a copy of the invoked subprogram (if the invoked and invoking subprograms are in the same program unit). To allow subprogram inlining, either accept the default value of the `PLSQL_OPTIMIZE_LEVEL` compilation parameter (which is 2) or set it to 3.

The copied procedure almost always runs faster than the original call, because:

- The need to create and initialize the stack frame for the called procedure is eliminated
- The optimization can be applied over the combined text of the call context and the copied procedure body
- Propagation of constant actual arguments often causes the copied body to collapse under optimization

When inlining is achieved, you can see performance gains of 2–10 times.

With the recent releases, the PL/SQL compiler can automatically find calls that should be inlined, and can do the inlining correctly and quickly. There are some controls to specify where and when the compiler should do this work (using the `PLSQL_OPTIMIZATION_LEVEL` database parameter), but usually a general request is sufficient.

For Instructor Use Only.
This document should not be distributed.

Using Inlining

- Implement inlining by using two methods:
 - Oracle parameter `PLSQL_OPTIMIZE_LEVEL`
 - `PRAGMA INLINE`
- It is recommended that you inline:
 - Small programs
 - Programs that are frequently executed

ORACLE

When implementing inlining, it is recommended that the process be applied to smaller programs, and programs that execute frequently. For example, you may want to inline small helper programs.

There are two ways to use inlining:

1. Set the `PLSQL_OPTIMIZE_LEVEL` parameter to 3. When this parameter is set to 3, the PL/SQL compiler searches for calls that might profit from inlining and inlines the most profitable calls. Profitability is measured by those calls that help the program speed up the most and keep the compiled object program as short as possible.

```
ALTER SESSION SET plsql_optimize_level = 3;
```
2. Use `PRAGMA INLINE` in your PL/SQL code. This identifies whether a specific call should be inlined. Setting this pragma to “YES” has an effect only if the optimize level is set to two or higher.

For Instructor Use Only.
This document should not be distributed.

Inlining Concepts

Noninlined program:

```
CREATE OR REPLACE PROCEDURE small_pgm
IS
  a NUMBER;
  b NUMBER;

  PROCEDURE touch(x IN OUT NUMBER, y NUMBER)
  IS
  BEGIN
    IF y > 0 THEN
      x := x+1;
    END IF;
  END;

BEGIN
  a := b;
  FOR I IN 1..10 LOOP
    touch(a, -17);
    a := a*b;
  END LOOP;
END small_pgm;
```

Without inlining, this block of code would execute 10 times, irrespective of the value of "y" passed to the touch function.

ORACLE

The example shown in the slide will be expanded to show you how a procedure is inlined.

The `a:=a*b` assignment at the end of the loop looks like it could be moved before the loop; however, it cannot, because `a` is passed as an `IN OUT` parameter to the `TOUCH` procedure. The compiler cannot be certain what the procedure does to its parameters. This results in the multiplication and in the assignment being completed 10 times instead of only once, even though multiple executions are not necessary.

For Instructor Use Only.
This document should not be distributed.

Inlining Concepts

Examine the loop after inlining:

```
...  
BEGIN  
  a := b;  
  FOR i IN 1..10 LOOP  
    IF -17 > 0 THEN  
      a := a+1;  
    END IF;  
    a := a*b;  
  END LOOP;  
END small_pgm;  
...
```

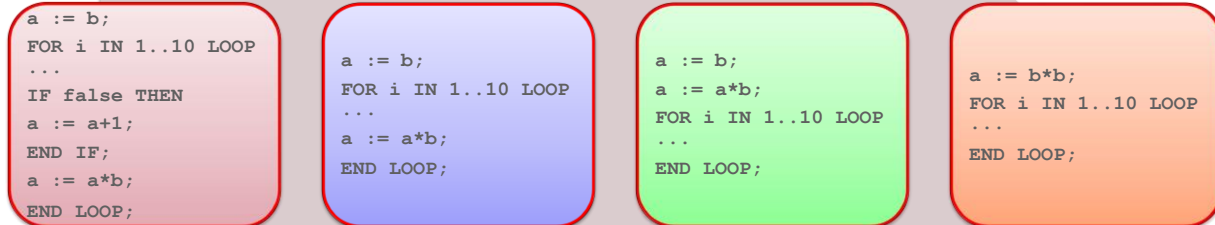
ORACLE

The code in the slide shows what happens to the loop after inlining.

For Instructor Use Only.
This document should not be distributed.

Inlining Concepts

The code in the loop is transformed as follows:



ORACLE

After inlining, procedure internals are visible to the compiler. It can transform the loop in several steps, as shown in the slide.

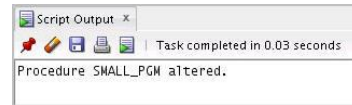
Instead of 11 assignments (one outside of the loop) and 10 multiplications, only one assignment and one multiplication are performed. If the loop ran a million times (instead of 10), the savings would be a million assignments. For code that contains deep loops that are executed frequently, inlining offers tremendous savings.

For Instructor Use Only.
This document should not be distributed.

Inlining: How to Enable It?

- Set the `PLSQL_OPTIMIZE_LEVEL` session-level parameter to a value of 2 or 3:

```
ALTER PROCEDURE small_pgm COMPILE  
PLSQL_OPTIMIZE_LEVEL = 3 REUSE SETTINGS;
```



- Setting it to 2 means no automatic inlining is attempted.
 - Setting it to 3 means automatic inlining is attempted, but no pragmas are necessary.
- Use `PRAGMA INLINE`, with the PL/SQL Subprogram.

ORACLE

To influence the optimizer to use inlining, you can set the `PLSQL_OPTIMIZE_LEVEL` parameter to a value of 2 or 3. By setting this parameter, you are making a request that inlining be used. It is up to the compiler to analyze the code and determine whether inlining is appropriate. When the optimize level is set to 3, the PL/SQL compiler searches for calls that might profit from inlining and inlines the most profitable calls.

In rare cases, if the overhead of the optimizer makes the compilation of very large applications take too long, you can lower the optimization by setting `PLSQL_OPTIMIZE_LEVEL=1` instead of its default value of 2. In even rarer cases, you might see a change in exception action, either an exception that is not raised at all, or one that is raised earlier than expected. Setting `PLSQL_OPTIMIZE_LEVEL=1`, prevents the code from being rearranged.

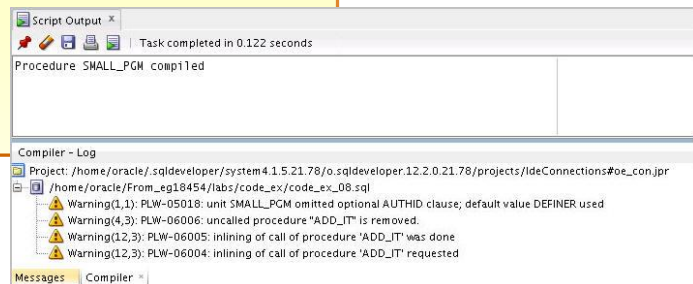
To enable inlining within a PL/SQL subroutine, you can use `PRAGMA INLINE` to suggest that a specific call be inlined.

For Instructor Use Only.
This document should not be distributed.

PRAGMA INLINE Example

After setting the `PLSQL_OPTIMIZE_LEVEL` parameter, use a **PRAGMA**:

```
CREATE OR REPLACE PROCEDURE small_pgm
IS
  a PLS_INTEGER;
  FUNCTION add_it(a PLS_INTEGER, b PLS_INTEGER)
  RETURN PLS_INTEGER
  IS
  BEGIN
    RETURN a + b;
  END;
BEGIN
  PRAGMA INLINE (add_it, 'YES');
  a := add_it(3, 4) + 6;
END small_pgm;
```



ORACLE

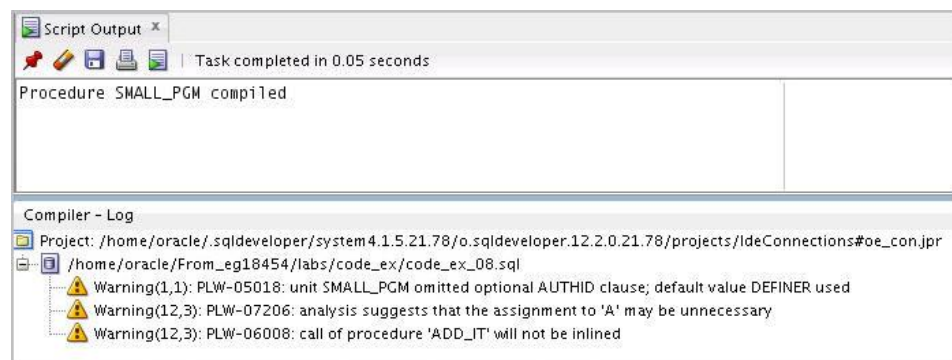
Within a PL/SQL subroutine, you can use `PRAGMA INLINE` to suggest that a specific call be inlined. When using `PRAGMA INLINE`, the first argument is the simple name of a subroutine, a function name, a procedure name, or a method name. The second argument is either the constant string 'NO' or 'YES'. The `PRAGMA` can go before any statement or declaration. If you put it in the wrong place, you receive a syntax error message from the compiler.

In the slide, you have set the second parameter to 'YES', to enable inlining.

To identify that a specific call should not be inlined, use:

```
PRAGMA INLINE (function_name, 'NO');
```

Setting the `PRAGMA INLINE` to 'NO' always works, regardless of any other pragmas that might also apply to the same statement. The pragma also applies at all optimization levels, and it applies no matter how badly the compiler would like to inline a particular call. If you are certain that you do not want some code inlined (perhaps due to the large size), you can set this to NO. You can see the response when you enable the compiler warnings.



Inlining: Summary

- Pragmas apply only to calls in the next statement following the pragma.
- Programs that make use of smaller helper subroutines are good candidates for inlining.
- Only local subroutines can be inlined.
- You cannot inline an external subroutine.
- Inlining can increase the size of a unit.
- Use inlining with deterministic functions.

ORACLE

The compiler inlines code automatically, provided that you are using native compilation and have set the `PLSQL_OPTIMIZE_LEVEL` to 3. If you have set `PLSQL_Warnings = 'enable:all'`, using the SQL*Plus `SHOW ERRORS` command displays the name of the code that is inlined.

- The PLW-06004 compiler message tells you that a `pragma INLINE('YES')` referring to the named procedure was found. The compiler will, if possible, inline this call.
- The PLW-06005 compiler message tells you the name of the code that is inlined.

Alternatively, you can query the `USER/ALL/DBA_ERRORS` dictionary view.

Deterministic functions compute the same outputs for the same inputs every time the functions are invoked, and have no side effects. The PL/SQL compiler can figure out whether a function is deterministic; it may not find all that truly are, but it finds many of them. It never mistakes a nondeterministic function for a deterministic function.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Configuring the compiler
- Enabling intraunit inlining
- Tuning PL/SQL code



ORACLE

For Instructor Use Only.
This document should not be distributed.

Why PL/SQL Tuning?

- Executing PL/SQL code adds a CPU overhead and memory overhead to the application performance.
- Tuning is the process of modifying your code to improve performance, while keeping the functionality intact.
- You must identify programming constructs, which are adding overhead to the execution, and tune them for better performance.



ORACLE

For Instructor Use Only.
This document should not be distributed.

Tuning PL/SQL Code

You can tune your PL/SQL code to meet your performance needs by:

- Identifying the data type and constraint issues
 - Data type conversion
 - The `NOT NULL` constraint
 - `PLS_INTEGER`
 - `SIMPLE_INTEGER`
- Writing smaller executable sections of code
- Using bulk binds
- Using the `FORALL` support with bulk binding
- Handling and saving exceptions with the `SAVE EXCEPTIONS` syntax
- Tuning Conditional constructs
- Tuning PL/SQL procedure calls

ORACLE

By tuning your PL/SQL code, you can customize its performance to best meet your needs.

In the following slides, you learn about some of the main PL/SQL tuning issues that can improve the performance of your PL/SQL applications.

For Instructor Use Only.
This document should not be distributed.

Avoid Implicit Data Type Conversion

- PL/SQL performs implicit conversions between structurally different data types.
- Implicit conversion is context-sensitive, therefore, might generate unpredictable results.
- To avoid unexpected values, use explicit conversion wherever required.



ORACLE

At run time, PL/SQL automatically performs implicit conversions between structurally different data types. By avoiding implicit conversions, you can improve the performance of your code.

The following guidelines can help you minimize implicit conversions:

1. If a variable is to be either inserted into a table column or assigned a value from a table column, then give the variable the same data type as the table column.
2. Make each literal the same data type as the variable to which it is assigned or the expression in which it appears.
3. Convert values from SQL data types to PL/SQL data types and then use the converted values in expressions.

For example, convert `NUMBER` values to `PLS_INTEGER` values and then use the `PLS_INTEGER` values in expressions. `PLS_INTEGER` operations use hardware arithmetic, so they are faster than `NUMBER` operations, which use library arithmetic. To avoid implicit data type conversion, use the following built-in functions:

- `TO_DATE`
 - `TO_NUMBER`
 - `TO_CHAR`
 - `CAST`
4. Before assigning a value of one SQL data type to a variable of another SQL data type, explicitly convert the source value to the target data type, using a SQL conversion function.
 5. Overload your subprograms with versions that accept parameters of different data types and optimize each version for its parameter types.

NOT NULL Constraint

```
PROCEDURE calc_m IS
  m NUMBER NOT NULL:=0;
  a NUMBER;
  b NUMBER;
BEGIN
  m := a + b;
END;
```

The value of the expression $a + b$ is assigned to a temporary variable, which is then tested for nullity.

```
PROCEDURE calc_m IS
  m NUMBER; --no constraint
  ...
BEGIN
  m := a + b;
  IF m IS NULL THEN
    -- raise error
  END IF;
END;
```

This is a better way to check nullity; no performance overhead.

ORACLE

In PL/SQL, using the `NOT NULL` constraint incurs a small performance cost. Therefore, use it with care. Consider the first example in the slide that uses the `NOT NULL` constraint for m .

Because m is constrained by `NOT NULL`, the value of the expression $a + b$ is assigned to a temporary variable, which is then tested for nullity. If the variable is not null, its value is assigned to m . Otherwise, an exception is raised. However, if m were not constrained, the value would be assigned to m directly.

A more efficient way to write the same example is with an `IF` statement as shown in the slide.

Note that the subtypes `NATURALN` and `POSITIVEN` are defined as the `NOT NULL` subtypes of `NATURAL` and `POSITIVE`. Using them incurs the same performance cost as seen in the slide's first example.

Using the NOT NULL Constraint	Not Using the Constraint
Slower	Faster
No extra coding is needed.	It requires extra coding, which is error prone.
When an error is implicitly raised, the value of m is preserved.	When an error is explicitly raised, the old value of m is lost.

PLS_INTEGER Data Type for Integers

Use `PLS_INTEGER` when dealing with integer data.

- It is an efficient data type for integer variables.
- It requires less storage than `INTEGER` or `NUMBER`.
- Its operations use machine arithmetic, which is faster than library arithmetic.

ORACLE

When you need to declare an integer variable, use the `PLS_INTEGER` data type, which is the most efficient numeric type. That is because `PLS_INTEGER` values require less storage than `INTEGER` or `NUMBER` values, which are represented internally as 22-byte Oracle numbers. Also, `PLS_INTEGER` operations use machine arithmetic, so they are faster than `BINARY_INTEGER`, `INTEGER`, or `NUMBER` operations, which use library arithmetic.

Furthermore, `INTEGER`, `NATURAL`, `NATURALN`, `POSITIVE`, `POSITIVEN`, and `SIGNTYPE` are constrained subtypes. Their variables require precision checking at run time that can affect the performance.

The `BINARY_FLOAT` and `BINARY_DOUBLE` data types are also faster than the `NUMBER` data type.

For Instructor Use Only.
This document should not be distributed.

Using the `SIMPLE_INTEGER` Data Type

- Definition:
 - Is a predefined subtype
 - Has the range `-2147483648 .. 2147483648`
 - Does not include a null value
 - Is allowed anywhere in PL/SQL where the `PLS_INTEGER` data type is allowed
- Benefits:
 - Eliminates the overhead of overflow checking
 - Is estimated to be 2–10 times faster when compared with the `PLS_INTEGER` type with native PL/SQL compilation

ORACLE

The `SIMPLE_INTEGER` data type is a predefined subtype of the `BINARY_INTEGER` (or `PLS_INTEGER`) data type that has the same numeric range as `BINARY_INTEGER`. It differs significantly from `PLS_INTEGER` in its overflow semantics. Incrementing the largest `SIMPLE_INTEGER` value by one produces the smallest value, and decrementing the smallest value by one produces the largest value. These “wrap around” semantics conform to the Institute of Electrical and Electronics Engineers (IEEE) standard for 32-bit integer arithmetic.

The key features of the `SIMPLE_INTEGER` predefined subtype are the following:

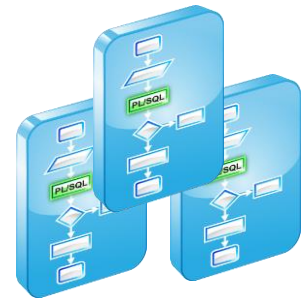
- Includes the range of `-2147483648.. +2147483648`
- Has a `NOT NULL` constraint
- Wraps rather than overflows
- Is faster than `PLS_INTEGER`

Without the overhead of overflow checking and null checking, the `SIMPLE_INTEGER` data type provides significantly better performance than `PLS_INTEGER` when the `PLSQL_CODE_TYPE` parameter is set to `native`, because arithmetic operations on the former are performed directly in the machine’s hardware. The performance difference is less noticeable when the `PLSQL_CODE_TYPE` parameter is set to `interpreted`; however, even with this setting, the `SIMPLE_INTEGER` type is faster than the `PLS_INTEGER` type.

For Instructor Use Only.
This document should not be distributed.

Modularizing Your Code

- Limit the number of lines of code between a `BEGIN` and an `END` to about a page or 60 lines of code.
- Use packaged programs to keep each executable section small.
- Use local procedures and functions to hide logic.
- Use a function interface to hide formulas and business rules.



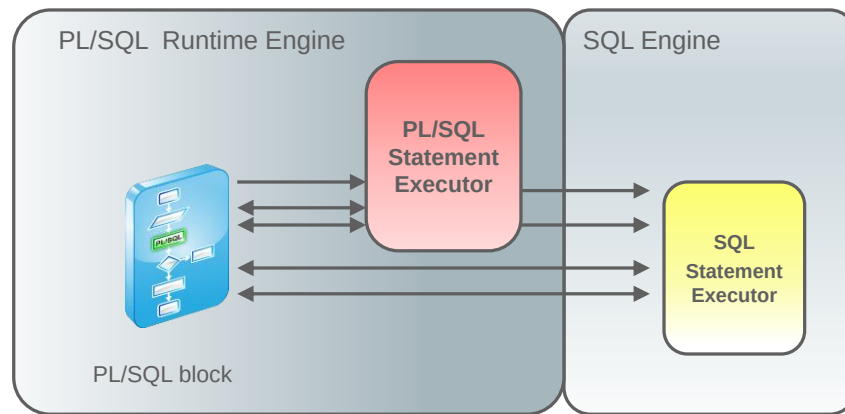
ORACLE

By writing smaller sections of executable code, you can make the code easier to read, understand, and maintain. When developing an application, use a stepwise refinement. Make a general description of what you want your program to do, and then implement the details in subroutines. Using local modules and packaged programs can help keep each executable section small. This makes it easier for you to debug and refine your code.

For Instructor Use Only.
This document should not be distributed.

Bulk Binding

Use bulk binds to reduce context switches between the PL/SQL engine and the SQL engine.



ORACLE

While executing a PL/SQL block, based on the requirement there might be context switches between PL/SQL statement executor and SQL statement executor.

With bulk binds, you can improve performance by decreasing the number of context switches between the SQL and PL/SQL engines. When a PL/SQL program executes, each time a SQL statement is encountered, there is a switch between the PL/SQL engine and the SQL engine. The more the number of switches, the less the efficiency.

Improved Performance

Bulk binding enables you to implement array fetching. With bulk binding, entire collections, not just individual elements, are passed back and forth. Bulk binding can be used with nested tables, varrays, and associative arrays.

The more the rows affected by a SQL statement, the greater is the performance gain with bulk binding.

For Instructor Use Only.
This document should not be distributed.

FORALL Instead of FOR

Bind whole arrays of values simultaneously, rather than looping to perform fetch, insert, update, and delete on multiple rows.

If you have to execute a `FOR` loop, instead of:

```
...  
FOR i IN 1 .. 50000 LOOP  
    INSERT INTO bulk_bind_example_tbl  
        VALUES (...);  
END LOOP; ...
```

Use:

```
...  
FORALL i IN 1 .. 50000  
    INSERT INTO bulk_bind_example_tbl  
        VALUES (...);  
...
```

ORACLE

In the first example shown in the slide, one row at a time is inserted into the target table. In the second example, the `FOR` loop is changed to a `FORALL` (which has an implicit loop) and all the immediately subsequent DML statements are processed in bulk. The entire code examples, along with the timing statistics for running each `FOR` loop example, are as follows.

First, create the demonstration table:

```
CREATE TABLE bulk_bind_example_tbl (  
    num_col NUMBER,  
    date_col DATE,  
    char_col VARCHAR2(40));
```

Then, set the SQL*Plus `TIMING` variable on. Setting it on enables you to see the approximate elapsed time of the last SQL statement:

```
SET TIMING ON
```

Then, run this block of code that includes a `FOR` loop to insert 50,000 rows:

```
DECLARE  
    TYPE typ_numlist IS TABLE OF NUMBER;  
    TYPE typ_datelist IS TABLE OF DATE;  
    TYPE typ_charlist IS TABLE OF VARCHAR2(40)  
        INDEX BY PLS_INTEGER;
```

```

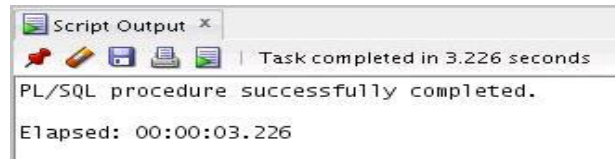
n typ_numlist := typ_numlist();
d typ_datelist := typ_datelist();
c typ_charlist;

```

```

BEGIN
  FOR i IN 1 .. 50000 LOOP
    n.extend;
    n(i) := i;
    d.extend;
    d(i) := sysdate + 1;
    c(i) := lpad(1, 40);
  END LOOP;
  FOR I in 1 .. 50000 LOOP
    INSERT INTO bulk_bind_example_tbl
      VALUES (n(i), d(i), c(i));
  END LOOP;
END;
/

```



Finally, run this block of code that includes a FORALL loop to insert 50,000 rows. Note the significant decrease in the timing when using FORALL processing:

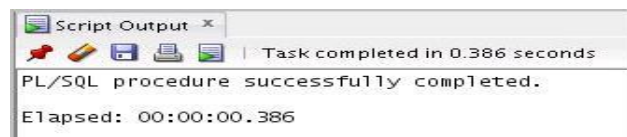
```

DECLARE
  TYPE typ_numlist IS TABLE OF NUMBER;
  TYPE typ_datelist IS TABLE OF DATE;
  TYPE typ_charlist IS TABLE OF VARCHAR2(40)
    INDEX BY PLS_INTEGER;

  n typ_numlist := typ_numlist();
  d typ_datelist := typ_datelist();
  c typ_charlist;

BEGIN
  FOR i IN 1 .. 50000 LOOP
    n.extend;
    n(i) := i;
    d.extend;
    d(i) := sysdate + 1;
    c(i) := lpad(1, 40);
  END LOOP;
  FORALL I in 1 .. 50000
    INSERT INTO bulk_bind_example_tbl
      VALUES (n(i), d(i), c(i));
END;
/

```



BULK COLLECT

Use BULK COLLECT to improve performance:

```
CREATE OR REPLACE PROCEDURE process_customers
(p_account_mgr customers.account_mgr_id%TYPE)
IS
  TYPE typ_numtab IS TABLE OF
    customers.customer_id%TYPE;
  TYPE typ_chartab IS TABLE OF
    customers.cust_last_name%TYPE;
  TYPE typ_emailtab IS TABLE OF
    customers.cust_email%TYPE;
  v_custnos      typ_numtab;
  v_last_names   typ_chartab;
  v_emails       typ_emailtab;
BEGIN
  SELECT customer_id, cust_last_name, cust_email
  BULK COLLECT INTO v_custnos, v_last_names, v_emails
  FROM customers
  WHERE account_mgr_id = p_account_mgr;
  ...
END process_customers;
```

ORACLE

When you require a large number of rows to be returned from the database, you can use the BULK COLLECT option for queries. This option enables you to retrieve multiple rows of data in a single request. The retrieved data is then populated into a series of collection variables. This query runs significantly faster than if it were done without the BULK COLLECT.

You can use the BULK COLLECT option with explicit cursors too:

```
BEGIN
  OPEN cv_customers INTO customers_rec;
  FETCH cv_customers BULK COLLECT INTO
    v_custnos, v_last_name, v_mails;
  ...
```

You can also use the LIMIT option with BULK COLLECT. This gives you control over the amount of processed rows in one step.

```
FETCH cv_customers BULK COLLECT
  INTO v_custnos, v_last_name, v_email
  LIMIT 200;
```

For Instructor Use Only.
This document should not be distributed.

BULK COLLECT

Use the `RETURNING` clause to retrieve information about the rows that are being modified:

```
DECLARE
  TYPE      typ_replist IS VARRAY(100) OF NUMBER;
  TYPE      typ_numlist IS TABLE OF
              orders.order_total%TYPE;
  repids     typ_replist :=
              typ_replist(153, 155, 156, 161);
  totlist    typ_numlist;
  c_big_total CONSTANT NUMBER := 60000;
BEGIN
  FORALL i IN repids.FIRST..repids.LAST
    UPDATE orders
      SET   order_total = .95 * order_total
    WHERE  sales_rep_id = repids(i)
    AND    order_total > c_big_total
    RETURNING order_total BULK COLLECT INTO Totlist;
END;
```

ORACLE

Often, applications need information about the row that is affected by a SQL operation, for example, to generate a report or take action. Using the `RETURNING` clause, you can retrieve information about the rows that you modified with the `INSERT`, `UPDATE`, and `DELETE` statements. This can improve performance, because it enables you to make changes, and at the same time, collect information about the data being changed. As a result, fewer network round trips, less server CPU time, fewer cursors, and less server memory are required. Without the `RETURNING` clause, you need two operations: one to make the change and the other to retrieve information about the change. In the example in the slide, the `order_total` information is retrieved from the `ORDERS` table and collected into the `totlist` collection. The `totlist` collection is returned in bulk to the PL/SQL engine.

If you did not use the `RETURNING` clause, you would need to perform two operations: one for the `UPDATE`, and another for the `SELECT`:

```
UPDATE orders SET order_total = .95 * order_total
WHERE  sales_rep_id = p_id
AND    order_total > c_big_total;

SELECT order_total FROM orders
WHERE  sales_rep_id = p_id AND order_total > c_big_total;
```

In the following example, you update the credit limit of a customer and at the same time retrieve the customer's new credit limit into a SQL Developer environment variable:

```
CREATE OR REPLACE PROCEDURE change_credit
    (p_in_id    IN    customers.customer_id%TYPE,
     o_credit OUT NUMBER)
IS
BEGIN
    UPDATE customers
    SET    credit_limit = credit_limit * 1.10
    WHERE  customer_id = p_in_id
    RETURNING credit_limit INTO o_credit;
END change_credit;
/
VARIABLE g_credit NUMBER
EXECUTE change_credit(109, :g_credit)
PRINT g_credit
```

Exception While Bulk Collecting

- You can use the `SAVE EXCEPTIONS` keyword in your `FORALL` statements:

```
FORALL index IN lower_bound..upper_bound
SAVE EXCEPTIONS
{insert_stmt | update_stmt | delete_stmt}
```

- Exceptions raised during execution are saved in the `%BULK_EXCEPTIONS` cursor attribute.
- The attribute is a collection of records with two fields:

Field	Definition
ERROR_INDEX	Holds the iteration of the <code>FORALL</code> statement where the exception was raised
ERROR_CODE	Holds the corresponding Oracle error code

ORACLE

To handle the exceptions encountered during a `BULK BIND` operation, you can add the keyword `SAVE EXCEPTIONS` to your `FORALL` statement. Without it, if a row fails during the `FORALL` loop, the loop execution is terminated. `SAVE EXCEPTIONS` allows the loop to continue processing and is required if you want the loop to continue.

All exceptions raised during the execution are saved in the `%BULK_EXCEPTIONS` cursor attribute, which stores a collection of records. This cursor attribute is available only from the exception handler.

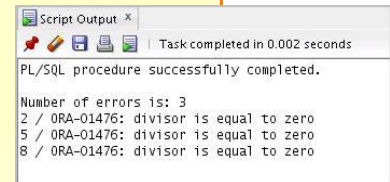
Each record has two fields. The first field, `%BULK_EXCEPTIONS(i).ERROR_INDEX`, holds the “iteration” of the `FORALL` statement during which the exception was raised. The second field, `BULK_EXCEPTIONS(i).ERROR_CODE`, holds the corresponding Oracle error code.

The values stored by `%BULK_EXCEPTIONS` always refer to the most recently executed `FORALL` statement. The number of exceptions is saved in the count attribute of `%BULK_EXCEPTIONS`; that is, `%BULK_EXCEPTIONS.COUNT`. Its subscripts range from 1 to `COUNT`. If you omit the `SAVE EXCEPTIONS` keyword, execution of the `FORALL` statement stops when an exception is raised. In that case, `SQL%BULK_EXCEPTIONS.COUNT` returns 1, and `SQL%BULK_EXCEPTIONS` contains just one record. If no exception is raised during the execution, `SQL%BULK_EXCEPTIONS.COUNT` returns 0.

For Instructor Use Only.
This document should not be distributed.

Handling FORALL Exceptions

```
DECLARE
  TYPE NumList IS TABLE OF NUMBER;
  num_tab      NumList :=
    NumList(100,0,110,300,0,199,200,0,400);
  bulk_errors EXCEPTION;
  PRAGMA      EXCEPTION_INIT (bulk_errors, -24381 );
BEGIN
  FORALL i IN num_tab.FIRST..num_tab.LAST
  SAVE EXCEPTIONS
  DELETE FROM orders WHERE order_total < 500000/num_tab(i);
EXCEPTION WHEN bulk_errors THEN
  DBMS_OUTPUT.PUT_LINE('Number of errors is: '
    || SQL%BULK_EXCEPTIONS.COUNT);
  FOR j in 1..SQL%BULK_EXCEPTIONS.COUNT
  LOOP
    DBMS_OUTPUT.PUT_LINE (
      TO_CHAR(SQL%BULK_EXCEPTIONS(j).error_index) ||
      ' / ' ||
      SQLERRM(-SQL%BULK_EXCEPTIONS(j).error_code) );
  END LOOP;
END;
```



ORACLE

In the example in the slide, the `EXCEPTION_INIT` pragma defines an exception named `BULK_ERRORS` and associates the name with the `ORA-24381` code, which is an “Error in Array DML.” The PL/SQL block raises the predefined exception `ZERO_DIVIDE` when `i` equals 2, 5, 8. After the bulk bind is completed, `SQL%BULK_EXCEPTIONS.COUNT` returns 3, because the code tried to divide by zero three times. To get the Oracle error message (which includes the code), you pass `SQL%BULK_EXCEPTIONS(i).ERROR_CODE` to the error-reporting function `SQLERRM`.

Tuning Conditional Control Statements

In logical expressions, PL/SQL stops evaluating the expression as soon as the result is determined.

- In an `OR` conditional statement, place the condition, which is most likely to evaluate `TRUE` first in the evaluation order.
- In an `AND` conditional statement, place the condition, which is most likely to evaluate `FALSE` first in the evaluation order.

ORACLE

In logical expressions, improve performance by carefully ordering conditional constructs.

When evaluating a logical expression, PL/SQL stops evaluating the expression as soon as the result is determined. While evaluating an `OR` condition, the evaluation would stop when it encounters a `TRUE` condition. Therefore, place the condition, which is most likely to evaluate to `TRUE` first in the evaluation order.

In case of an `AND` expression, the evaluation of the expression will stop when it encounters a `FALSE` condition. Therefore place the condition, which is most likely to evaluate to `FALSE` first in the evaluation order.

Such placement of conditions will reduce redundant comparisons, thus improving the performance.

For Instructor Use Only.
This document should not be distributed.

Tuning Conditional Control Statements

If your business logic results in one condition being true, use the `ELSIF` syntax for mutually exclusive clauses:

```
IF v_acct_mgr = 145 THEN
  process_acct_145;
END IF;
IF v_acct_mgr = 147 THEN
  process_acct_147;
END IF;
IF v_acct_mgr = 148 THEN
  process_acct_148;
END IF;
IF v_acct_mgr = 149 THEN
  process_acct_149;
END IF;
```



```
IF v_acct_mgr = 145
THEN
  process_acct_145;
ELSIF v_acct_mgr = 147
THEN
  process_acct_147;
ELSIF v_acct_mgr = 148
THEN
  process_acct_148;
ELSIF v_acct_mgr = 149
THEN
  process_acct_149;
END IF;
```



ORACLE

If you have a situation where you are checking a list of choices for a mutually exclusive result, use the `ELSIF` syntax, because it offers the most efficient implementation. With `ELSIF`, after a branch evaluates to `TRUE`, the other branches aren't executed.

In the example shown on the right in the slide, every `IF` statement is executed. In the example on the left, after a branch is found to be true, the rest of the branch conditions aren't evaluated.

Sometimes you don't need an `IF` statement. For example, the following code can be rewritten without an `IF` statement:

```
IF date_ordered < sysdate + 7 THEN
  late_order := TRUE;
ELSE
  late_order := FALSE;
END IF;
```

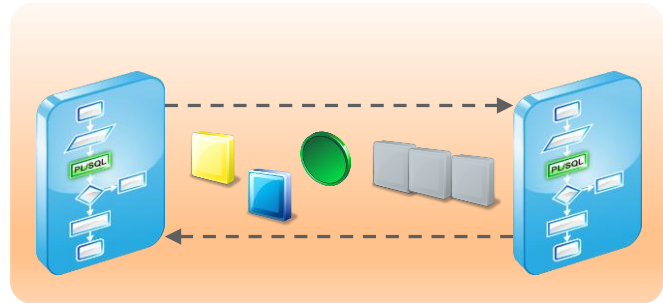
--rewritten without an IF statement:

```
late_order := date_ordered < sysdate + 7;
```

For Instructor Use Only.
This document should not be distributed.

Passing Data Between PL/SQL Programs

- The flexibility built into PL/SQL enables you to pass:
 - Simple scalar variables
 - Complex data structures
- You can use the `NOCOPY` hint to improve performance with the `IN OUT` parameters.



ORACLE

You can pass simple scalar data or complex data structures between PL/SQL programs.

When passing collections as parameters, you may encounter a slight decrease in performance as compared with passing scalar data, but the performance is still comparable. However, when passing `IN OUT` parameters that are complex (such as collections) to a procedure, you will experience significantly more overhead, because a copy of the parameter value is stored before the routine is executed. The stored value must be kept in case an exception occurs.

You can use the `NOCOPY` compiler hint to improve performance in this situation. `NOCOPY` instructs the compiler not to make a backup copy of the parameter that is being passed. However, be careful when you use the `NOCOPY` compiler hint, because your results are not predictable if your program encounters an exception.

For Instructor Use Only.
This document should not be distributed.

Passing Data Between PL/SQL Programs

Pass records as parameters to encapsulate data, and write and maintain less code:

```
DECLARE
  TYPE CustRec IS RECORD (
    customer_id      customers.customer_id%TYPE,
    cust_last_name   VARCHAR2(20),
    cust_email       VARCHAR2(30),
    credit_limit      NUMBER(9,2));
  ...
  PROCEDURE raise_credit (cust_info CustRec);
```

ORACLE

You can declare user-defined records as formal parameters of procedures and functions as shown in the slide. By using records to pass values, you are encapsulating the data being passed. This requires less coding than defining, assigning, and manipulating each record field individually.

When you call a function that returns a record, use the notation:

```
function_name(parameters).field_name
```

For example, the following call to the `NTH_HIGHEST_ORD_TOTAL` function references the `ORDER_TOTAL` field in the `ORD_INFO` record:

```
DECLARE
  TYPE OrdRec IS RECORD (
    v_order_id      NUMBER(6),
    v_order_total   REAL);
  v_middle_total REAL;
  FUNCTION nth_highest_total (n INTEGER) RETURN OrdRec IS
    order_info OrdRec;
  BEGIN
    ...
    RETURN order_info; -- return record
  END;
  BEGIN
    -- call function
    v_middle_total := nth_highest_total(10).v_order_total;
  ...
```

Passing Data Between PL/SQL Programs

Use collections as arguments:

```
PACKAGE cust_actions IS
  TYPE NameTabTyp IS TABLE OF
    customer.cust_last_name%TYPE
    INDEX BY PLS_INTEGER;
  TYPE CreditTabTyp IS TABLE OF
    customers.credit_limit%TYPE
    INDEX BY PLS_INTEGER;
  ...
  PROCEDURE credit_batch( name_tab  IN NameTabTyp ,
                          credit_tab IN CreditTabTyp,
                          ... );
  PROCEDURE log_names ( name_tab IN NameTabTyp );
END cust_actions;
```

ORACLE

You can declare collections as formal parameters of procedures and functions. In the example in the slide, associative arrays are declared as the formal parameters of two packaged procedures. If you were to use scalar variables to pass the data, you would need to code and maintain many more declarations.

For Instructor Use Only.
This document should not be distributed.

Quiz

Q

Which of the following statements are true?

- a. Use the native mode during development.
- b. Because the native code does not have to be interpreted at run time, it runs faster.
- c. The interpreted compilation is the default compilation method.
- d. To change a compiled PL/SQL object from interpreted code type to native code type, you must set the `PLSQL_CODE_TYPE` parameter to `NATIVE`, and then recompile the program.

?

ORACLE

Answer: b, c, d

For Instructor Use Only.
This document should not be distributed.

Quiz

Q

You can tune your PL/SQL code by:

- a. Writing longer executable sections of code
- b. Avoiding bulk binds
- c. Using the `FORALL` support with bulk binding
- d. Handling and saving exceptions with the `SAVE EXCEPTIONS` syntax
- e. Rephrasing conditional statements

?

ORACLE

Answer: c, d, e

For Instructor Use Only.
This document should not be distributed.

Quiz

Q

Which of the following statements are true with reference to inlining?

- a. Pragmas apply only to calls in the next statement following the pragma.
- b. Programs that make use of smaller helper subroutines are bad candidates for inlining.
- c. Only local subroutines can be inlined.
- d. You cannot inline an external subroutine.
- e. Inlining can decrease the size of a unit.

?

ORACLE

Answer: a, c, d

For Instructor Use Only.
This document should not be distributed.

Summary

In this lesson, you should have learned how to:

- Decide when to use native or interpreted compilation
- Tune your PL/SQL application. Tuning involves:
 - Using the `RETURNING` clause and bulk binds when appropriate
 - Rephrasing conditional statements
 - Identifying data type and constraint issues
 - Understanding when to use SQL and PL/SQL
- Identify opportunities for inlining PL/SQL code
- Use native compilation for faster PL/SQL execution



ORACLE

There are several methods that help you tune your PL/SQL application.

When tuning PL/SQL code, consider using the `RETURNING` clause and bulk binds to improve processing. Be aware of conditional statements with an `OR` clause. Place the fastest processing condition first. There are several data type and constraint issues that can help in tuning an application.

By using native compilation, you can benefit from performance gains for computation-intensive procedural operations.

For Instructor Use Only.
This document should not be distributed.

Practice 8: Overview

This practice covers the following topics:

- Tuning PL/SQL code to improve performance
- Coding with bulk binds to improve performance

ORACLE

In this practice, you tune some of the code that you created for the OE application.

- Break a previously built subroutine into smaller executable sections.
- Pass collections into subroutines.
- Add error handling for `BULK INSERT`.

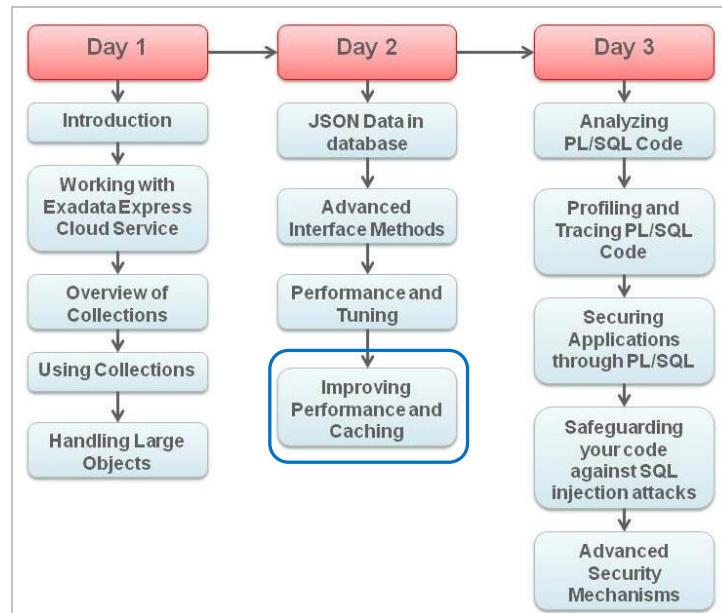
Use the OE schema for this practice.

For Instructor Use Only.
This document should not be distributed.

Improving Performance with Caching

ORACLE

Course Agenda



ORACLE

For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to do the following:

- Describe Result Cache
- Write queries that use the result cache hint
- Use the `DBMS_RESULT_CACHE` package
- Set up PL/SQL functions to use PL/SQL result caching



ORACLE

In this lesson, you learn about the Oracle Database 19c caching techniques that can improve performance. You examine the improvement on the performance of queries by caching the results of a query in memory, and then using the cached results in future executions of the query or query fragments. The cached results reside in the result cache memory portion of the shared global area (SGA).

The result-caching mechanism of the PL/SQL cross-session function provides applications with a language-supported and system-managed means for storing the results of PL/SQL functions in an SGA, which is available to every session that runs the application.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

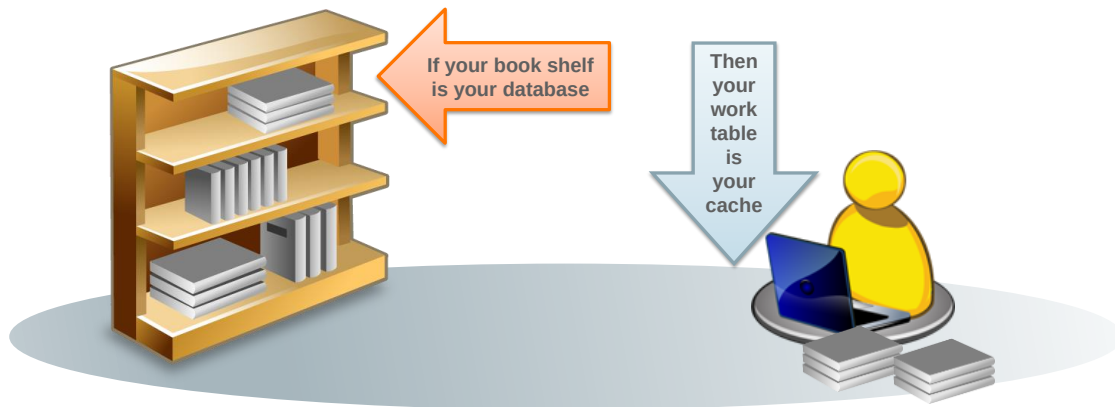
- Caching in the Database Instance
 - Result Caching
 - SQL Result Caching
 - PL/SQL Result Caching
 - Oracle Database In-Memory



ORACLE

For Instructor Use Only.
This document should not be distributed.

What Is Caching?



ORACLE

Caching is a technique used to improve the performance of an application. The performance overhead on applications is generally due to memory access and I/O. To reduce this overhead, you create a small high-speed memory area called cache. This cache stores frequently used data, thus reducing the overhead of frequent memory access.

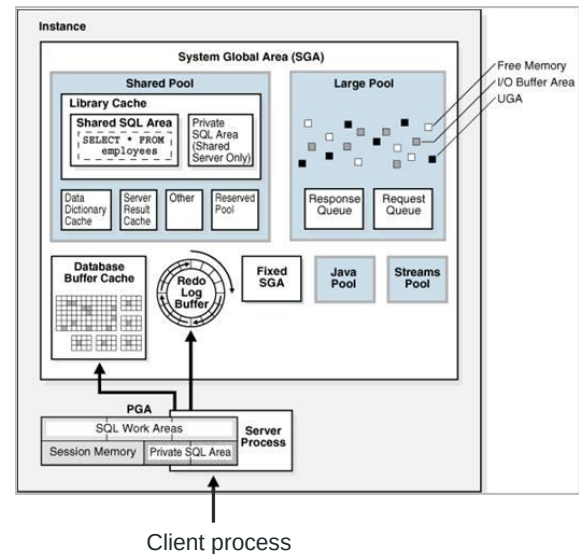
As shown in the slide, if a bookshelf is your database where you have all the files then your work table is your cache. You prefer to keep frequently used books on your work table to avoid the overhead of walking to the bookshelf to fetch the book. Oracle database instance uses caching at various levels in its memory architecture to improve performance.

For Instructor Use Only.
This document should not be distributed.

Memory Architecture

A database instance has the following memory structures:

- System Global Area (SGA)
- Program Global Area (PGA)
- User Global Area (UGA)
- Software code areas



ORACLE

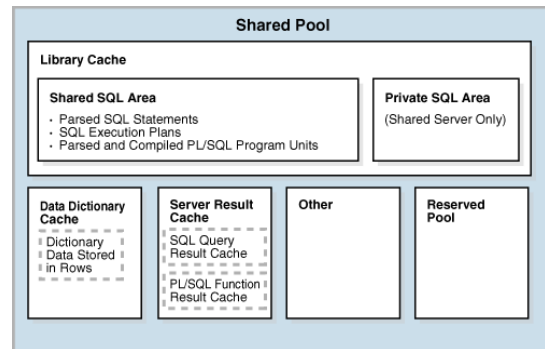
An Oracle Database server comprises a database and a database instance. A database is the set of files stored on the disk. A database instance refers to a set of memory structures that manage database files. An instance can exist independently of the database.

A database instance comprises of the following memory structures:

- **System Global Area (SGA):** The SGA is a group of shared memory structures that contain data and control information for one database instance. Examples of SGA components include the database buffer cache and shared SQL areas. Starting in Oracle Database 12c Release 1 (12.1.0.2), the SGA can contain an In Memory Column Store (IM column store), which enables data to be populated in memory in a columnar format.
- **Program Global Area(PGA):** A PGA is a memory region that contain data and control information for a server or background process. Each client accessing the database a PGA associated with it.
- **User Global Area(UGA):** The UGA is memory associated with a user session. Stores the session state of an user.
- **Software code areas:** Software code areas are portions of memory used to store code that is being run or can be run.

Caching in the Database Instance

- Database Buffer Cache in the database instance stores currently used or recently used database blocks.
- The shared pool of the database instance has the following cache components:
 - Library cache
 - Data Dictionary cache
 - Server Result cache(optional)



ORACLE

The database buffer cache, also called the *buffer cache*, is the memory area that stores copies of data blocks read from data files. A buffer is a main memory address in which the buffer manager temporarily caches a currently or recently used data block to optimize physical I/O. All users concurrently connected to a database instance share access to the buffer cache.

The shared pool in the SGA has Library Cache, Data Dictionary Cache and an optional Server Result Cache.

The Library cache stores the executable (parsed or compiled) form of recently referenced SQL and PL/SQL code.

The data dictionary cache stores data referenced from the data dictionary.

The server result cache is an optional cache that stores query and PL/SQL function results within the shared pool.

You can tune the performance of the application by tuning the configuration of Database Buffer Cache, Library Cache and Data Dictionary Cache. You can learn more on this in the course Oracle Database: Performance Management and Tuning Ed 1.

Server Result Cache is an optional cache, you can define whether the result of a SQL query or PL/SQL unit can be cached or not at the query level or unit level. You may also configure your instance to not use the result cache. We will look at the usage of Result Cache in the following slides.

For Instructor Use Only.
This document should not be distributed.

What Is Result Caching?

- You can store SQL query and PL/SQL function results in a result cache.
- Subsequent executions of the same query or function can be served directly out of the cache, improving response times.
- This technique can be especially effective for SQL queries and PL/SQL functions that are executed frequently with the same parameters.
- Cached query results become invalid when the database data accessed by the query is modified.
- Result caching brings huge benefits when the data in the database doesn't change often.

ORACLE

The new SQL query result cache enables explicit caching of queries and query fragments in an area of the shared pool called “result cache memory.” When a query is executed, the result cache is built up and the result is returned. The database can then use the cached results for subsequent query executions with the same parameters, thereby resulting in faster response times. Cached query results become invalid when the data in the database objects being accessed by the query is modified.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Caching in the Database Instance
- **Result Caching**
- SQL Result Caching
- PL/SQL Result Caching
- Oracle Database In-Memory



ORACLE

For Instructor Use Only.
This document should not be distributed.

Configuring the Server Result Cache

- The size of the result cache is relative to the size of the shared pool and the memory management mechanism in place.
- You can use the following initialization parameters to configure the result cache:
 - `RESULT_CACHE_MAX_SIZE`
 - `RESULT_CACHE_MAX_RESULT`
 - `RESULT_CACHE_REMOTE_EXPIRATION`

ORACLE

By default, on database startup, Oracle Database allocates memory to the server result cache in the shared pool. The memory size allocated depends on the memory size of the shared pool and the selected memory management system:

- **Automatic shared memory management:** If you are managing the size of the shared pool using the `SGA_TARGET` initialization parameter, Oracle Database allocates 0.50% of the value of the `SGA_TARGET` parameter to the result cache.
- **Manual shared memory management:** If you are managing the size of the shared pool using the `SHARED_POOL_SIZE` initialization parameter, then Oracle Database allocates 1% of the shared pool size to the result cache.

The size of the server result cache grows until it reaches the maximum size. Query results larger than the available space in the cache are not cached. The database employs a Least Recently Used (LRU) algorithm to age out cached results, but does not otherwise automatically release memory from the server result cache.

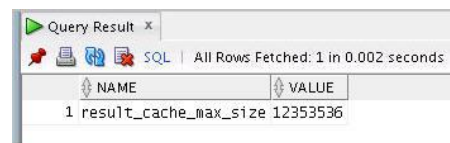
For Instructor Use Only.
This document should not be distributed.

Parameter	Description
RESULT_CACHE_MAX_SIZE	Specifies the memory allocated to the server result cache. To disable the server result cache, set this parameter to 0.
RESULT_CACHE_MAX_RESULT	Specifies the maximum amount of server result cache memory (in percent) that can be used for a single result. Valid values are between 1 and 100. The default value is 5%. You can set this parameter at the system or session level.
RESULT_CACHE_REMOTE_EXPIRATION	Specifies the expiration time (in minutes) for a result in the server result cache that depends on remote database objects. The default value is 0, which specifies that results using remote objects will not be cached. If a non-zero value is set for this parameter, DML on the remote database does not invalidate the server result cache. Setting this parameter to a nonzero value can produce stale answers (for example, if the remote table used by a result is modified at the remote database).

Setting Result_Cache_Max_Size

- Set `Result_Cache_Max_Size` from the command line or in an initialization file created by a DBA.
- The cache size is dynamic and can be changed either permanently or until the instance is restarted.

```
SQL> SELECT name, value
2 FROM v$parameter
3 WHERE name = 'result_cache_max_size';
```



The screenshot shows a 'Query Result' window with a table containing one row. The table has two columns: 'NAME' and 'VALUE'. The row shows 'result_cache_max_size' with a value of '12353536'. The window also indicates 'All Rows Fetched: 1 in 0.002 seconds'.

	NAME	VALUE
1	result_cache_max_size	12353536

ORACLE

By default, the server-side result cache is configured to use a very small portion of the shared pool. You can manually set the result cache memory size by using the `RESULT_CACHE_MAX_SIZE` initialization parameter. Setting `RESULT_CACHE_MAX_SIZE` to 0 during database startup disables the server-side result cache. `RESULT_CACHE_MAX_SIZE` cannot be dynamically changed if the value is set to 0 during database startup in the SPFILE (server parameter file) or the `init.ora` (initialization) file.

For Instructor Use Only.
This document should not be distributed.

Setting the Result Cache Mode

- Use the `RESULT_CACHE_MODE` initialization parameter in the database initialization parameter file to set the result cache mode at the database level.
- `RESULT_CACHE_MODE` can be set to:
 - **MANUAL (default):** You must add the `RESULT_CACHE` hint to your queries for the results to be cached.
 - **FORCE:** Results are always stored in the result cache memory, if possible.
- You can set the result cache mode at the table level by using the `ALTER TABLE` command.
- You can set the result cache mode at the query level or PL/SQL program unit level explicitly.

ORACLE

You can enable query result cache at the database level by using the `RESULT_CACHE_MODE` initialization parameter in the database initialization parameter file. The same parameter can also be used at the session level by using the `ALTER SESSION` command. `RESULT_CACHE_MODE` can be set to:

- **MANUAL (default):** You must add the `RESULT_CACHE` hint to your queries for the results to be cached or to be served out of the cache. The `RESULT_CACHE` hint can also be added in subqueries and inline views.
- **FORCE:** Results are always stored in the result cache memory, if possible. The result may not be cached in situations such as the result exceeds the maximum result size or cache size.

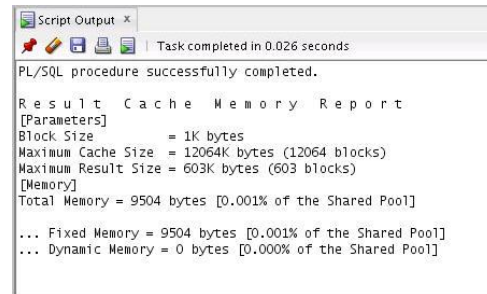
The use of the SQL query result cache introduces the `ResultCache` operator in the query execution plan.

For Instructor Use Only.
This document should not be distributed.

Using the DBMS_RESULT_CACHE Package

The DBMS_RESULT_CACHE package provides an interface for a DBA to manage memory allocation for SQL query result cache and the PL/SQL function result cache.

```
EXECUTE DBMS_RESULT_CACHE.MEMORY_REPORT
```



```
Script Output x
Task completed in 0.026 seconds
PL/SQL procedure successfully completed.

Result Cache Memory Report
[Parameters]
Block Size           = 1K bytes
Maximum Cache Size   = 12064K bytes (12064 blocks)
Maximum Result Size  = 603K bytes (603 blocks)
[Memory]
Total Memory = 9504 bytes [0.001% of the Shared Pool]

... Fixed Memory = 9504 bytes [0.001% of the Shared Pool]
... Dynamic Memory = 0 bytes [0.000% of the Shared Pool]
```

ORACLE

You can use the DBMS_RESULT_CACHE package to perform various operations, such as bypassing the cache, retrieving statistics on the cache memory usage, and flushing the cache. For example, to view the memory allocation statistics, use `dbms_result_cache.memory_report`. Following are DBMS_RESULT_CACHE subprograms:

Subprogram	Description
BYPASS Procedure	Sets the bypass mode for the Result Cache
FLUSH function and procedure	Attempts to remove all the objects from the Result Cache, and depending on the arguments retains or releases the memory and retains or clears the statistics
INVALIDATE functions and procedures	Invalidates all the result-set objects that dependent upon the specified dependency object
INVALIDATE_OBJECT functions and procedures	Invalidates the specified result-set object(s)
MEMORY_REPORT procedure	Produces the memory usage report for the Result Cache
STATUS function	Checks the status of the Result Cache

Lesson Agenda

- Caching in the Database Instance
- Result Caching
- **SQL Result Caching**
- PL/SQL Result Caching
- Oracle Database In-Memory



ORACLE

For Instructor Use Only.
This document should not be distributed.

SQL Query Result Cache

- SQL Query Result Cache holds the results of a SQL query or a query fragment.
- The cached results are reused to improve performance when the query is executed again.



ORACLE

You can improve the performance of your queries by caching the results of a query in memory, and then using the cached results in future executions of the query or query fragments. The cached results reside in the result cache memory portion of the SGA. This feature is designed to speed up query execution on systems with large memories.

For Instructor Use Only.
This document should not be distributed.

SQL Query Result Cache

- Scenario:
 - You need to find the greatest average value of credit limit grouped by state over the whole population.
 - The query returns a large number of rows being analyzed to yield a few rows or one row.
 - In your query, the data changes fairly slowly (say every hour), but the query is repeated fairly often (say every second).
- Solution:
 - Use the new optimizer hint `/*+ result_cache */` in your query:

```
SELECT /*+ result_cache */  
      AVG(cust_credit_limit), cust_state_province  
FROM sh.customers  
GROUP BY cust_state_province;
```

ORACLE

SQL result caching is useful when your queries need to analyze a large number of rows to return a small number of rows or a single row.

Two new optimizer hints are available to turn on and turn off SQL result caching:

```
/*+ result_cache */  
/*+ no_result_cache */
```

These hints enable you to override the settings of the `RESULT_CACHE_MODE` initialization parameter.

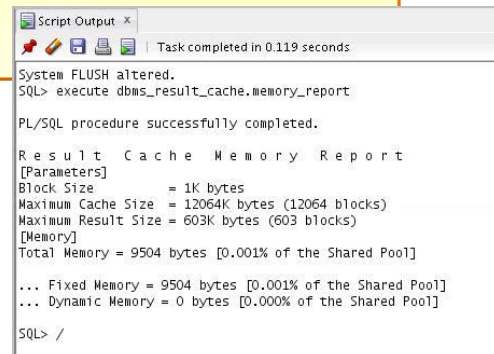
You can execute `DBMS_RESULT_CACHE.MEMORY_REPORT` to produce a memory usage report of the result cache.

For Instructor Use Only.
This document should not be distributed.

Examining the Memory Cache

```
--- flush.sql
--- Start with a clean slate. Flush the cache and shared pool.
--- Verify that memory was released.
SET ECHO ON
SET FEEDBACK 1
SET SERVEROUTPUT ON

execute dbms_result_cache.flush
alter system flush shared_pool
/
execute dbms_result_cache.memory_report
```



Script Output x

Task completed in 0.119 seconds

System FLUSH altered.
SQL> execute dbms_result_cache.memory_report

PL/SQL procedure successfully completed.

Result Cache Memory Report
[Parameters]
Block Size = 1K bytes
Maximum Cache Size = 12064K bytes (12064 blocks)
Maximum Result Size = 603K bytes (603 blocks)
[Memory]
Total Memory = 9504 bytes [0.001% of the Shared Pool]
... Fixed Memory = 9504 bytes [0.001% of the Shared Pool]
... Dynamic Memory = 0 bytes [0.000% of the Shared Pool]
SQL> /

ORACLE

Executing the `memory_report` procedure of the `DBMS_RESULT_CACHE` package provides information about the result cache.

For Instructor Use Only.
This document should not be distributed.

Examining the Execution Plan for a Query

```
--- plan_query1.sql
--- Generate the execution plan.
--- (The query name Q1 is optional)
explain plan for
select /*+ result_cache q_name(Q1) */ * from orders;

--- Display the execution plan.
select plan_table_output from
table(dbms_xplan.display('plan_table',null,'serial'));
```

Query Result x

SQL | All Rows Fetched: 14 in 0.453 seconds

PLAN_TABLE_OUTPUT

1 Plan hash value: 1275100350

2

3 -----

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		105	3885	3 (0)	00:00:01
1	RESULT CACHE	b3kc4dpapvgxf0v4fwutyzy38v				
2	TABLE ACCESS FULL	ORDERS	105	3885	3 (0)	00:00:01

9 -----

10

11 Result Cache Information (identified by operation id):

12 -----

13

14 1 - column-count=8; dependencies=(OE.ORDERS); name="select /*+ result_cache q_name(Q1) */ * from orders"

ORACLE

You examine the execution plan for two queries, and then execute both the queries. After executing both the queries, you view the memory allocation and usage statistics.

First, execute the code shown in the slide and examine the execution plan for the first query. The query uses the `RESULT_CACHE` optimizer hint.

For Instructor Use Only.
This document should not be distributed.

Examining Another Execution Plan

```

--- plan_query2.sql
set echo on
--- Generate the execution plan. (The query name Q2 is optional)
explain plan for
select c.customer_id, o.ord_count
from (select /*+ result_cache q_name(Q2) */
      customer_id, count(*) ord_count
      from orders
      group by customer_id) o, customers c
where o.customer_id = c.customer_id;

--- Display the execution plan.
--- using the code in ORACLE_HOME/rdbms/admin/utlxpls

select plan_table output from table(dbms_xplan.display('plan_table',
null,'serial'));

```

Result Cache Information (identified by operation id):

```

-----
3 - column-count=2; dependencies=(OE.ORDERS); name="select /*+ result_cache q_name(Q2) */ customer_id, count(*) ord_count from orders group by customer_id"

```

ORACLE

Execute the code shown in the slide and examine the execution plan for the second query. This query also uses the `RESULT_CACHE` optimizer hint.

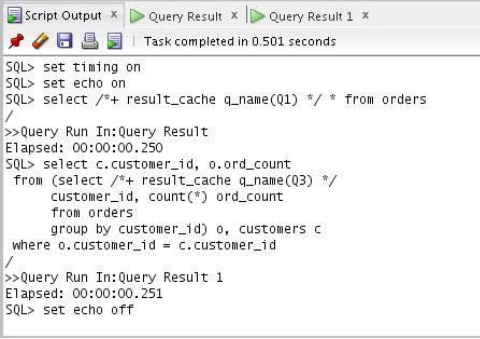
Script Output x Query Result x	
SQL All Rows Fetched: 25 in 0.025 seconds	
PLAN_TABLE_OUTPUT	
1 Plan hash value: 2892511806	
2	
3	

4	Id Operation Name Rows Bytes Cost (%CPU) Time
5	
6	0 SELECT STATEMENT 47 1410 1 (0) 00:00:01
7	1 NESTED LOOPS 47 1410 1 (0) 00:00:01
8	* 2 VIEW 47 1222 1 (0) 00:00:01
9	3 RESULT CACHE 2vawk7zuptxxw7zddpsnpp96gz
10	4 HASH GROUP BY 47 188 1 (0) 00:00:01
11	5 INDEX FULL SCAN ORD_CUSTOMER_IX 105 420 1 (0) 00:00:01
12	* 6 INDEX UNIQUE SCAN CUSTOMERS_PK 1 4 0 (0) 00:00:01
13	

14	
15 Predicate Information (identified by operation id):	
16	
17	
18 2 - filter("O"."CUSTOMER_ID">0)	
19 6 - access("O"."CUSTOMER_ID"="C","CUSTOMER_ID")	
20 filter("C"."CUSTOMER_ID">0)	
21	
22 Result Cache Information (identified by operation id):	
23	
24	
25 3 - column-count=2; dependencies=(OE.ORDERS); name="select /*+ result_cache q_name(Q2) */ customer_id, count(*) ord_count from orders group by customer_id"	

Executing Both Queries

```
--- query3.sql
--- Cache result of both queries, then use the cached result.
Set timing on
set echo on
select /*+ result_cache q_name(Q1) */ * from orders;
select c.customer_id, o.ord_count
  from (select /*+ result_cache q_name(Q3) */
        customer_id, count(*) ord_count
        from orders
       group by customer_id) o, customers c
 where o.customer_id = c.customer_id;
```



Script Output x Query Result x Query Result 1 x
Task completed in 0.501 seconds

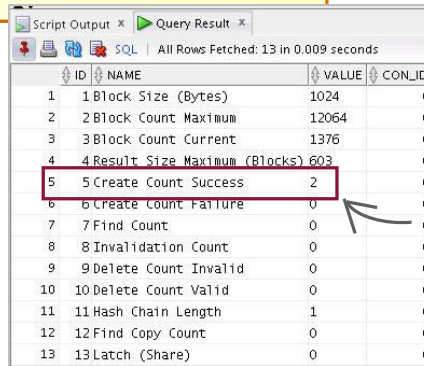
```
SQL> set timing on
SQL> set echo on
SQL> select /*+ result_cache q_name(Q1) */ * from orders
/
>>Query Run In:Query Result
Elapsed: 00:00:00.250
SQL> select c.customer_id, o.ord_count
  from (select /*+ result_cache q_name(Q3) */
        customer_id, count(*) ord_count
        from orders
       group by customer_id) o, customers c
 where o.customer_id = c.customer_id
/
>>Query Run In:Query Result 1
Elapsed: 00:00:00.251
SQL> set echo off
```

ORACLE

You have examined the execution plan for both the queries. Now execute both the queries by executing the code shown in the slide.

Viewing Cache Results Created

```
col name format a55
select * from v$result_cache_statistics
/
```



ID	NAME	VALUE	CON_ID
1	1Block Size (Bytes)	1024	0
2	2Block Count Maximum	12064	0
3	3Block Count Current	1376	0
4	4Result Size Maximum (Blocks)	603	0
5	5Create Count Success	2	0
6	6Create Count Failure	0	0
7	7Find Count	0	0
8	8Invalidation Count	0	0
9	9Delete Count Invalid	0	0
10	10Delete Count Valid	0	0
11	11Hash Chain Length	1	0
12	12Find Copy Count	0	0
13	13Latch (Share)	0	0

Number of cache results
successfully created

ORACLE

The next step is to run a query against `V$RESULT_CACHE_STATISTICS` to view the memory allocation and usage statistics. You can view the memory allocation by executing the code as shown in the slide.

Note that the `CREATE COUNT` has the number of cache results that were successfully created (one for each query statement).

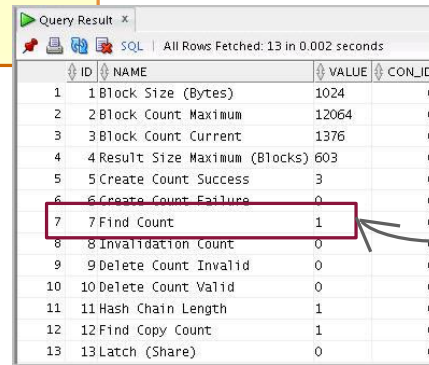
In the next steps, you re-execute the queries and view the cache results found.

For Instructor Use Only.
This document should not be distributed.

Viewing Cache Results Found

- Re-execute the same query and check the cache statistics.

```
col name format a55  
select * from v$result_cache_statistics  
/
```



ID	NAME	VALUE	CON_ID
1	1 Block Size (Bytes)	1024	0
2	2 Block Count Maximum	12064	0
3	3 Block Count Current	1376	0
4	4 Result Size Maximum (Blocks)	603	0
5	5 Create Count Success	3	0
6	6 Create Count Failure	0	0
7	7 Find Count	1	0
8	8 Invalidation Count	0	0
9	9 Delete Count Invalid	0	0
10	10 Delete Count Valid	0	0
11	11 Hash Chain Length	1	0
12	12 Find Copy Count	1	0
13	13 Latch (Share)	0	0

Successful
finds in
cache.

ORACLE

Query `V$RESULT_CACHE_STATISTICS` again to view memory allocation and usage statistics. Do this by again executing the code shown in the slide.

The Find Count keeps track of the number of results that were successfully found in the cache.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Caching in the Database Instance
- Result Caching
- SQL Result Caching
- **PL/SQL Result Caching**
- Oracle Database In-Memory

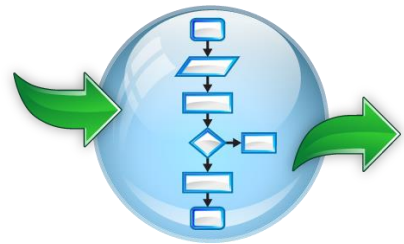


ORACLE

For Instructor Use Only.
This document should not be distributed.

PL/SQL Function Result Cache

- PL/SQL Function Result Cache stores the results of function execution.
- You can cache a function result by including a `RESULT CACHE` clause in function definition.
- Good candidates for result caching are frequently invoked functions that depend on relatively static data.



ORACLE

The PL/SQL function result cache is a subset of the server result cache that stores function result sets.

PL/SQL function code can include a request to cache its results. Upon invocation of this function, the system checks the cache. If the cache contains the result from a previous function call with the same parameter values, then the system returns the cached result to the invoker and does not execute the function body again. If the cache does not contain the result, then the system executes the function body and adds the result (for these parameter values) to the cache before returning control to the invoker.

For Instructor Use Only.
This document should not be distributed.

Marking PL/SQL Function Results to Be Cached

- Scenario:
 - You need a PL/SQL function that derives a complex metric.
 - The data that your function calculates changes slowly, but the function is frequently called.
- Solution:
 - Use the new `RESULT_CACHE` clause in your function definition.
 - You can also have the cache purged when a dependent table experiences a DML operation, by using the `RELIES_ON` clause.

ORACLE

To enable result caching for a function, use the `RESULT_CACHE` clause in your PL/SQL function. Using this clause results in the following:

- If a result-cached function is called, the system checks the cache.
- If the cache contains the result from a previous call to the function with the same parameter values, the system returns the cached result to the caller and doesn't execute the function body again.
- If the cache doesn't contain the result, the system executes the function body and adds the result (for these parameter values) to the cache before returning control to the caller.

The cache can accumulate many results—one result for every unique combination of parameter values with which each result-cached function was called. If the system needs more memory, it ages out (deletes) one or more cached results.

You can specify the database objects that are used to compute a cached result, so that if any of them is updated, the cached result becomes invalid and must be recomputed.

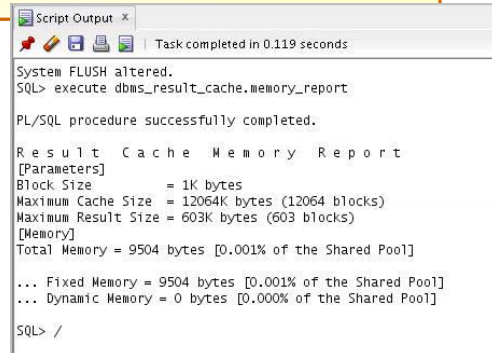
The best candidates for result caching are functions that are called frequently but depend on information that changes infrequently or never.

For Instructor Use Only.
This document should not be distributed.

Clearing the Shared Pool and Result Cache

```
--- flush.sql
--- Start with a clean slate. Flush the cache and shared pool.
--- Verify that memory was released.
SET ECHO ON
SET FEEDBACK 1
SET SERVEROUTPUT ON

execute dbms_result_cache.flush
alter system flush shared_pool
/
execute dbms_result_cache.memory_report
```



Script Output x | Task completed in 0.119 seconds

System FLUSH altered.
SQL> execute dbms_result_cache.memory_report

PL/SQL procedure successfully completed.

Result Cache Memory Report
[Parameters]
Block Size = 1K bytes
Maximum Cache Size = 12064K bytes (12064 blocks)
Maximum Result Size = 603K bytes (603 blocks)
[Memory]
Total Memory = 9504 bytes [0.001% of the Shared Pool]
... Fixed Memory = 9504 bytes [0.001% of the Shared Pool]
... Dynamic Memory = 0 bytes [0.000% of the Shared Pool]
SQL> /

ORACLE

To understand the use of PL/SQL function result caching, clear the shared pool and the result cache again by performing the steps shown in the slide.

For Instructor Use Only.
This document should not be distributed.

Creating a PL/SQL Function by Using the RESULT_CACHE Clause

- Include the `RESULT_CACHE` option in the function definition.
- Optionally, include the `RELIES_ON` clause.

```
CREATE OR REPLACE FUNCTION ORD_COUNT(cust_no number)
RETURN NUMBER
RESULT_CACHE RELIES_ON (orders)
IS
  v_count NUMBER;
BEGIN
  SELECT COUNT(*) INTO v_count
  FROM orders
  WHERE customer_id = cust_no;

  return v_count;
end;
```

Specifies that the
result should be
cached

Specifies the table upon
which the function has a
dependency

ORACLE

When writing code for the PL/SQL result cache option, you need to:

- Include the `RESULT_CACHE` option in the function declaration section of a package
- Include the `RESULT_CACHE` option in the function definition
- Optionally, include the `RELIES_ON` clause to specify tables or views on which the function results depend

In the example shown in the slide, the `ORD_COUNT` function has result caching enabled through the `RESULT_CACHE` option in the function declaration. In this example, the `RELIES_ON` clause is used to identify the `ORDERS` table on which the function results depend.

You can also run `DBMS_RESULT_CACHE.MEMORY_REPORT` to view the result cache memory results.

For Instructor Use Only.
This document should not be distributed.

Calling the PL/SQL Function Inside a Query

```
select cust_last_name, ord_count(customer_id) no_of_orders
   from customers
  where cust_last_name = 'MacGraw'
```

CUST_LAST_NAME	NO_OF_ORDERS
MacGraw	3

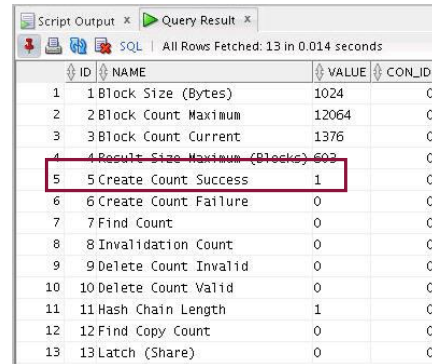
ORACLE

Call the `ORD_COUNT` PL/SQL function inside a query by performing the query shown in the slide. While defining the function you have defined that the results of function evaluation be cached. The result of the function invoked with a `customer_id` of a customer whose last name is 'MacGraw' is stored in the cache.

For Instructor Use Only.
This document should not be distributed.

Viewing Cache Results Created

```
col name format a55  
select * from v$result_cache_statistics  
/
```



ID	NAME	VALUE	CON_ID
1	1 Block Size (Bytes)	1024	0
2	2 Block Count Maximum	12064	0
3	3 Block Count Current	1376	0
4	4 Result Size Maximum (Blocks)	603	0
5	5 Create Count Success	1	0
6	6 Create Count Failure	0	0
7	7 Find Count	0	0
8	8 Invalidation Count	0	0
9	9 Delete Count Invalid	0	0
10	10 Delete Count Valid	0	0
11	11 Hash Chain Length	1	0
12	12 Find Copy Count	0	0
13	13 Latch (Share)	0	0

ORACLE

You can query `V$RESULT_CACHE_STATISTICS` to view the memory allocation and usage statistics. Do this by performing the steps shown in the slide.

For Instructor Use Only.
This document should not be distributed.

Calling the PL/SQL Function Again

```
select cust_last_name, ord_count(customer_id) no_of_orders
from customers
where cust_last_name = 'MacGraw'
```

CUST_LAST_NAME	NO_OF_ORDERS
MacGraw	3

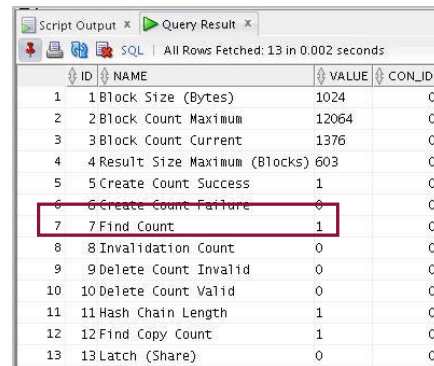
ORACLE

Run the query again, as shown in the slide.

For Instructor Use Only.
This document should not be distributed.

Viewing Cache Results Found

```
col name format a55
select * from v$result_cache_statistics
/
```



ID	NAME	VALUE	CON_ID
1	1 Block Size (Bytes)	1024	0
2	2 Block Count Maximum	12064	0
3	3 Block Count Current	1376	0
4	4 Result Size Maximum (Blocks)	603	0
5	5 Create Count Success	1	0
6	6 Create Count Failure	0	0
7	7 Find Count	1	0
8	8 Invalidation Count	0	0
9	9 Delete Count Invalid	0	0
10	10 Delete Count Valid	0	0
11	11 Hash Chain Length	1	0
12	12 Find Copy Count	1	0
13	13 Latch (Share)	0	0

ORACLE

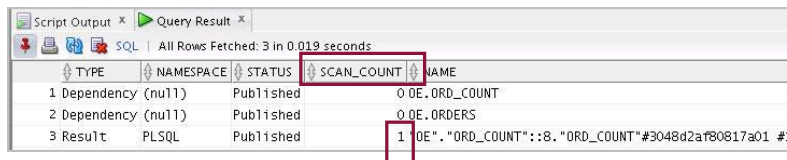
Query `V$RESULT_CACHE_STATISTICS` again to view the memory allocation and usage statistics. Do this by performing the steps shown in the slide.

Note that the `FIND COUNT` statistic now has a value of 1. This is the number of cache results that were successfully found (one for each query statement).

For Instructor Use Only.
This document should not be distributed.

Confirming That the Cached Result Was Used

```
select type, namespace,status, scan_count,name  
from v$result_cache_objects  
/
```



	TYPE	NAMESPACE	STATUS	SCAN_COUNT	NAME
1	Dependency (null)		Published	0	OE.ORD_COUNT
2	Dependency (null)		Published	0	OE.ORDERS
3	Result	PLSQL	Published	1	"OE"."ORD_COUNT"::8."ORD_COUNT"#3048d2af80817a01 #1

ORACLE

Query `V$RESULT_CACHE_OBJECTS` to confirm that the cached result was used, by performing the steps shown in the slide. The `SCAN_COUNT` column indicates the number of times the object was accessed.

Lesson Agenda

- Caching in the Database Instance
- Result Caching
- SQL Result Caching
- PL/SQL Result Caching
- Oracle Database In-Memory



ORACLE

For Instructor Use Only.
This document should not be distributed.

Oracle Database In-Memory

- A dual format architecture of 19c supports both the traditional row format and in-memory column format.
- An In-Memory column store is maintained in the SGA.
- The column format provides high performance for analytical processing.
- You can enable and configure in-memory column store in your database as a system administrator.

ORACLE

Oracle Database 19c has a unique "dual-format" architecture that enables data to be maintained in both the existing Oracle row format, for OLTP operations, and a new purely in-memory columnar format, optimized for analytical processing.

Data is traditionally stored in a row format. In a row format database, each new transaction or record stored in the database is represented as a new row in a table. That row is made up of multiple columns, with each column representing a different attribute about that record. A row format is ideal for online transaction systems, as it allows quick access to all of the columns in a record since all of the data for a given record are kept together in-memory and on-storage.

A column format database stores each of the attributes about a transaction or record in a separate column structure. A column format is ideal for analytics, as it allows for faster data retrieval when only a few columns are selected but the query accesses a large portion of the data set.

Oracle Database In-Memory (Database In-Memory) provides the best of both worlds by allowing data to be simultaneously populated in both an in-memory row format (the buffer cache) and a new in-memory column format: a dual-format architecture.

Note that the dual-format architecture does not double memory requirements.

You can learn more about In-Memory column store here:

<http://www.oracle.com/technetwork/database/in-memory/overview/twp-oracle-database-in-memory-2245633.html>

For Instructor Use Only.
This document should not be distributed.

Quiz

Q

Which of the following statements are true?

- a. When a query is executed, the result cache is built up in the result cache memory.
- b. Subsequent executions of the same query or function can be served directly out of the cache, improving response times.
- c. This technique should not be used for SQL queries and PL/SQL functions that are executed frequently.
- d. Cached query results remain valid even after the database data accessed by the query is modified.

?

ORACLE

Answer: a, b

For Instructor Use Only.
This document should not be distributed.

Quiz

You can set the `RESULT_CACHE_MODE` to `FORCE` at the session level by using the `ALTER SESSION` command, so that the results of all the queries are always stored in the result cache memory.

- a. True
- b. False

ORACLE

Answer: a

Quiz

Q

You can use the `DBMS_RESULT_CACHE` package to:

- a. Bypass the cache
- b. Retrieve statistics on the cache memory usage
- c. Flush the cache
- d. None of the above

?

ORACLE

Answer: a, b, c

For Instructor Use Only.
This document should not be distributed.

Quiz

Q

On querying `V$RESULT_CACHE_STATISTICS` to view the memory allocation and usage statistics, the number of cache results successfully ***found*** is denoted by:

- a. The `CREATE COUNT SUCCESS` statistic
- b. The `FIND COUNT` statistic
- c. The `INVALIDATION COUNT` statistic
- d. The `HASH CHAIN LENGTH` statistic

?

ORACLE

Answer: b

For Instructor Use Only.
This document should not be distributed.

Quiz

Q

You can create a function successfully that has both invoker's rights and is result cached.

- a. True
- b. False

?

ORACLE

Answer: a

It is possible to create a unit that has both invoker's rights (declared `AUTHID CURRENT USER`) and is result cached (with the declaration `RESULT CACHE`).

For Instructor Use Only.
This document should not be distributed.

Summary

In this lesson, you should have learned how to:

- Improve memory usage by caching SQL result sets
- Write queries that use the result cache hint
- Use the `DBMS_RESULT_CACHE` package
- Set up PL/SQL functions to use PL/SQL result caching



ORACLE

In this lesson, you saw the Oracle Database 19c caching techniques that can improve performance.

Practice 9: Overview

This practice covers the following topics:

- Writing code to use SQL caching
- Writing code to use PL/SQL caching

ORACLE

In this practice, you implement SQL query result caching and PL/SQL result function caching. You run scripts to measure the cache memory values, manipulate queries and functions to turn caching on and off, and then examine cache statistics. This practice uses the `OE` schema.

For Instructor Use Only.
This document should not be distributed.